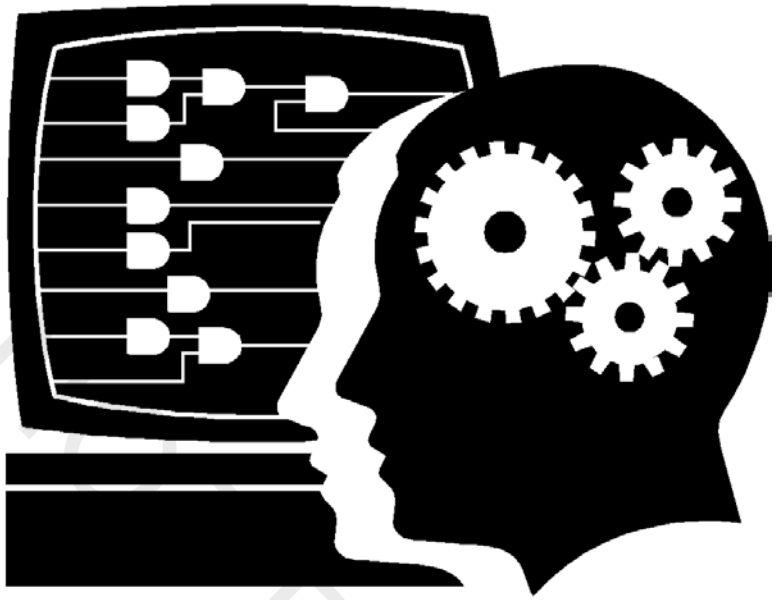


Customer Training

The Quartus Prime Software: Foundation

A-MNL-QP-F-15-1-v1

http://www.altera.com/customertraining/ILT/P-CSTN-QP-F-15-1-v1.zip

Table of Contents The Quartus Prime Software: Foundation	Page Numbers
Objectives	1
Class Agenda	2
Edition Comparison and Migration Paths	4
Quartus Prime Feature Overview	5
Quartus Prime Projects	13
New Project Wizard	14
Project Navigator	19
Quartus Prime Project Files	20
Project Management	22
IP Management	22
Design Methodology	30
Design Entry Methods	32
Creating New Files	33
Verilog & VHDL	34
Altera IP Cores	36
Qsys	47
State Machine Editor	47
Memory Editor	48
Other System Design Entry Tools	52
Compilation	57
Compilation Design Flows	60
Message Window	63
Compilation Report	66
Netlist Viewers	69
Chip Planner	74
Settings & Assignments	83
Settings	84
Assignments (Logic Options)	88
Design Assistant	95
Optimization Advisors	95
I/O Planning	98
Pin Planner	100
BluePrint Platform Designer	111
A Taste of Programming & Configuration	118
Class Summary	122
Altera Technical Support	124
Device Programming Appendix	125

The Quartus Prime Software: Foundation

Course Objectives

- Create a new Quartus® Prime project
- Choose supported design entry methods
- Compile a design into a programmable logic device (PLD)
- Locate resulting compilation information
- Create design constraints (assignments & settings)
- Manage I/O assignments
- Prepare for programming/configuring a PLD

Class Schedule

Start	End	Topic
8:30	9:00	Introduction and Feature Overview
9:00	9:45	Project & IP Management
9:45	10:00	Lab 1: New Project Wizard
10:00	11:15	Design Methodology & Design Entry
11:15	11:45	Lab 2: Design Entry
11:45	12:30	Lunch
12:30	1:30	Compilation
1:30	2:00	Lab 3: Compilation
2:00	2:45	Settings & Assignments
2:45	3:15	Lab 4: Settings & Assignments
3:15	4:30	I/O Planning & Taste of Programming/Config.
4:30	5:00	Lab 5: I/O Planning

= Fixed start/stop times

3

The Quartus Prime Software: Foundation

Introduction to Altera® Devices and Tools

Product Portfolio Overview

CPLDs/FPGAs	FPGAs/SoCs	FPGAs/SoCs	FPGAs/SoCs	Power SoCs
Lowest Cost, Non-volatile	Cost/Power Balance	Mid-range	High Performance High Bandwidth	High-efficiency Power Management

RESOURCES

Embedded Soft and Hard Processors	Design Software	Development Kits	Intellectual Property (IP)
Nios II ARM	Quartus Prime Design Software, SoC EDS, DSP Builder		▪ Industrial ▪ Computing ▪ Enterprise 

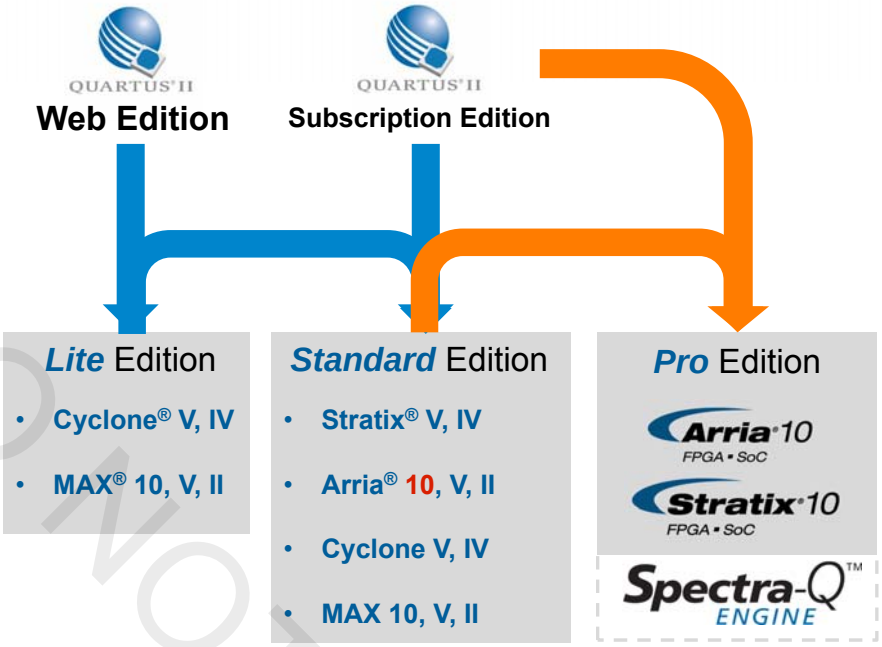
Quartus Prime Software: Three Editions

Web Edition	Lite Edition (LE)
Subscription Edition	Standard Edition (SE)
	Pro Edition (PE)

Feature comparison available on Altera web site

Quartus Prime software download center

Migration Paths



Free online training: *Migrating to the Quartus Prime Pro Edition Software*

Quartus Prime Licensing

- **Web** Edition → **Lite** Edition (LE)
  - No change
  - No license file required
- **Subscription** Edition → **Standard** Edition (SE)
  - No change
  - Previous Subscription Edition license will work with Standard Edition
- **Pro** Edition (PE)
  - Requires a new feature line
- Licensing FAQ: *https://www.altera.com/support/support-resources/download/licensing/q-and-a.html*
- Self-Service Licensing Center: *https://mysupport.altera.com/AlteraLicensing/license/index.html*

The Quartus Prime Software: Foundation

Quartus Prime Design Software Feature Overview

Quartus Prime Design Software

- Fully-integrated development tool
  - Multiple design entry methods
  - Logic synthesis
  - Place & route
  - Device programming
- Simulation
  - Supports standard HDL simulation tools
  - Includes ModelSim®-Altera Starter Edition tool
    - Optional upgrade to ModelSim-Altera Edition tool
  - See comparison
    - https://www.altera.com/products/design-software/model---simulation/modelsim-altera-software.html

Quartus Prime Design Software Features

	Quartus Prime Tool/Feature
Operating system	▪ 64-bit Windows and Linux support
Licensing	▪ Node-locked and network licensing support
Project creation	▪ New Project Wizard
Design entry	▪ Text Editor (HDL support) ▪ Schematic Editor ▪ State Machine Editor ▪ IP Catalog *(replaces MegaWizard Plug-In Manager)* ▪ Qsys system design tool ▪ DSP Builder Standard/Advanced Blockset ▪ OpenCL support ▪ 3rd-party design entry tool support
Constraint (assignment) entry	▪ Assignment Editor ▪ Text Editor support of Synopsys Design Constraints (SDC) ▪ Pin Planner ▪ BluePrint Platform Designer *(Pro Edition only)* ▪ Scripting (Tcl) support
Design processing/compilation (synthesis and fitting)	▪ Quartus Integrated Synthesis (QIS) or Spectra-Q™ Synthesis *(Pro)* ▪ 3rd party EDA synthesis tool support ▪ Quartus Prime Fitter *(Lite & Standard)* ▪ Spectra-Q Hybrid Placer & Router *(Standard & Pro)*
Design evaluation and debugging	▪ RTL Viewer ▪ Technology Map Viewers ▪ State Machine Viewer

11

Quartus Prime Design Software Features (cont.)

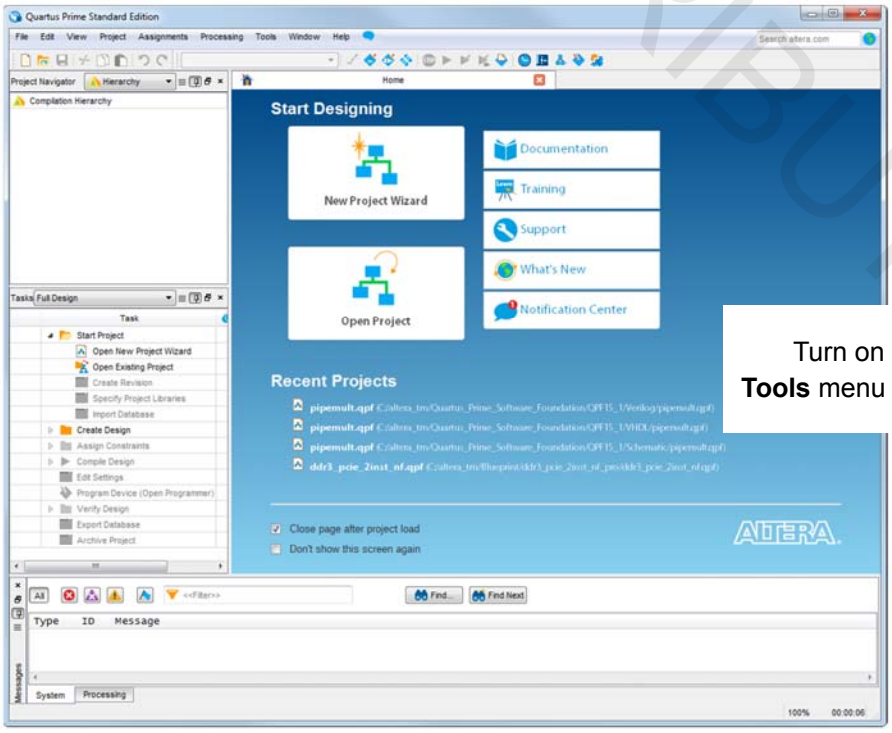
Category	Quartus Prime Tool/Feature	
Power analysis	▪ PowerPlay power analyzer	
Static timing analysis	▪ TimeQuest timing analyzer	
Simulation	▪ ModelSim-Altera Starter Edition ▪ ModelSim-Altera Edition ▪ 3rd party EDA simulation tool support	
Chip layout viewing and modification	▪ Chip Planner ▪ Resource Property Editor	
Programming file generation	▪ Quartus Prime Assembler	
FPGA/CPLD programming	▪ Quartus Prime Programmer	
Hardware debugging tools	▪ SignalTap II embedded logic analyzer ▪ In-System Sources and Probes ▪ SignalProbe incremental routing	▪ System Console ▪ Transceiver Toolkit ▪ In-System Memory Content Editor
Design optimization and productivity improvement	▪ Design Assistant ▪ Rapid Recompile ▪ Quartus Prime incremental compilation ▪ Physical synthesis optimization ▪ Design Space Explorer II (DSE)	

12

Quartus Prime Edition Use in this Training

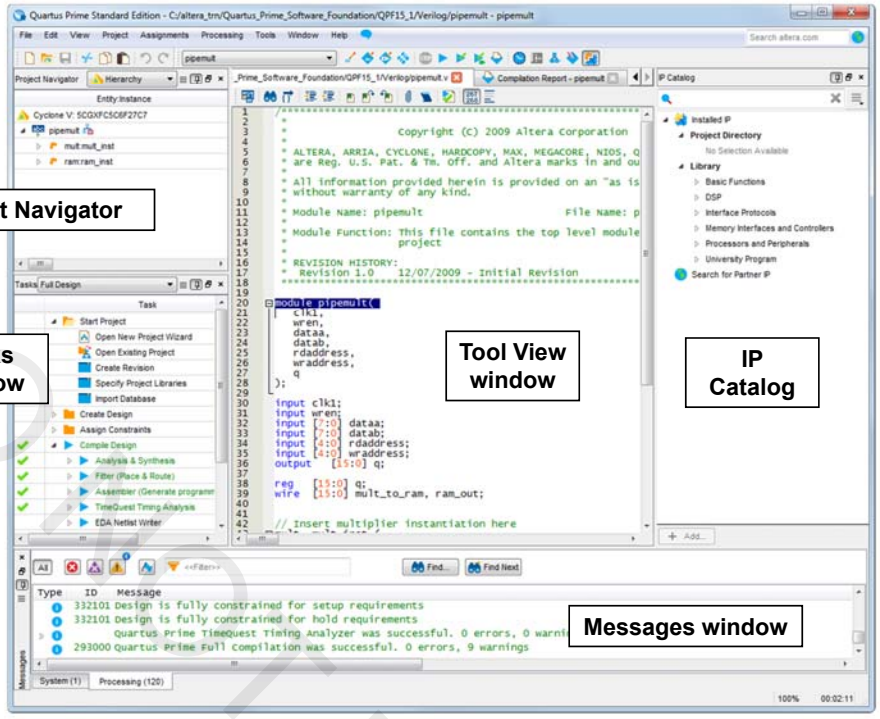
- Training focuses mostly on Standard Edition
  - Tool usage
  - Screen shots
  - Lab exercises
- Most features identical between all 3 editions
- Pro Edition-specific features will be highlighted

Welcome to the Quartus Prime Software!



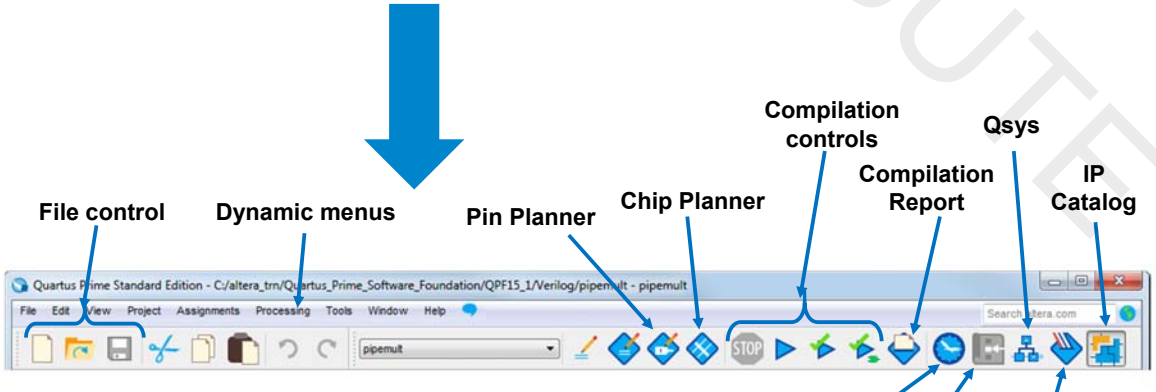
Turn on or off in **Tools** menu → **Options**

Quartus Prime Default Operating Environment



Main Toolbar

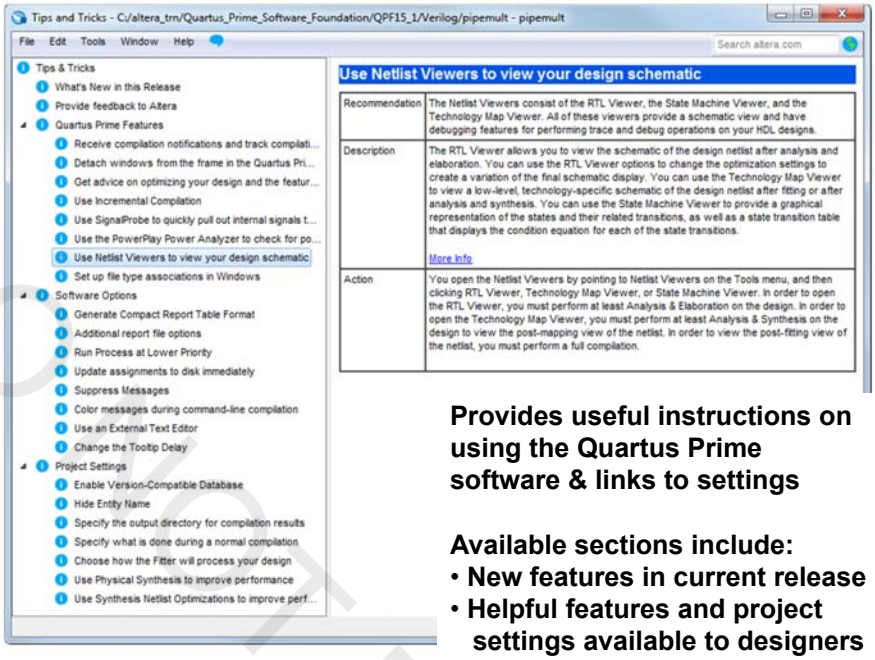
Right-click toolbar and select **Large icons**



To customize toolbars
Tools menu → **Customize…**

Tips & Tricks Advisor

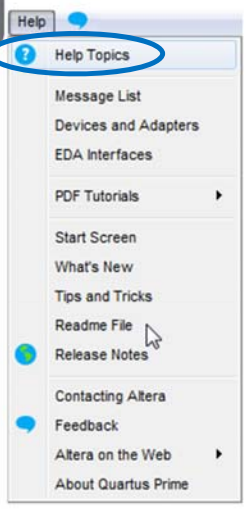
Help menu → **Tips & Tricks**

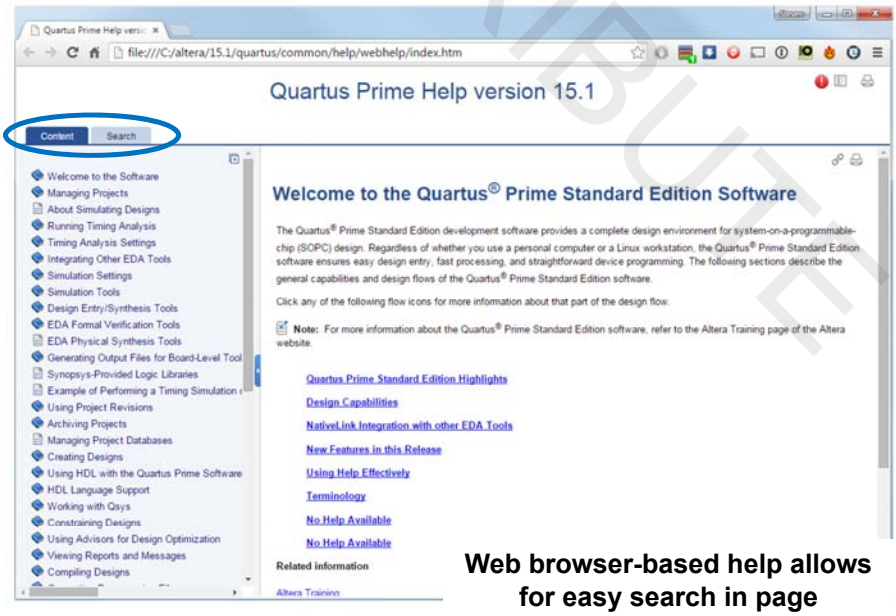


Provides useful instructions on using the Quartus Prime software & links to settings

Available sections include:
- **New features in current release**
- **Helpful features and project settings available to designers**

Built-In Help System



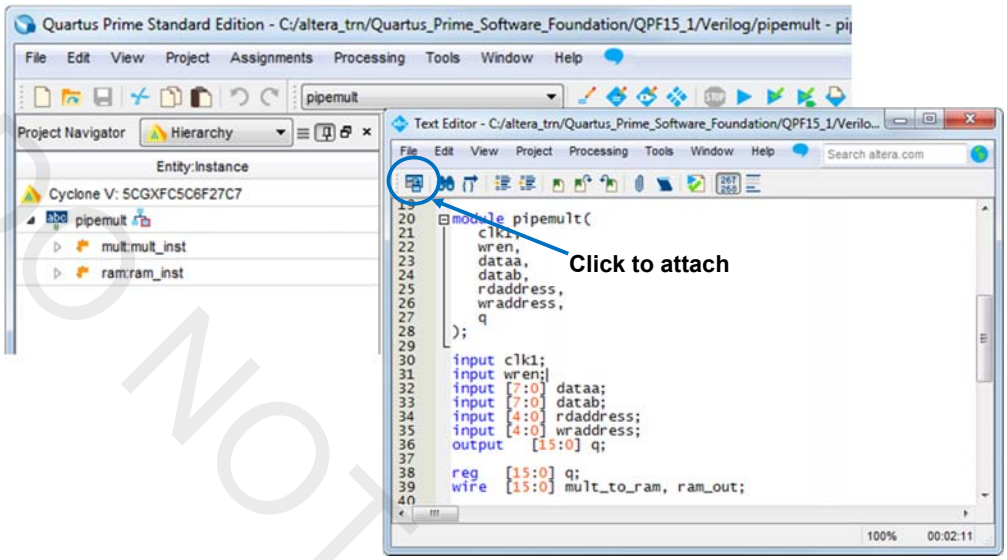


Web browser-based help allows for easy search in page

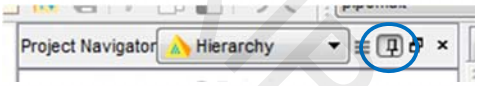
*Note: Bug in 15.1 causes issues with Help in Chrome. Workaround is to click **OK** in JavaScript dialog that appears, then change URL to **index_frames.htm** (instead of **.html**).*

Detachable Windows

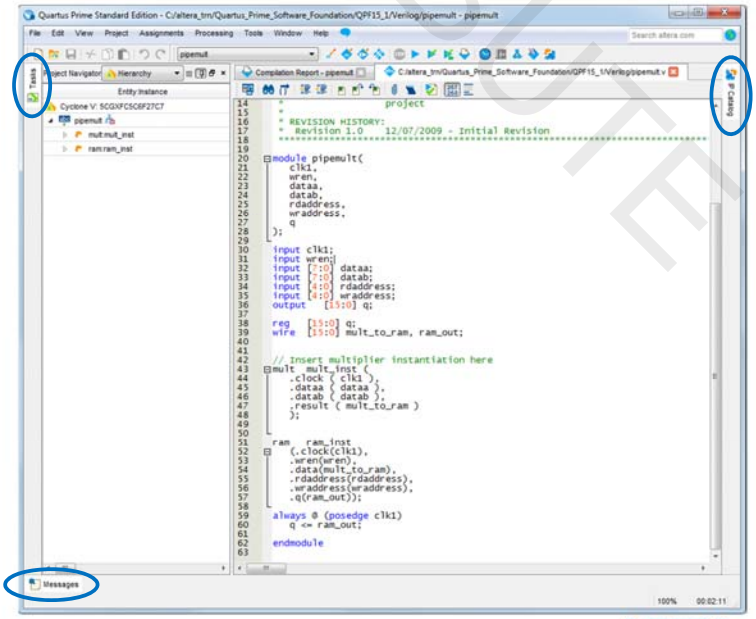
- Separate child windows from the Quartus Prime GUI frame (**Window** menu → **Detach/Attach Window**)



Auto Hide/Show Dockable Windows



- Quickly hide/unhide dockable windows by “unpinning”
- Hover over a window’s tab to make it temporarily visible
- Re-pin to keep the window visible

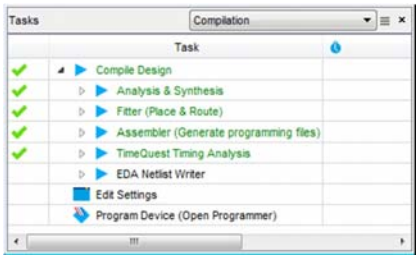


Tasks Window

- Easy access to most Quartus Prime functions
- Organized into related tasks within task flows

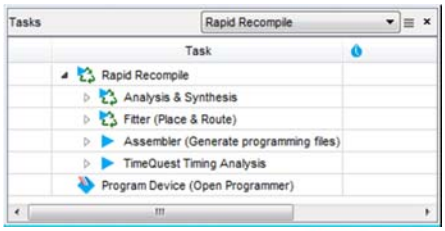
Full Design Flow
Perform all project tasks

Compilation Flow
Focus on compilation tasks



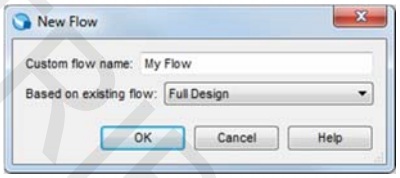
Double-click any task to run

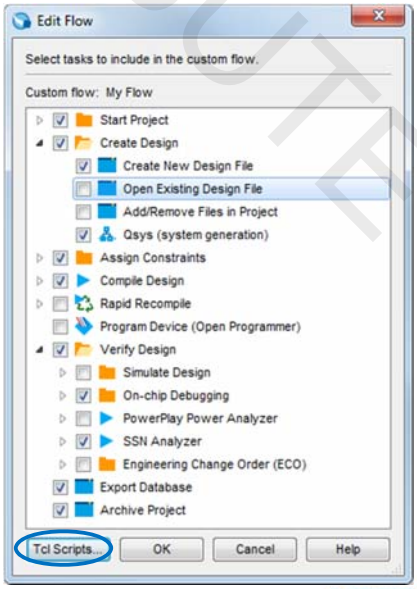
Rapid Recompile
Faster compile iterations

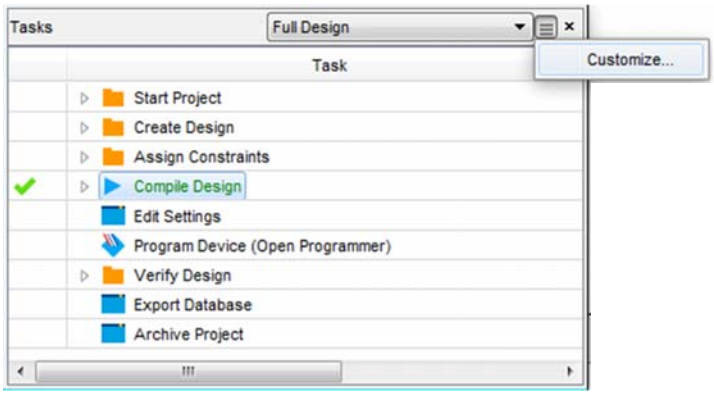


Custom Task Flow

- Create brand new flows or flows based off of existing ones
- Add Tcl scripts to flows for quick access



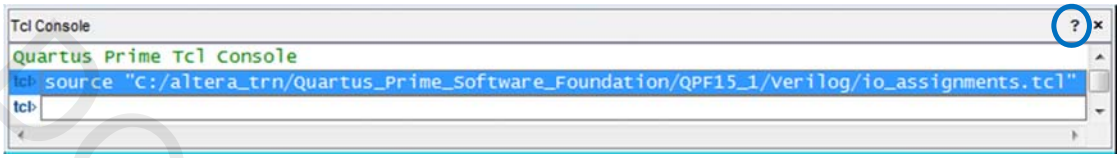




Tcl Console Window

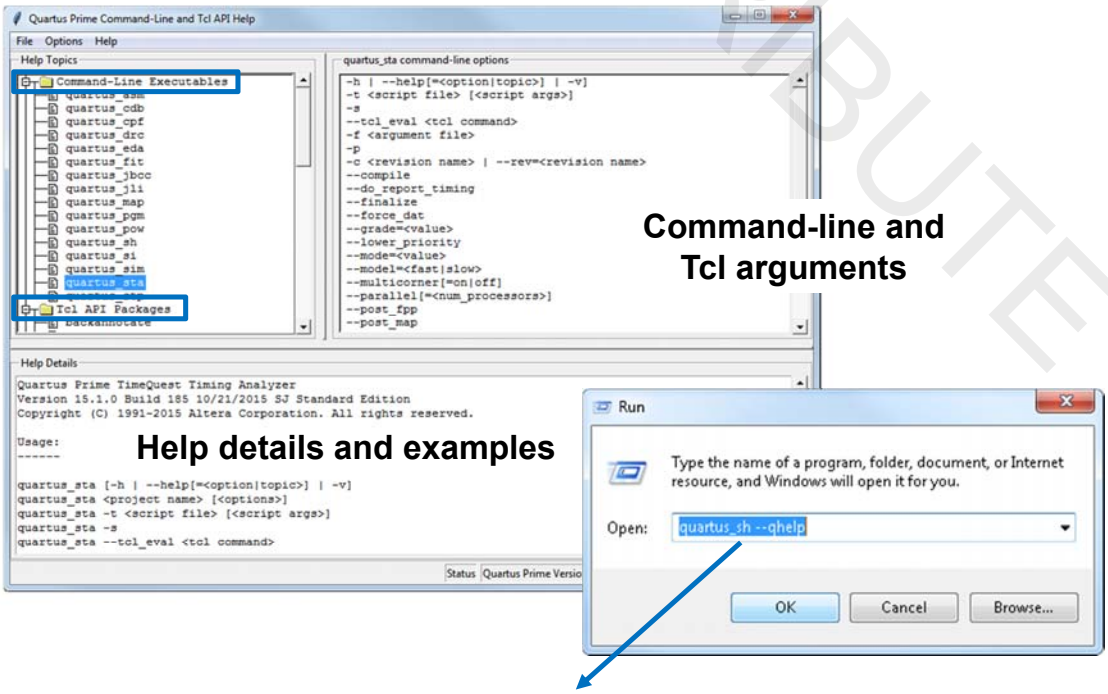
- Enter and execute Tcl commands directly in the GUI

View menu → **Utility Windows** → **Tcl Console**



- Execute from command-line using Tcl shell
  - `quartus_sh -shell`
- Run complete scripts from **Tools** menu → **Tcl Scripts…**

Tcl and Command Line Help



C:\<*install dir*>\quartus\bin64\quartus_sh --qhelp

Further Information

*Quartus Prime Handbook always available online in pdf format at (**Documentation** link at top of all altera.com web pages):*

Standard: https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/hb/qts/qts-qps-handbook.pdf

Pro: https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/hb/qts/qts-qpp-handbook.pdf

- Quartus Prime Handbook Volume 2 chapters
  - Command-Line Scripting
  - Tcl Scripting
- Free on-line training classes
  - Introduction to Tcl
    - http://www.altera.com/education/training/courses/ODSW1180
  - Quartus Prime Software Tcl Scripting
    - http://www.altera.com/education/training/courses/ODSW1190
- Tcl references on-line
  - Good reference: http://tmml.sourceforge.net/doc/tcl/

The Quartus Prime Software: Foundation

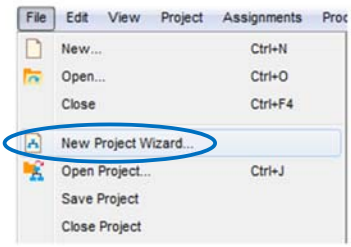
Quartus Prime Projects

Quartus Prime Projects

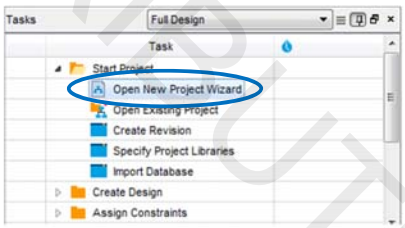
- Description
  - Collection of related design files & libraries
  - Must have a designated top-level entity
  - Target a single device
  - Store settings in Quartus Prime settings file (**.qsf**)
  - Compiled netlist information stored in **db** folder in project directory
- Create new projects with **New Project Wizard**
  - Can be created using Tcl scripts

New Project Wizard

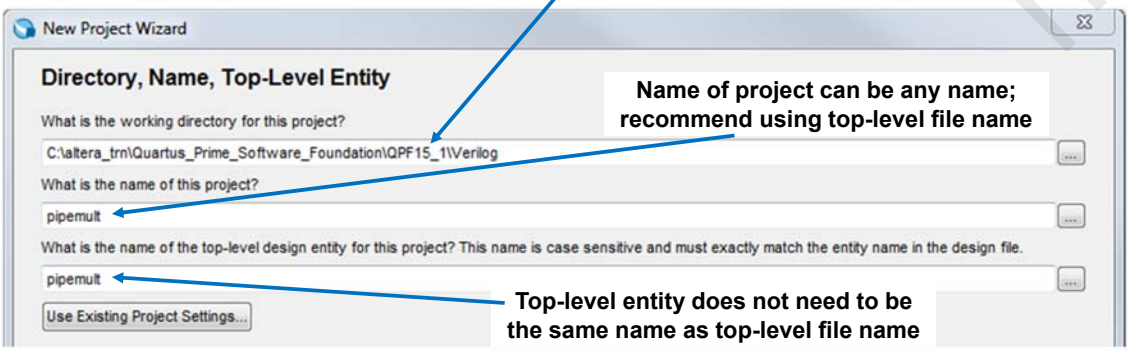
File menu



Tasks



Select working directory



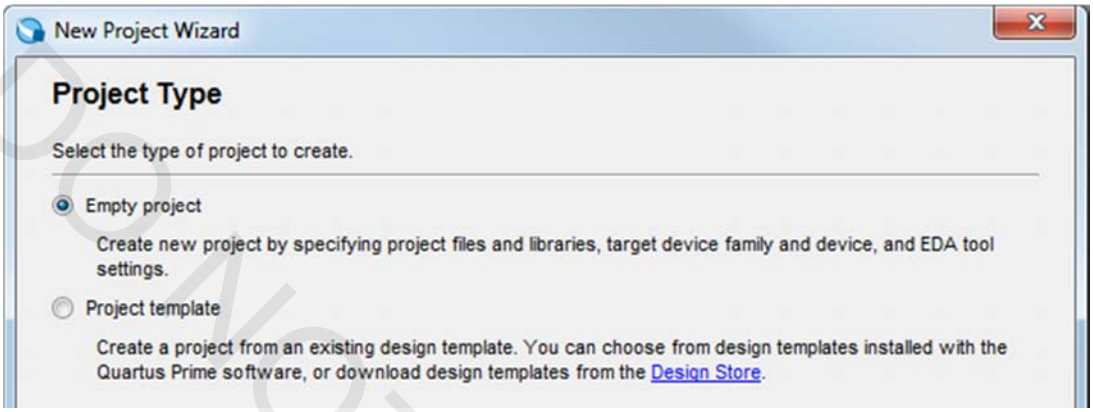
Name of project can be any name; recommend using top-level file name

Top-level entity does not need to be the same name as top-level file name

Tcl: project_new <project_name>

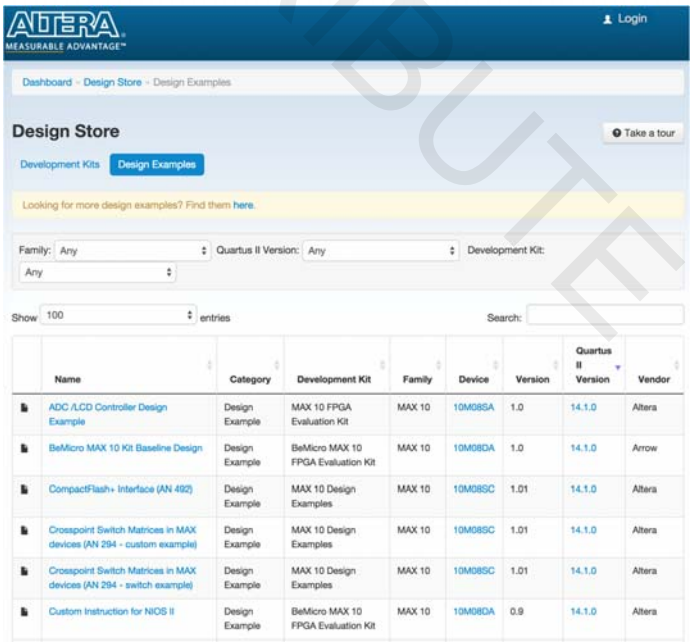
Project Type

- Create a blank project or use templates from Altera Design Store
  - https://cloud.altera.com/devstore/



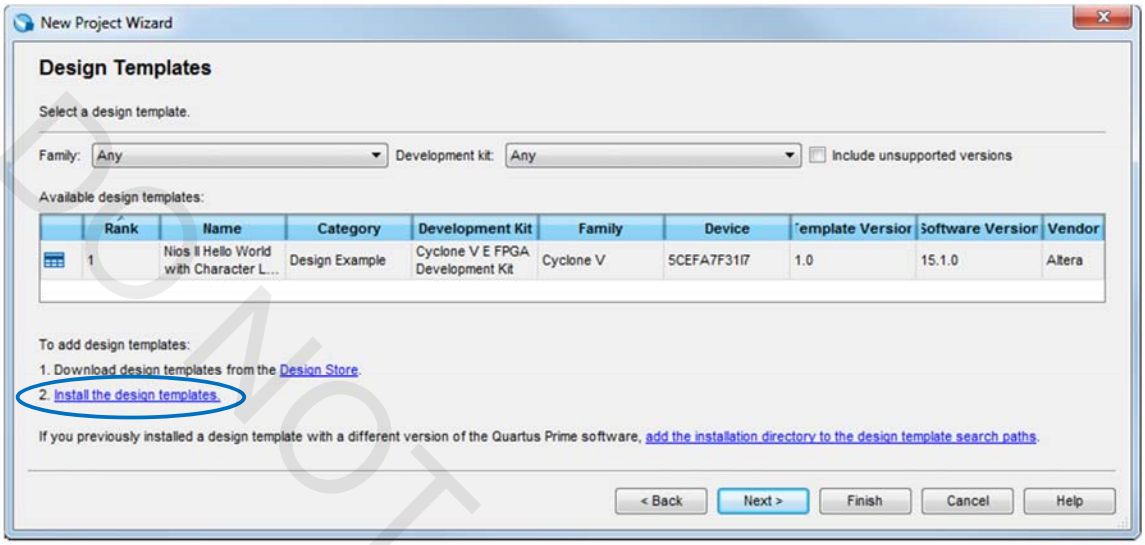
Altera Design Store

- Download complete example design templates for specific development kits
- Design examples include design files, device programming files, and software code as required
- Install **.par** files and select as template in New Project Wizard



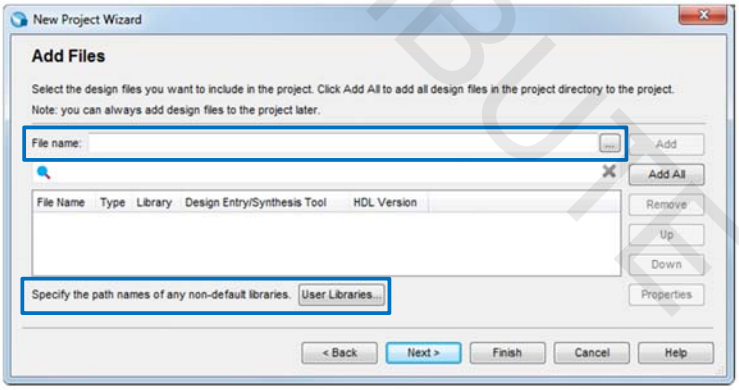
Install and Select Design Templates

- **devplatforms** folder created at **.par** installation location
- Templates available for all future project creation



Add Files

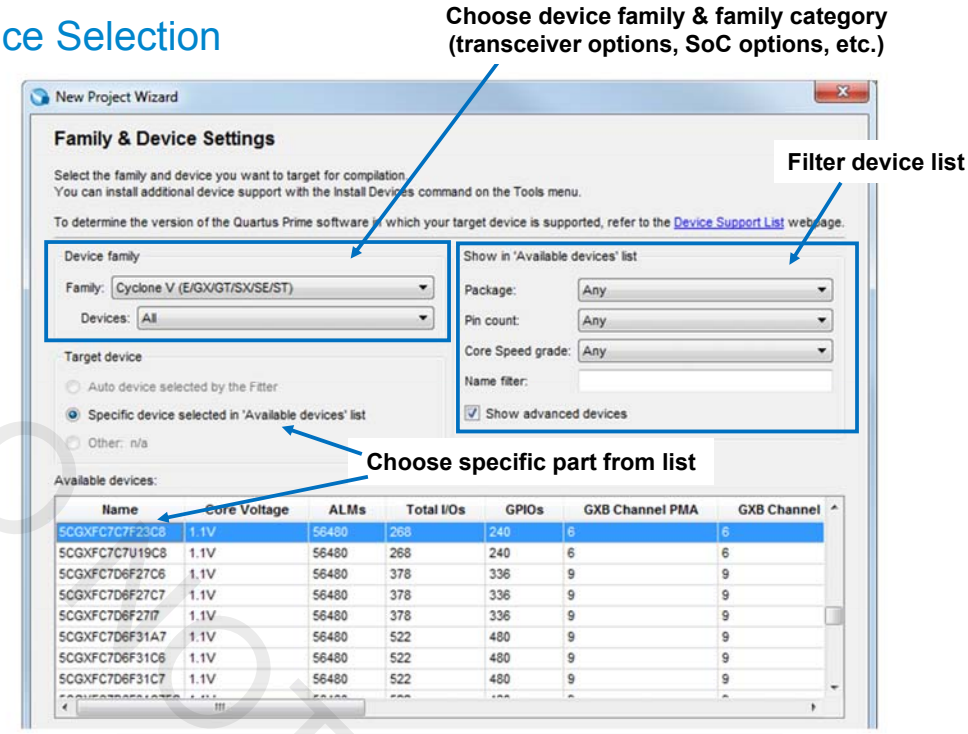
- Add design files
  - Graphic
  - VHDL
  - Verilog
  - SystemVerilog
  - EDIF
  - VQM



- Add library paths
  - User libraries
  - MegaCore® library
  - AMPP™ library
  - Pre-compiled VHDL packages

Tcl: set_global_assignment –name VHDL_FILE <filename.vhd>
Tcl: set_global_assignment –name USER_LIBRARIES <library_path_name>

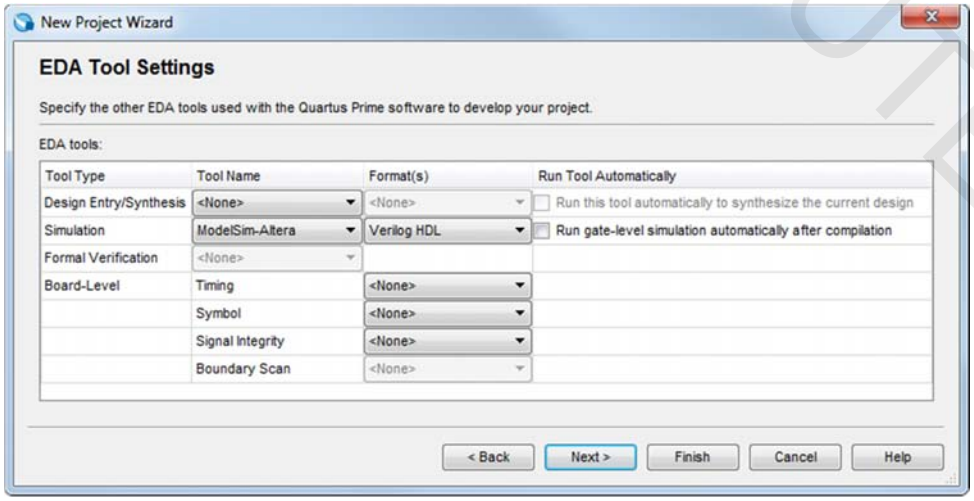
Device Selection



Tcl: set_global_assignment –name FAMILY "device family name"
Tcl: set_global_assignment –name DEVICE <part_number>

EDA Tool Settings

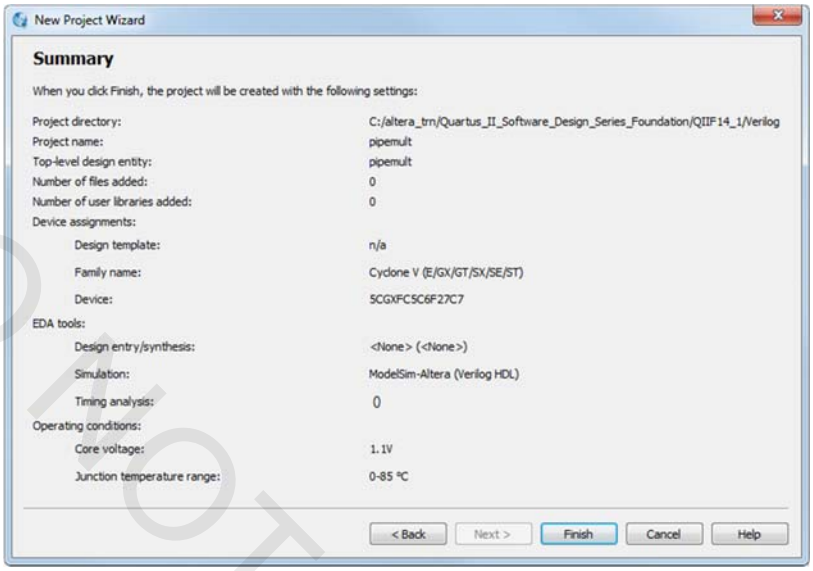
- Choose EDA tools and file formats
- Settings can be changed or added later



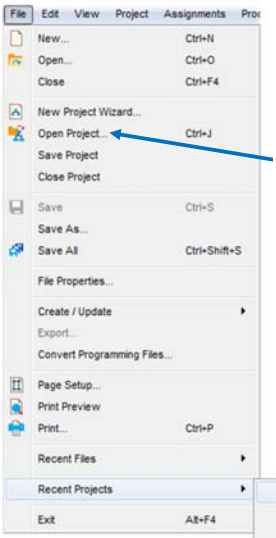
See handbook for Tcl command format

Done!

- Review results & click **Finish** when done



Opening an Existing Project

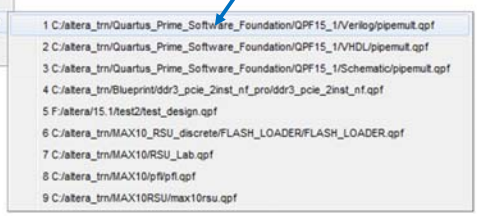


From File menu

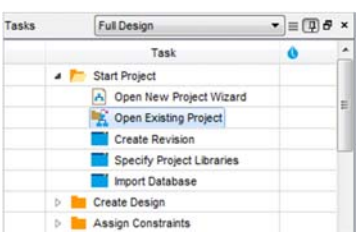
Home window

Double-click .qpf file

Select from recent projects



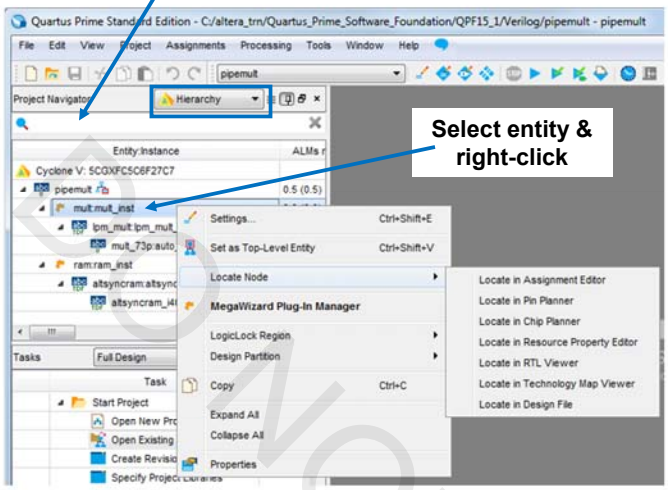
From Tasks window



Tcl: project_open <project_name>

Project Navigator: **Hierarchy**

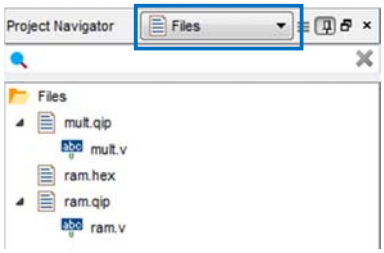
Filter project hierarchy (click ≡ to enable)



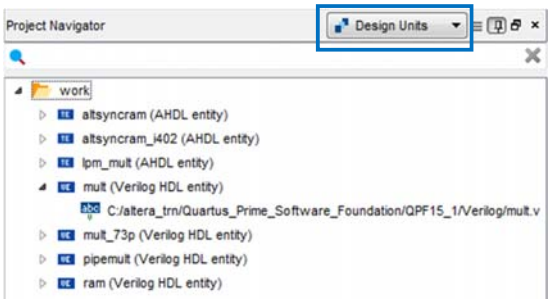
- Displays project hierarchy after project is analyzed
- Uses
  - Set top-level entity
  - Set incremental design partition
  - Make entity-level assignments
  - View resource usage
  - Locate in design file or viewers/floorplans (cross-probing)

Full compilation or
Processing menu → **Start** → **Start Hierarchy Elaboration**

Project Navigator: **Files** & **Design Units**



- **Files**
  - Shows files explicitly added to project
    - Open files
    - Remove files from project
    - Set new top-level entity
    - Specify VHDL library
    - Select file-specific synthesis tool
  - Can also use **Project** menu → **Add/Remove Files in Project…**



- **Design Units**
  - Displays design unit & type
    - VHDL entity and architecture
    - Verilog module
    - AHDL (Altera HDL) subdesign
    - Block diagram filename
  - Expanded unit displays file which instantiates design unit

Quartus Prime Project Files & Folders

- Quartus Prime Project File (**.qpf**)
- Quartus Prime Defaults File (**.qdf**)
- Quartus Prime Settings File (**.qsf**)
- Synopsys® Design Constraints (**.sdc**)
  - Contains timing constraints
  - Learn about SDC and using the TimeQuest timing analyzer
    - http://www.altera.com/education/training/courses/ODSW1115
- **db** folder
  - Contains compiled design information
  - May also see **incremental_db** for incremental compilation information
- **output_files** folder (customize location/name in project settings)
  - Generated compilation report files
  - Programming files generated by the Quartus Prime Assembler

Project & Defaults Files

- Quartus Prime Project File (**.qpf**)
  - Quartus Prime version
  - Time stamp
  - Active revision(s)
    - *Discussed in a moment*

fir_filter.qpf

```
QUARTUS_VERSION = "15.1"
DATE = "13:57:59  November 12, 2015"

# Revisions

PROJECT_REVISION = “filtref”
PROJECT_REVISION = “filtref_new”
```

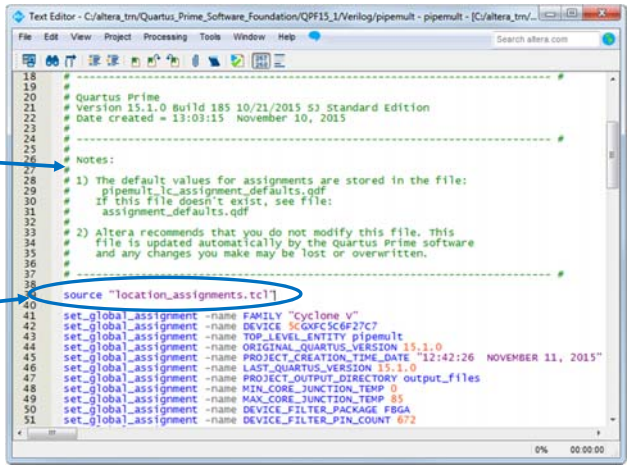
- Quartus Prime Defaults Files (**.qdf**)
  - Stores Quartus Prime project settings & assignment defaults
  - Example names: *assignment_defaults.qdf* or *<revision_name>_assignment_defaults.qdf*
  - Found in local project or *altera\<version>\quartus\bin64* directory

Quartus Prime Settings File (**.qsf**)

- Stores all settings & assignments *except* timing
- Uses Tcl syntax
- Can be edited manually by user

User comments start with #

Source other Tcl/.qsf files to organize assignments



*See the **Quartus Settings File Reference Manual** (https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/manual/mnl_qsf_reference.pdf) for more details on QSF assignments & syntax*

Constraint Files & Assignment Priority

1. **.qsf**
   - Highest priority
   - Assignments always used from here first
2. Revision-specific **.qdf** file located in project directory
   - *<revision_name>_assignment_defaults.qdf*
   - Created automatically in the project directory when a revision is opened in another version of the Quartus Prime software
3. **.qdf** located in project directory
   - *assignment_defaults.qdf*
   - Created automatically in project directory when project archived & restored
4. **.qdf** located in Quartus Prime **bin64** directory
   - Lowest priority
   - Assignments only used if not found in higher priority files

Project & IP Management

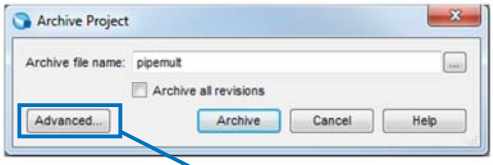
- Project archive & restore
- Project copy
- Revisions
- Project clean
- IP management tools

Project Archive

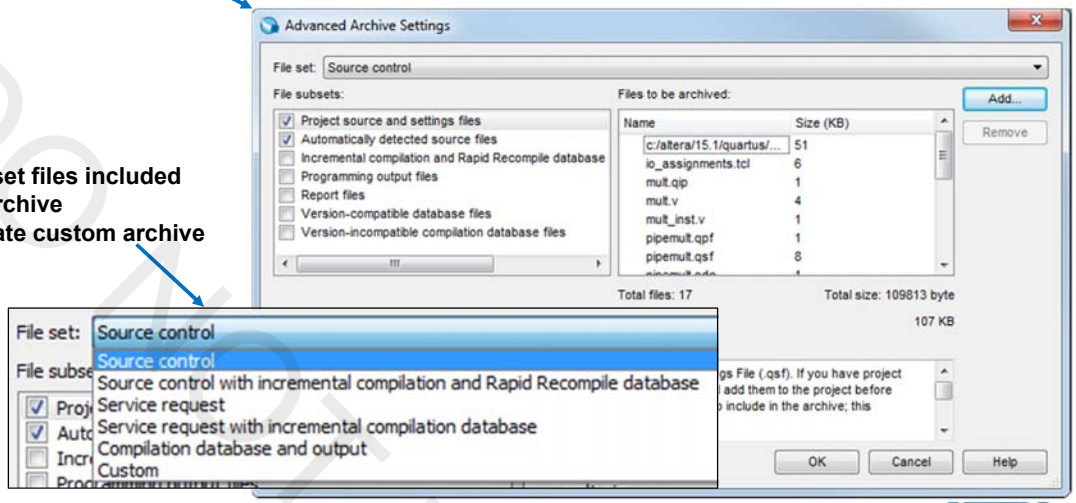
- Creates 2 files
  - Compressed Quartus Prime Archive File (**.qar**)
    - Includes design files, **.qpf** file, & **.qsf** file(s)
    - Option to include databases
    - Creates local **.qdf** file for archive
  - Archive activity log (**.qarlog**)
- Example uses
  - File storage (version control)
  - Project handoff: useful for sending to Altera support

Tcl: project_archive <project_name>

Project Archive (cont.)



Project menu or Tasks window

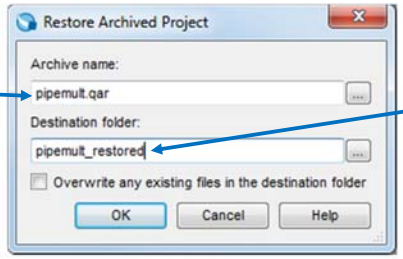


- **Preset files included in archive**
- **Create custom archive**

Restore Archived Project

- Decompresses **.qar** into specified directory
- Paths/directory structures to referenced files/libraries *outside* project directory must also be restored
  - Recreated in restore location based on nearest common *parent directory*
  - Example of referenced file paths in restored project destination:
    - *<destination folder>***/drive/C/***<entire path to all files in project directory>*
    - *<destination folder>***/drive/H/***<path to file(s) referenced on original H drive>*

Project menu



Archive file name

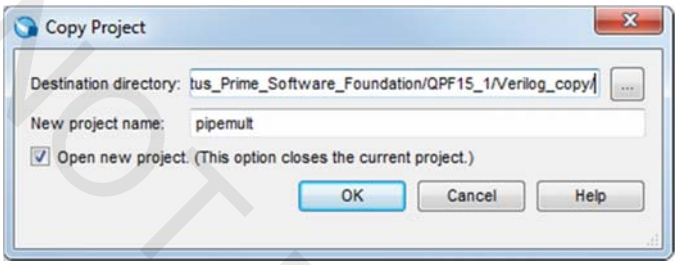
Destination of restored project

Tcl: project_restore <archive_file>

Project Copy

- Copy & save *exact* duplicate of project in new directory
  - Project file (**.qpf**)
  - Design files
  - Settings files
- Example use: duplicating work before editing design files
- User libraries are ***not*** copied; check paths
- New local **.qdf** not created; only copies **.qdf** if it exists

Project menu



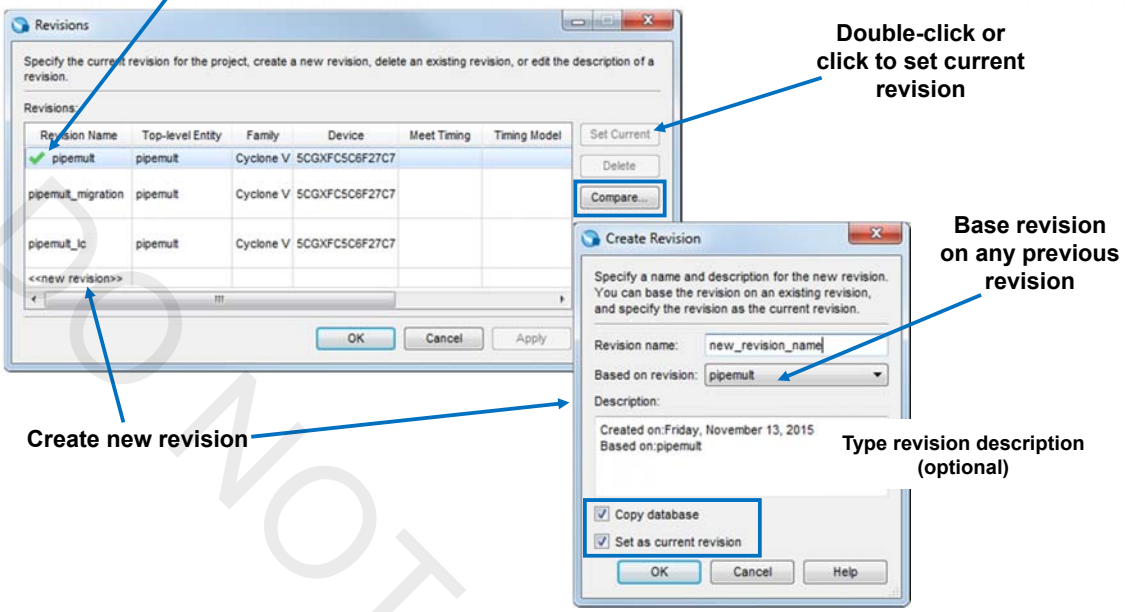
Revisions

- Explore new sets of constraints
- Compile options without losing previous work
- Compare results between revisions
- Revision-specific project files generated and stored in **db** directory
  - Copy and update current revision files (no recompile required)
  - Generate new ones when new revision created and compiled

Creating a Revision

Current revision

Project menu → **Revisions**
or **Tasks window**



Double-click or click to set current revision

Base revision on any previous revision

Create new revision

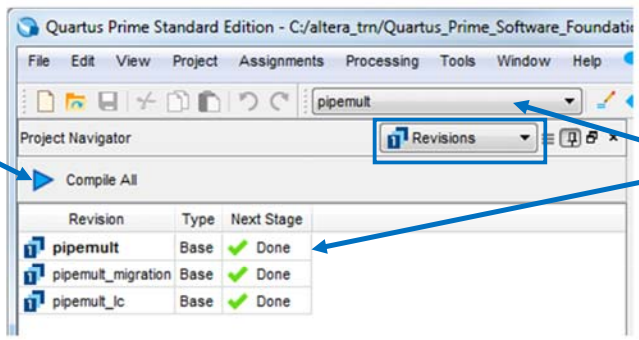
Type revision description (optional)

Tcl: create_revision <revision_name>

Project Revision Support

- **.qsf** created for each revision
  - *<revision_name>.qsf*
- Active revision names stored in **.qpf**
- Text file created for each revision (from description field)
  - *<revision_name>_description.txt*

Compile all revisions or right-click to compile individual revision



Quickly switch between revisions

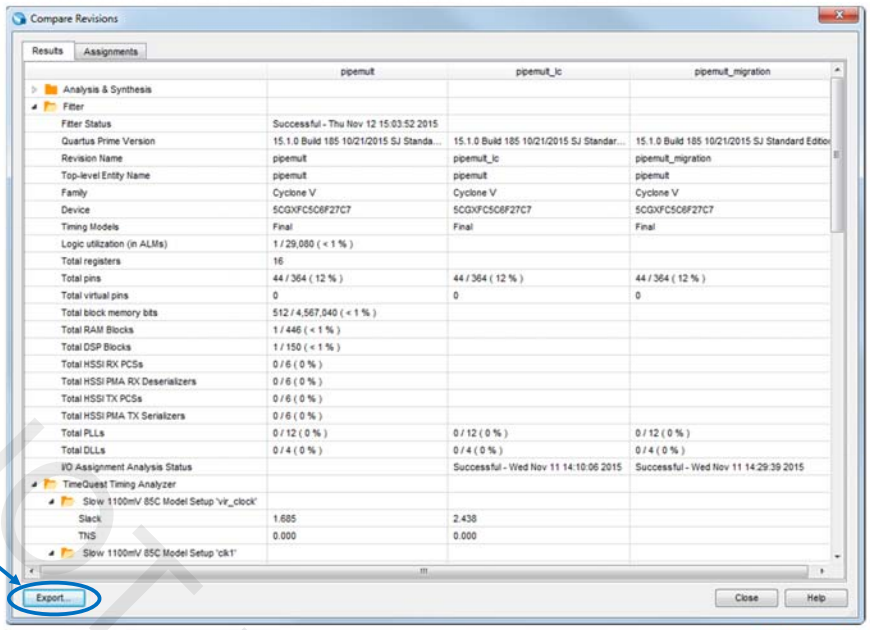
Tcl: project_open –revision <revision_name> <project_name>
Tcl: set_current_revision <revision_name>

Compare Revisions

- Click **Compare…** from the Revisions dialog box

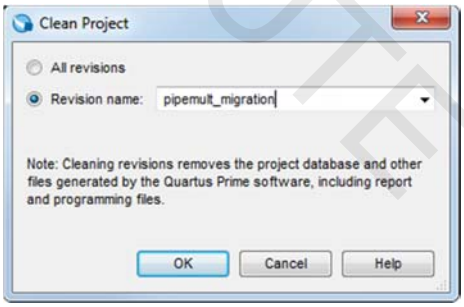
Detailed summary of revision assignments and results

Export to .csv file



Project Clean

- Cleans databases and output files generated for a given revision (or revisions)
  - Removes revision files in project subfolders
  - Removes reports, programming files, and other compiler-generated output
- Select **Clean Project** from the **Project** menu
  - Select from available revisions
  - Clean multiple revisions using wildcards



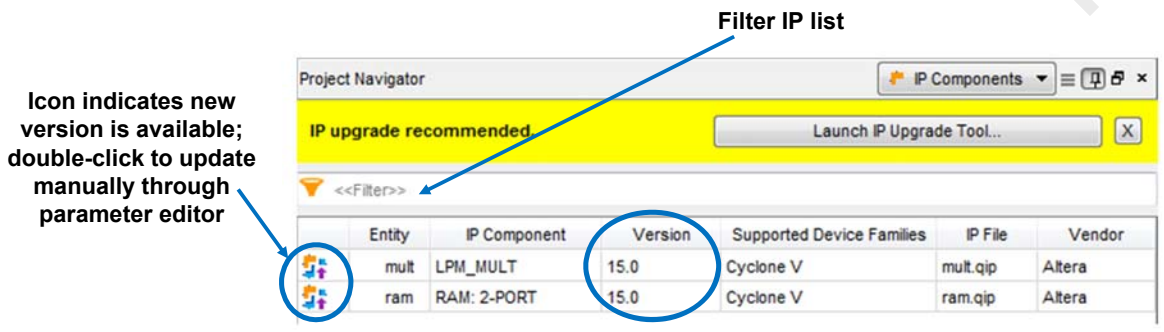
Tcl: project_clean –revision <revision_name> <project_name>

IP Management Tools

- Keep track of project IP cores
- Flag out-of-date IP cores
- Upgrade IP cores to latest version

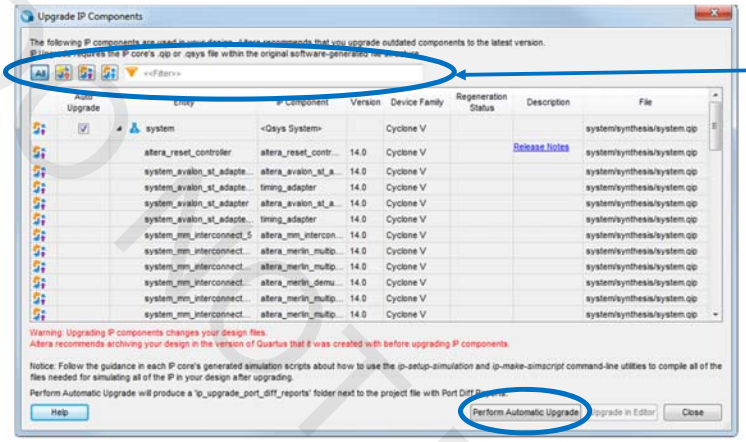
Project Navigator: **IP Components**

- View all IP in project
  - Component name/file, Quartus software version, vendor
- View if new version of IP available
  - Required upgrade
  - Optional upgrade
  - IP is up to date
- Right-click to upgrade or edit individual IP



Upgrading IP Components

- Software detects out-of-date IP when opening existing project
  - Manually open from **Project** menu, Project Navigator, or IP right-click
  - Displays required and optional upgrades
- Select IP block(s) and click **Perform Automatic Upgrade** if supported
  - Qsys systems and components must be regenerated in Qsys



Filter by name or upgrade status (Required or optional upgrade)

Test Your Knowledge: Quartus Prime Projects

1. Which of the following basic Quartus Prime project files is *not* created by the New Project Wizard?
   a. Quartus Prime Project File (**.qpf**)
   b. Quartus Prime Defaults File (**.qdf**)
   c. Quartus Prime Settings File (**.qsf**)
   d. Synopsys Design Constraints (**.sdc**)
   e. b & d
   f. a & c

2. Which of the basic project files is unique for each revision?
   a. Quartus Prime Project File (**.qpf**)
   b. Quartus Prime Defaults File (**.qdf**)
   c. Quartus Prime Settings File (**.qsf**)
   d. Synopsys Design Constraints (**.sdc**)

Exercise 1

Projects Summary

- Projects necessary for design processing
- Use New Project Wizard to create new projects
- Use Project Navigator to study file & entity relationships within project
- Project archive, copy, revisions, and clean provide easy-to-use project management

Project Support Resources

- *Managing Quartus Prime Projects* chapter in Volume 1 of the Quartus Prime Handbook

Quartus Prime Handbook always available online in pdf format at (*Documentation* link from any page on the web site):

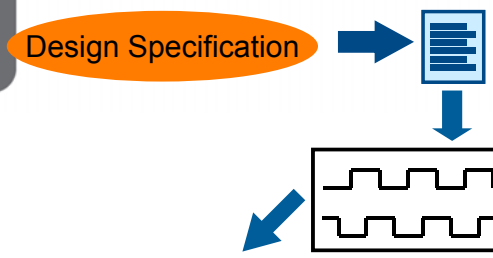
https://www.altera.com/support/literature/lit-index.html

The Quartus Prime Software: Foundation

Design Methodology

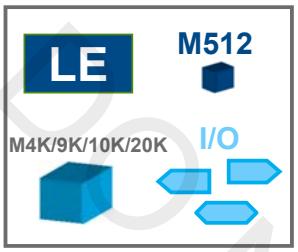
Typical PLD Design Flow

Design Specification



Schematic entry/RTL coding/Qsys
- Behavioral or structural description of design

RTL simulation
- Functional simulation
- Verify logic model & data flow

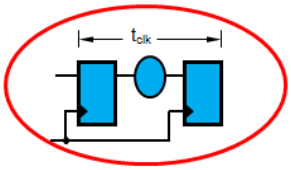


Synthesis (Mapping)
- Translate design into device specific primitives
- Optimization to meet required area & performance constraints
- Quartus Prime synthesis or 3rd party synthesis tools
- *Result*: **Post-synthesis netlist**

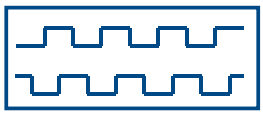
Place & route (Fitting)
- Map primitives to specific locations inside target technology with reference to area & performance constraints
- Specify routing resources to be used
- Quartus Prime Fitter or Spectra-Q Hybrid Placer & Router
- *Result:* **Post-fit netlist**

Quartus Prime Design Software

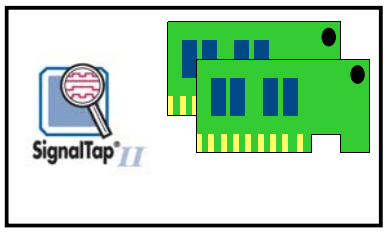
Typical PLD Design Flow



Timing analysis (TimeQuest Timing Analyzer)
- Verify performance specifications were met
- Static timing analysis



Gate level simulation (optional)*
- Simulation with timing delays taken into account
- Verify design will work in target technology



PC board simulation & test
- Simulate board design
- Program & test device on board
- Use **SignalTap™ II** Logic Analyzer or other on-chip tools for debugging

** Not supported in 20-nm and newer devices*

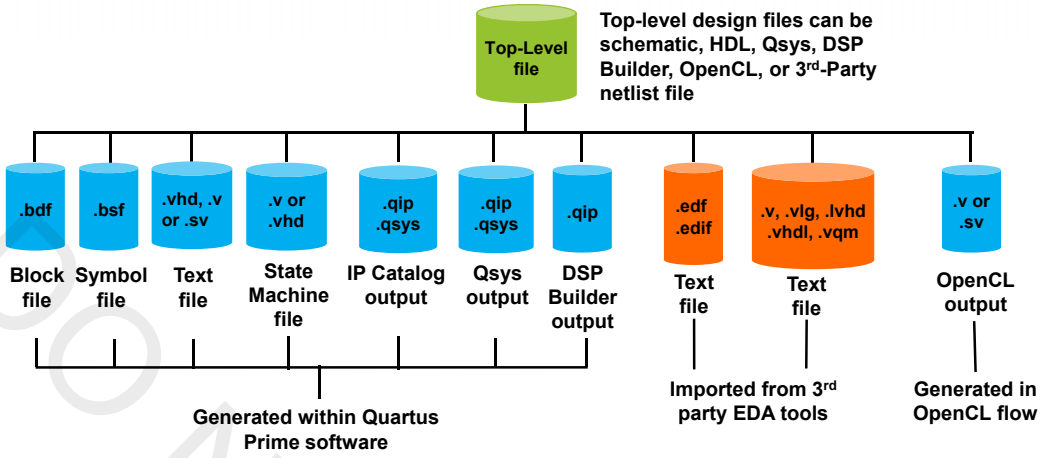
The Quartus Prime Software: Foundation

Design Entry

Design Entry Methods

- Quartus Prime design entry
  - Text editor
    - VHDL
    - Verilog or SystemVerilog
  - Schematic editor
    - Block Diagram File
  - System editor
    - Qsys
  - State machine editor
    - HDL from state machine file
  - Memory editor
    - HEX
    - MIF
- 3rd-party EDA tools
  - EDIF 2 0 0
  - Verilog Quartus Mapping (**.vqm**)

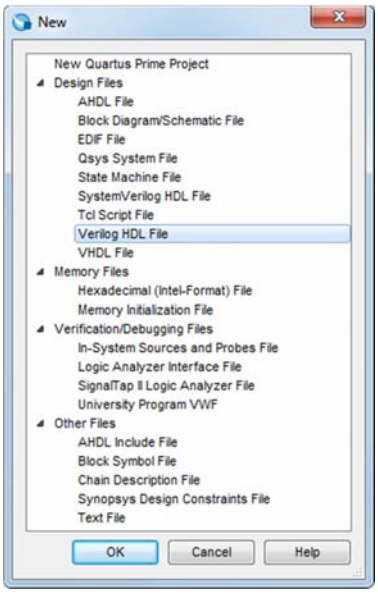
Design Entry File Types Supported



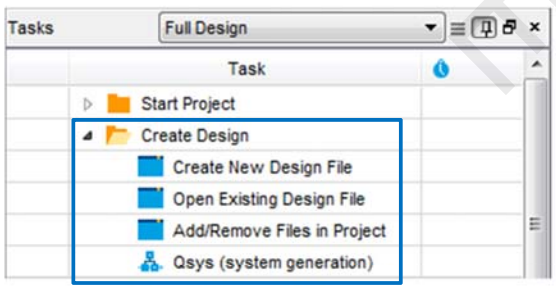
- Mixing & matching design files allowed

Creating New Design Files (& Others)

File menu → **New** or in Toolbar



Tasks window



Text Design Entry

- Quartus Prime Text Editor features
  - Block commenting
  - Line numbering in HDL text files
  - Bookmarks
  - Syntax coloring
  - Find/replace text
  - Find and highlight matching delimiters
  - Function collapse/expand
  - Create & edit **.sdc** files for TimeQuest timing analyzer
  - ***Preview/editing of full design and construct HDL templates***
- Enter text description
  - VHDL (**.vhd**, **.vhdl**)
  - Verilog (**.v**, **.vlg**, **.Verilog**, **.vh**)
  - SystemVerilog (**.sv**)

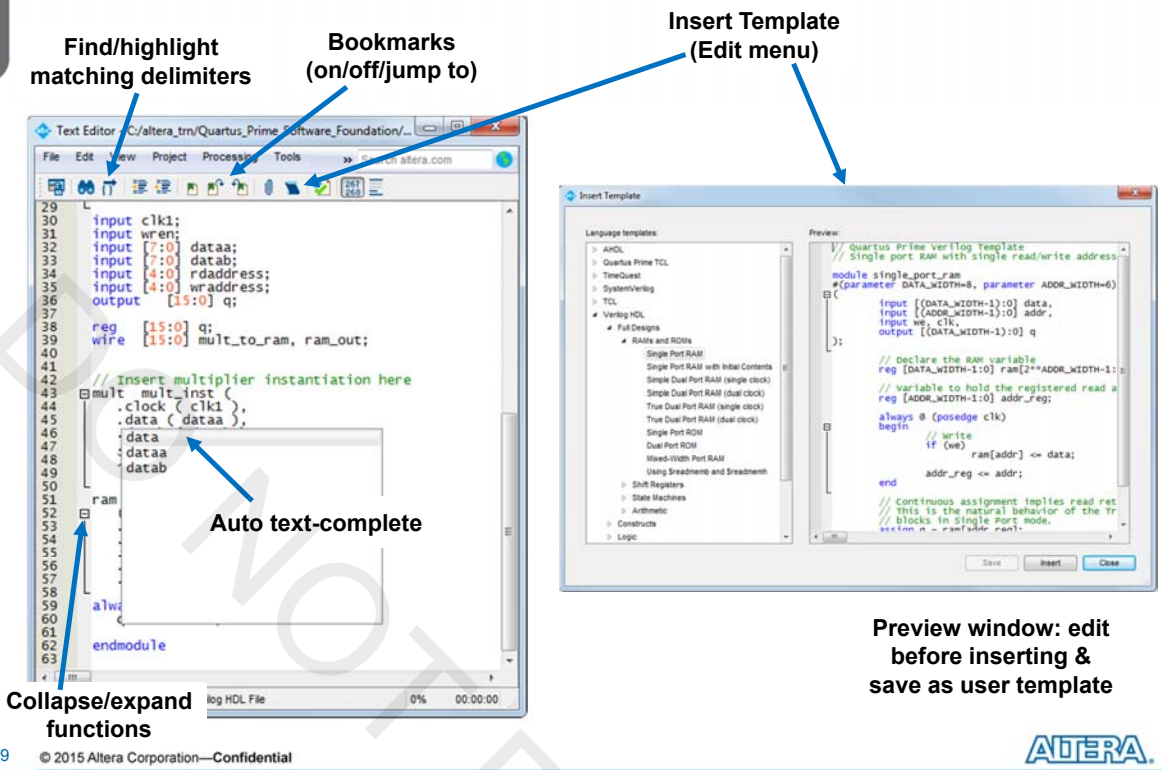
Note: select your default text editor from ***Tools*** *menu →* ***Options*** *→* ***Preferred Text Editor***

Verilog & VHDL

- **VHDL** - VHSIC hardware description language
  - IEEE Std 1076 (1987 & 1993) supported
  - Partial IEEE Std 1076-2008 support
  - IEEE Std 1076.3 (1997) synthesis packages supported
- **Verilog**
  - IEEE Std 1364 (1995 & 2001) & 1800 (SystemVerilog) supported
- Use Quartus Prime integrated synthesis to synthesize
- View supported commands in built-in help

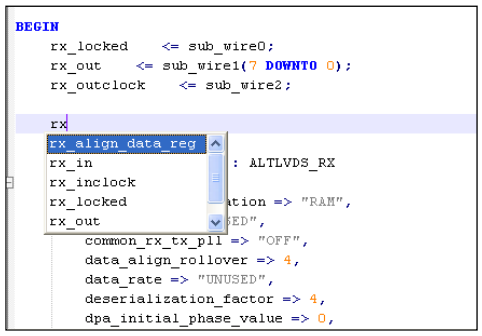
Learn more about HDL in Altera HDL customer training classes

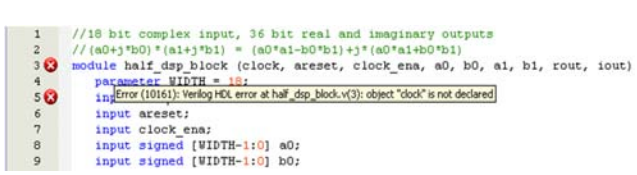
Text Editor Features



Additional Text Editor Features

- Auto-complete
- Smart highlighting
- Syntax color differentiation
- Error message indicator & tooltips





Schematic Design Entry

- Full-featured schematic design capability
- Schematic Editor uses
  - Create simple test designs to understand the functionality of an Altera IP: PLL, LVDS I/O, memory, etc…
  - Create top-level schematic for easy viewing & connection
  - Convert between schematic **.bdf**, block symbol **.bsf**, and HDL files

Note: Schematic entry online training available: Using the Quartus II Software: Schematic Design (http://www.altera.com/education/training/courses/ODSW1105)

Altera IP Cores

- Pre-made design blocks
- Benefits
  - Configurable, parameterized settings add flexibility & portability
  - "Drop-in" support to accelerate design entry
  - Pre-optimized for Altera architecture
- Many basic non-encrypted functions included for free in web and subscription editions
- More advanced IP available in two forms
  - IP Base Suite
  - MegaCore IP

IP Base Suite

- Free & installed with the Quartus Prime software
  - License required for Lite edition
    - Otherwise operate in OpenCore® Plus mode *(discussed in a moment)*
  - HDL simulation models installed in Quartus Prime libraries
- Included in IP Base Suite
  - FIR Compiler
  - Numerically Controlled Oscillator
  - Fast Fourier Transform Compiler
  - AXI BFMs (bus functional models) for simulation
  - DDR2/3/QDRII/LPDDR2/RLDRAMII SDRAM Controllers with UniPHY
  - *See https://www.altera.com/products/intellectual-property/design/ip-base-suite.html for details*

IP MegaCores

- Must purchase license (except IP Base Suite)
  - Logic for IP function is encrypted
- Two types
  - MegaCore IP: developed by Altera
  - Altera Megafunctions Partner Program (AMPP) IP
- All MegaCore functions & some AMPP functions support OpenCore Plus feature
  - Develop design using free version of core
  - HDL simulation models provided with IP
  - Generate time-limited, JTAG-tethered configuration/programming files
  - See AN320: OpenCore Plus Evaluation of Megafunctions

MegaCore IP Examples

- Triple-Speed Ethernet MAC
- 10Gb Ethernet MAC
- CRC Compiler
- IP Compiler for PCI Express® soft core
  - Note: Hard IP for PCI Express does not required a license
- Video and Image Processing Suite

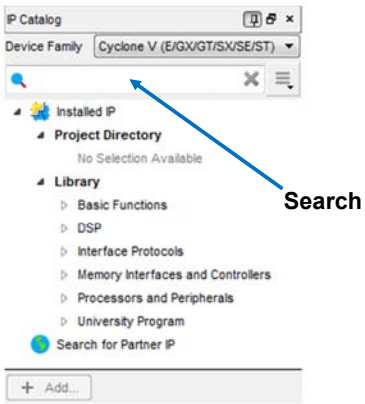
See https://www.altera.com/products/intellectual-property/overview.html for a complete list of Altera IP solutions

IP Catalog

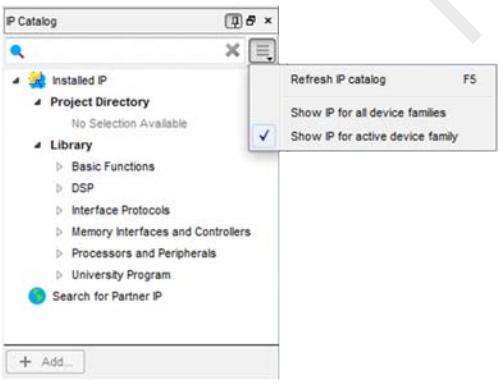
- Centralized tool for the implementation, configuration, & generation of IP cores optimized for a specific device family
- Includes built-in or custom IP

Tools menu → **IP Catalog** or
View menu → **Utility Windows** → **IP Catalog**

No project open
Generate IP for any device family



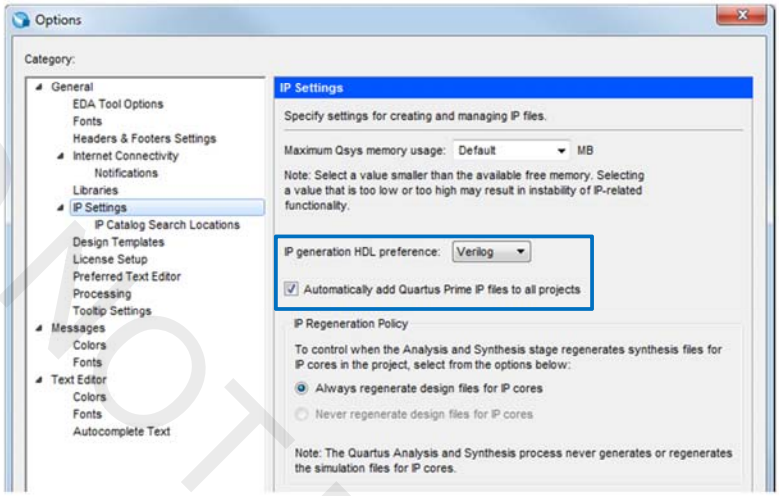
Project open
Generate IP for project device or any device family



IP Settings

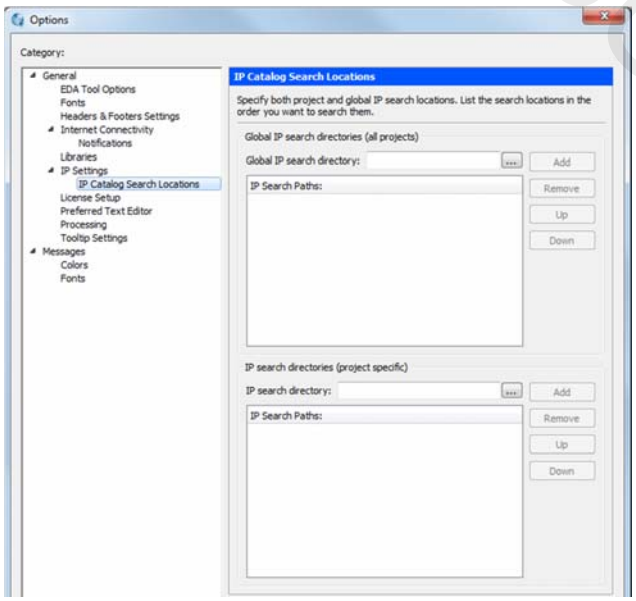
- Choose default output HDL for IP
- Automatically add generated IP to project

Tools menu → **Options**



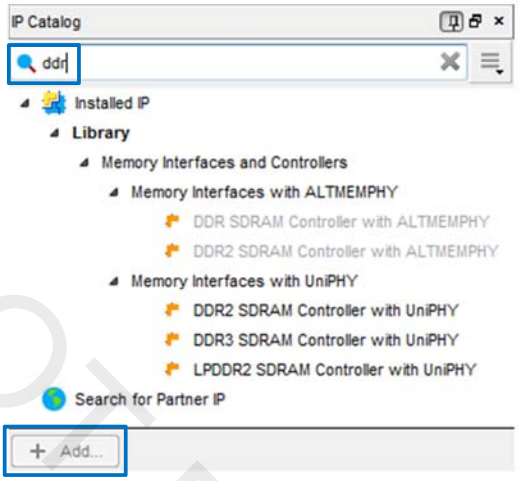
Search for IP

- Point to 3rd-party or custom IP for access in IP Catalog
- Paths shared with Qsys system design tool *(described later)*



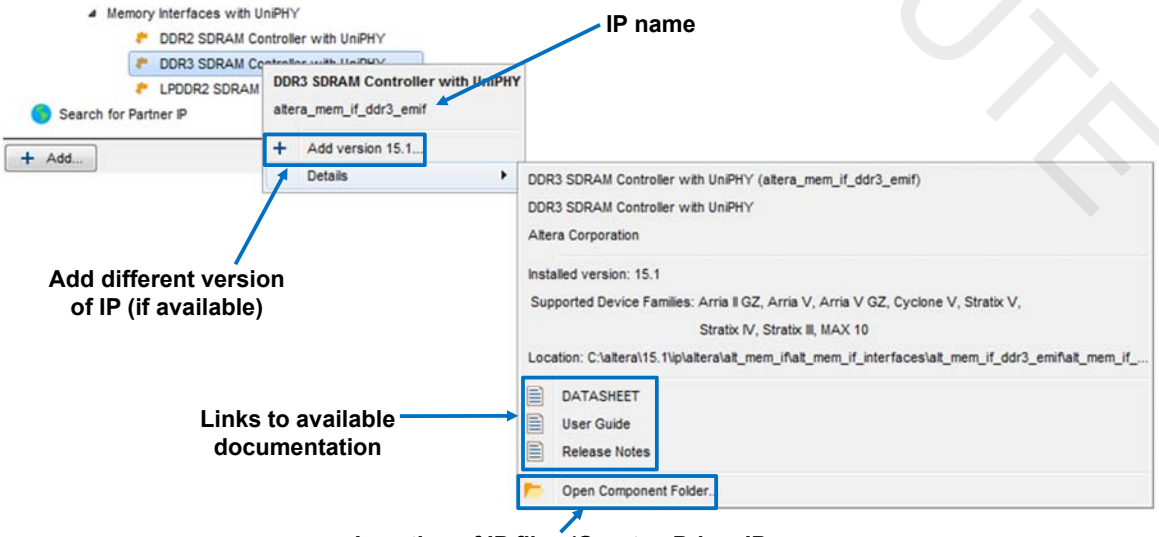
Using the IP Catalog

1. Select IP to create and click **Add**, or double-click IP
2. Name IP (for output file names)
3. Specify IP parameters
4. Generate IP files



IP Options and Documentation

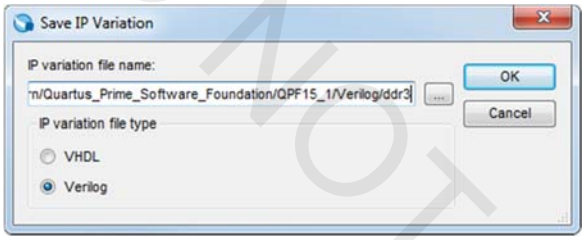
- Right-click IP in IP Catalog to add different versions or get help and documentation



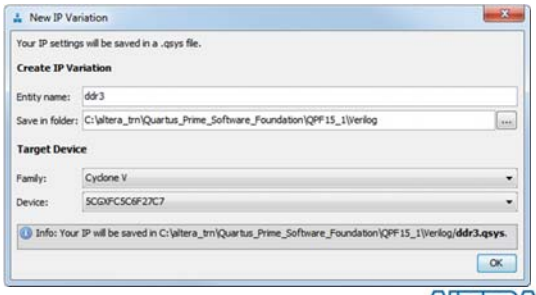
Parameterizing IP

- Classic MegaWizard™ Plug-In Manager or single-window IP Parameter Editor for creating most new IP cores or editing existing cores
- IP Parameter Editor used with:
  - All IP cores targeted to Arria 10 and newer devices
  - New variants of certain IP cores
- Make changes to existing IP through tools discussed earlier

Top-level IP variation file selection for classic parameter editors

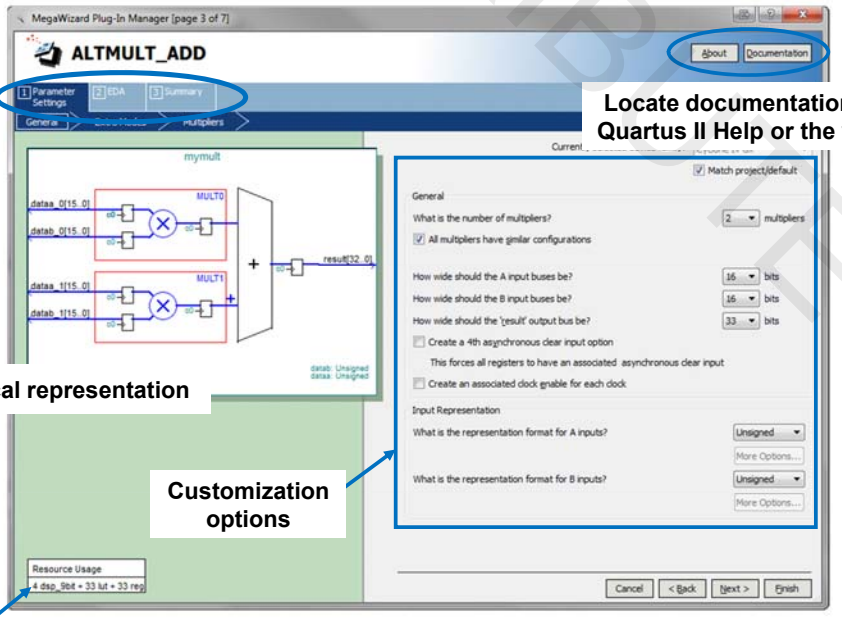


Top-level IP name for new Qsys-based IP Parameter Editor



MegaWizard Plug-In Manager Example

Multiply-Add IP core



Three step process to configure megafunction

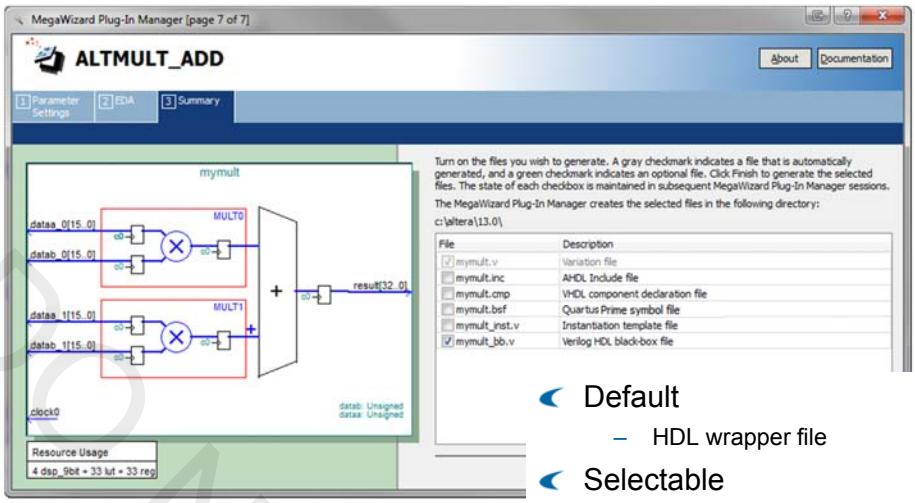
Locate documentation in Quartus II Help or the web

Updating graphical representation

Customization options

Estimated device resource usage

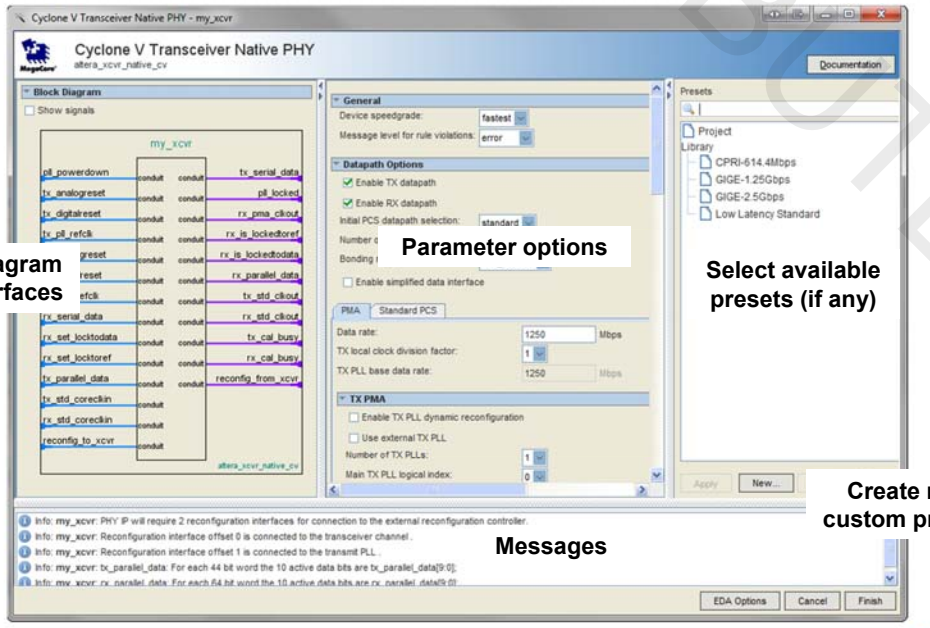
MegaWizard Plug-In Output File Selection



- Default
  - HDL wrapper file
- Selectable
  - HDL instantiation template
  - VHDL component declaration (**.cmp**)
  - Quartus Prime symbol (**.bsf**)
  - Verilog black box
  - Behavioral waveform (**.html**)

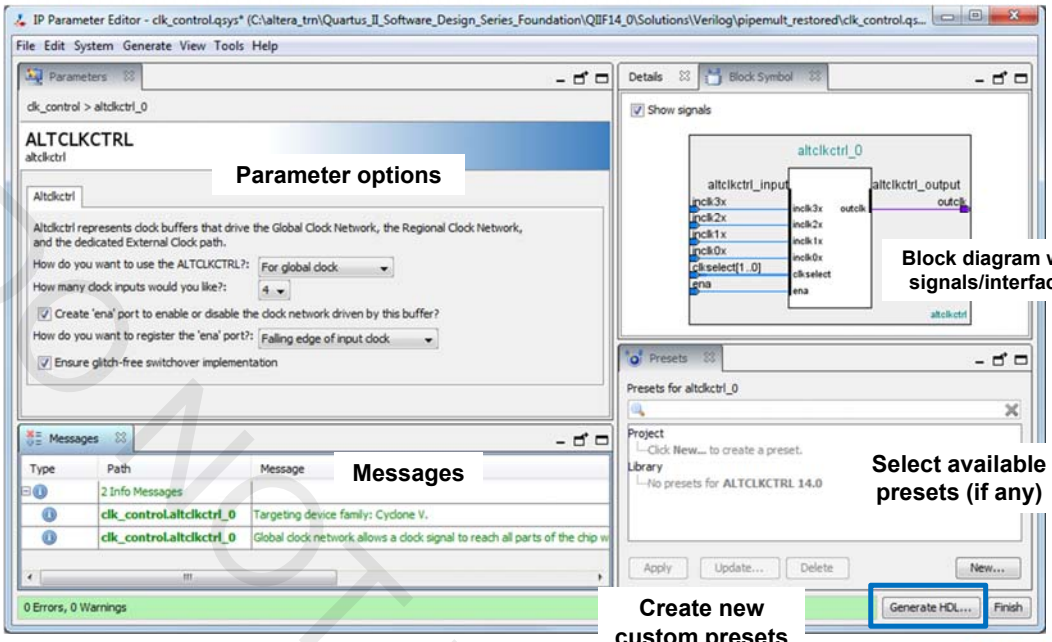
Single-Window Parameter Editor Example

- Variant of MegaWizard Plug-In Manager for some newer IP



IP Parameter Editor

- Generates a Qsys "system" for IP



Output Files

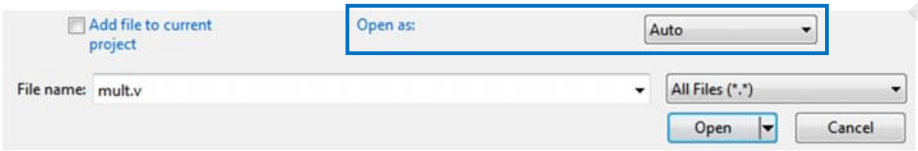
- Top-level folder files
  - *<IP_name>*.**qsys**
    - Main Qsys system file; can be opened using Qsys system design tool
  - *<IP_name>*.**sopcinfo**
    - XML file describing IP used for software development tools
- Located in *<IP_name>* folder
  - *<IP_name>*.**bsf**
    - Symbol file for Quartus Prime schematic editor *(optional)*
  - *<IP_name>*.**html**
    - HTML generation report
  - Instantiation templates
- Located in *<IP_name>*/**synthesis** folder
  - *<IP_name>*.**qip**
    - Script file that adds all files needed for synthesis to Quartus Prime project
  - *<IP_name>*.**[v|vhd]**
    - IP core top-level file connecting all components together
  - Submodule files for synthesis
    - Located in *<IP_name>*/**synthesis/submodules** folder
    - Combination of Verilog, SystemVerilog, and/or VHDL files representing IP
- Files for simulation (optional)

New IP Parameter Editor Output Files Example

- **project (folder)**
  - clk_ctrl.qsys
  - clk_ctrl.sopcinfo
  - **clk_ctrl (folder)**
    - clk_ctrl.bsf
    - clk_ctrl.html
    - clk_ctrl_inst.v
    - clk_ctrl_inst.vhd
    - clk_ctrl_inst.cmp
    - **synthesis (folder)**
      - clk_ctrl.qip
      - clk_ctrl.[v|vhd]
      - **submodules (folder)**
        - component.v, .vhd, and .sv files

Methods for Editing Existing IP Parameters

1. Double-click or right-click IP core in **IP Components** view of **Project Navigator**
2. Double-click in **Upgrade IP Components** dialog box
3. Open IP core's top-level **.v/.vhd** file as **Auto**

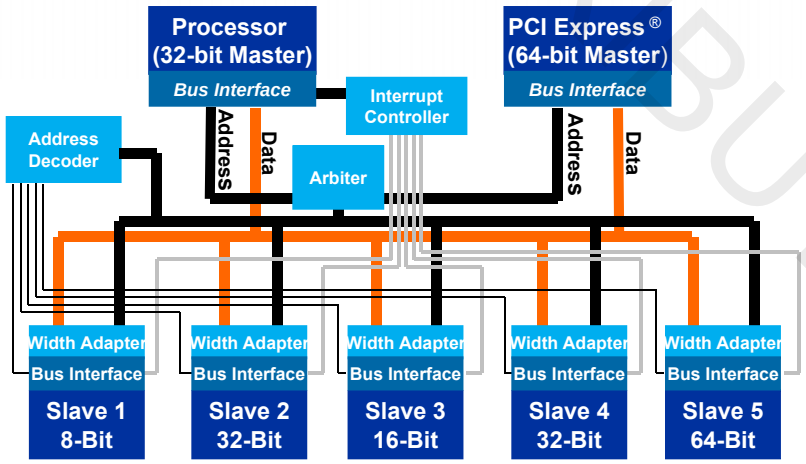


- Change parameters and regenerate IP core

System Design Entry With Qsys

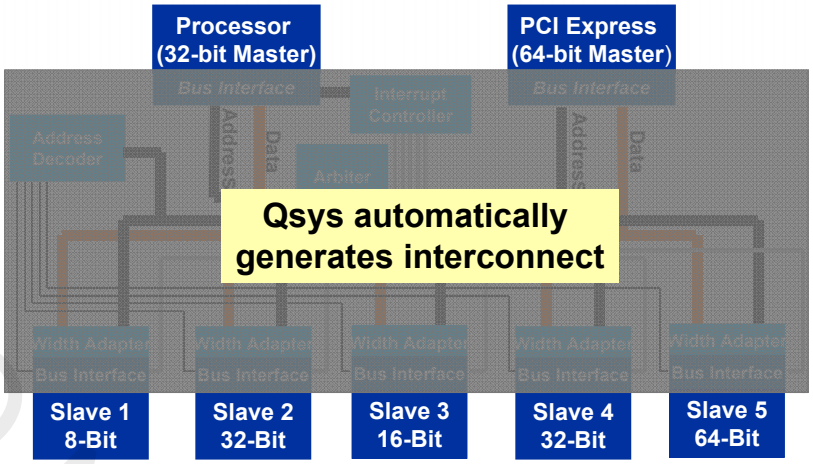
- Simplifies complex system development
  - Automatic interconnect generation
- Raises the level of design abstraction
  - High level design and system visualization
- Provides a standard platform: IP integration, custom IP authoring, IP verification
- Enables design re-use
- Scales easily to meet the needs of end product
- Reduces time to market
  - Reduces design development time
  - Eases verification

But What Exactly is Qsys?



- Components in system use different interfaces to communicate (some standard, some non-standard)
- Typical system requires significant engineering work to design custom interface logic
- Integrating design blocks and intellectual property (IP) is tedious and error-prone

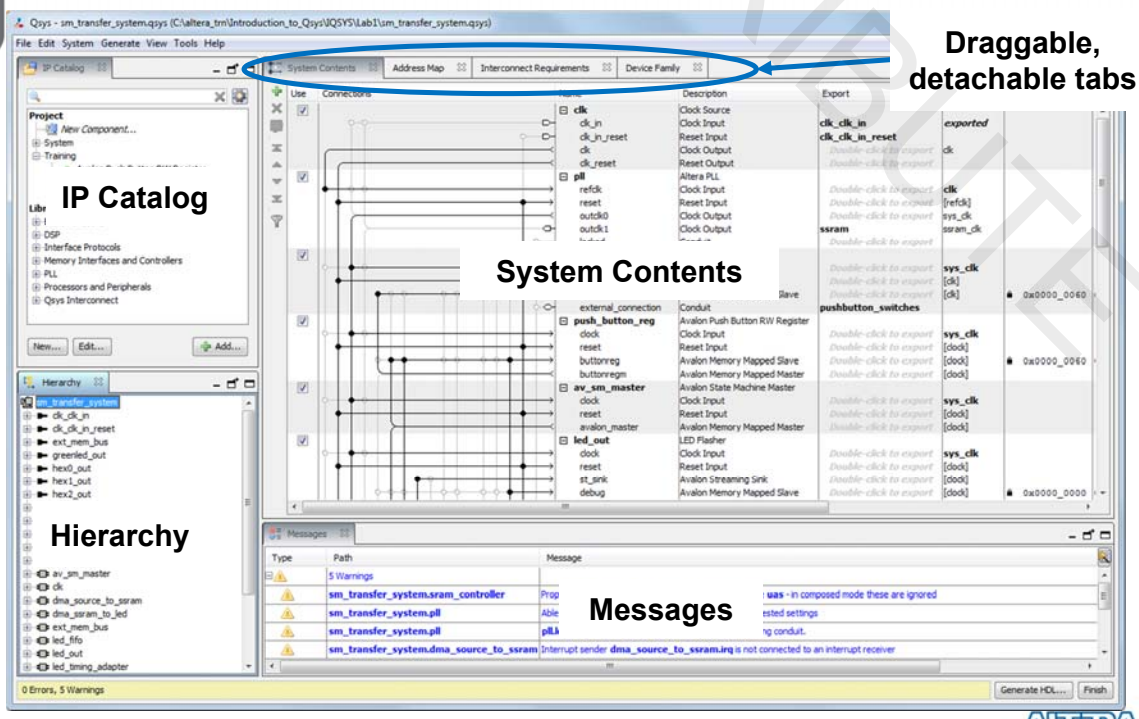
Automatic Interconnect Generation



- Avoids error-prone integration
- Saves development time with automatic logic & HDL generation
- Enables you to focus on value-add blocks

Qsys improves productivity by automatically generating the system interconnect logic

The Qsys GUI



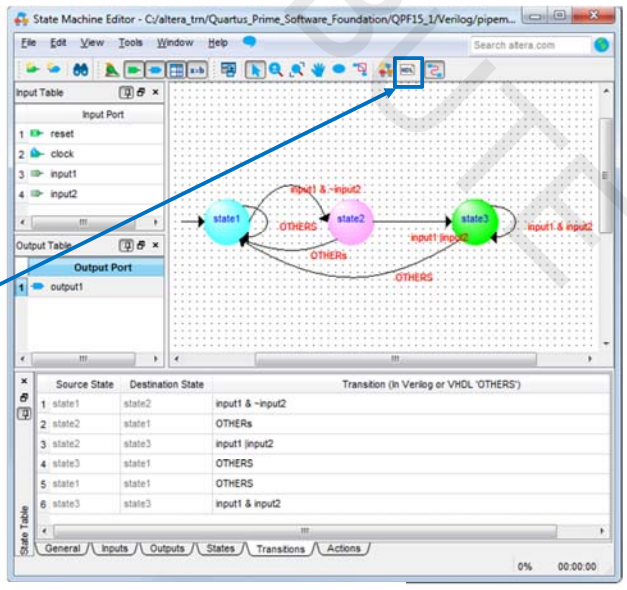
Qsys Resources

- Qsys chapters in Volume 1 of the Quartus Prime Handbook
- Instructor-led training
  - *Introduction to the Qsys System Integration Tool*
  - *Advanced Qsys System Integration Tool Methodologies*
- Online training
  - *Introduction to Qsys*
  - *Creating a System Design with Qsys*
  - *Advanced System Design Using Qsys* series
    - *Component & System Simulation*
    - *System Verification with System Console*
    - *Qsys System Optimization*
    - *Utilizing Hierarchy in Qsys Designs*

State Machine Editor

- Create state machines in GUI
  - Manually by adding individual states, transitions, and output actions
  - Automatically with State Machine Wizard (**Tools** menu & toolbar)
- Generate state machine HDL code (required)
  - VHDL
  - Verilog
  - SystemVerilog

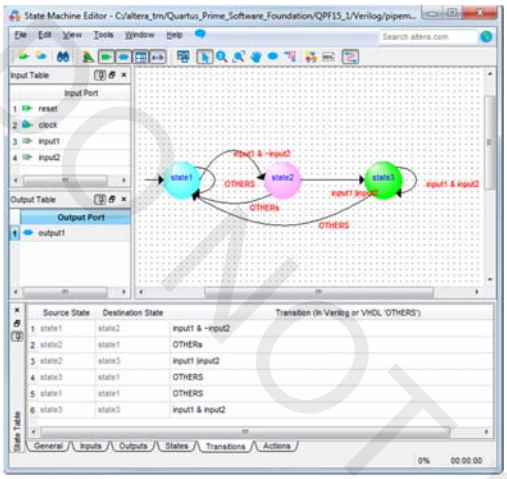
File menu → **New** or **Tasks** window
Select *State Machine File* (**.smf**)

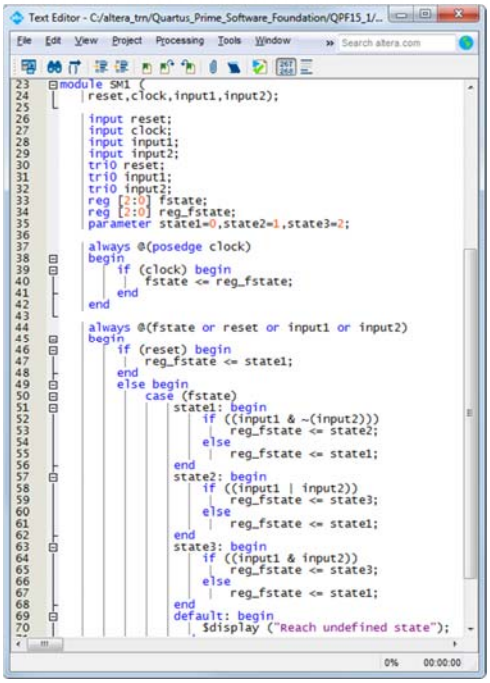


Double-click states & transitions to edit properties: name, equations, actions

From **.smf** to HDL

- Generate optimized code (Verilog, SV, or VHDL)
- Automatically added to project
- *Required for use*





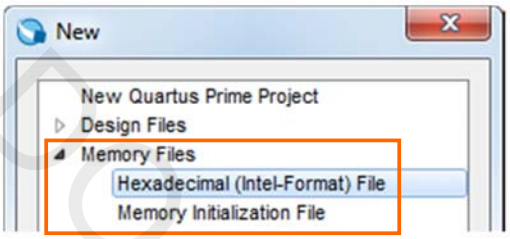
Memory Editor

- Create or edit memory initialization files
  - Intel HEX (**.hex**)
    - Preferred and compatible with 3rd party tools
  - Altera-specific (**.mif**) format
- Design entry
  - Initialization file data sent to device during device programming to initialize memory blocks
- Simulation
  - Use to initialize memory blocks before simulation or after breakpoints

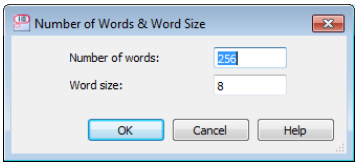
Create Memory Initialization File

File → New or Tasks window

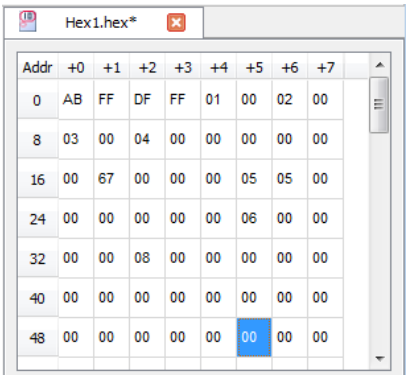
1) HEX or MIF format



2) Select memory size

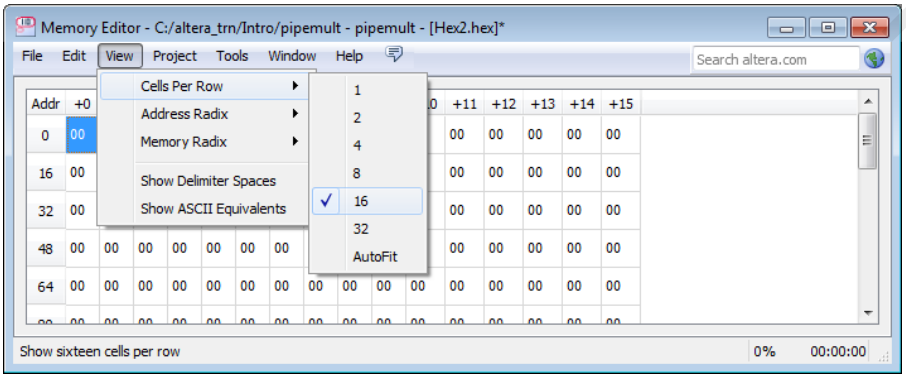


3) Memory space editor opens



Change Options

- View options of memory editor
  - **View** menu → select from available options



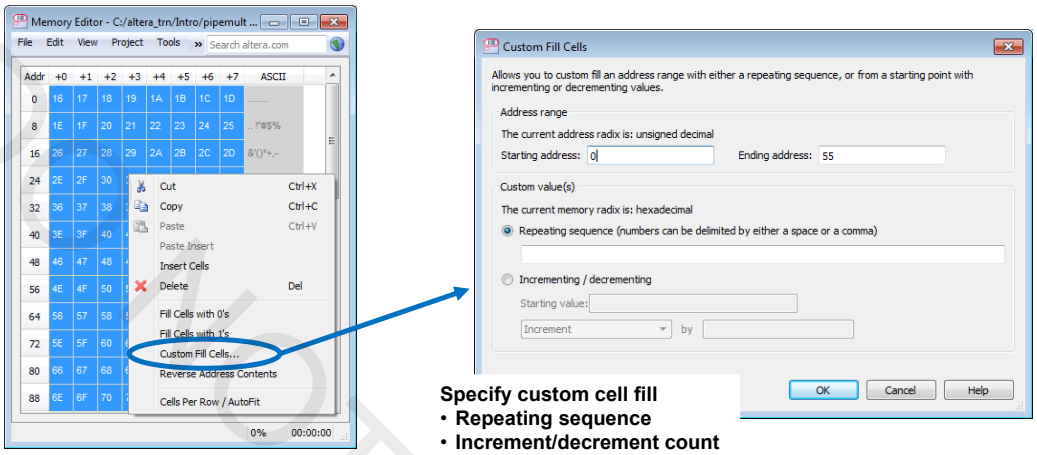
Edit Contents

- Edit contents of memory file
- Save memory file as **.hex** or **.mif** file

•Select address location & type in a value

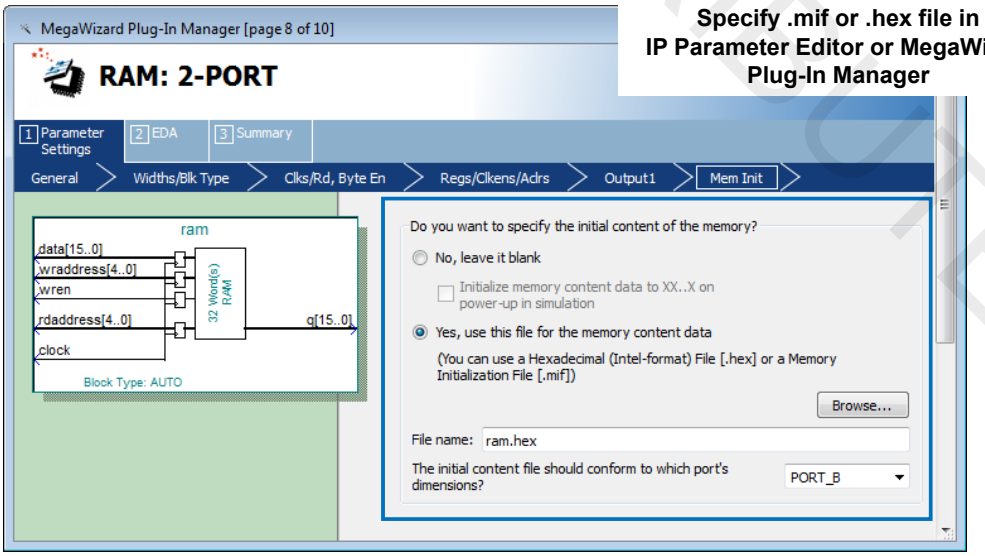
•Select the address & right-click to select fill option from menu

•Copy & paste from spreadsheet



Specify custom cell fill
- **Repeating sequence**
- **Increment/decrement count**

Using Memory File in Design

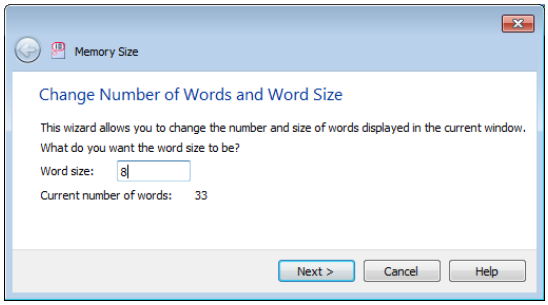


Specify .mif or .hex file in IP Parameter Editor or MegaWizard Plug-In Manager

May also specify .mif file in HDL using the `ram_init_file` synthesis attribute

Memory Size Wizard

- Allows editing the size of memory file
- Use the Memory Size Wizard (**Edit** menu)
  - Edit word size
  - Edit number of words
  - Specify how to handle word size change
    - Pad words
    - Combine words
    - Truncate words
    - Add words



Importing 3rd-Party EDA Tool Files

- Interface with industry-standard EDA tools that generate netlist files
  - EDIF 2 0 0 (**.edf**)
  - Verilog Quartus Mapping (**.vqm**)
- To import and use netlist files
  - Specify EDA tool in the Quartus Prime software settings
  - Instantiate block(s) in design
  - Add **.edf**/**.vqm** file(s) to Quartus Prime project

3rd-Party Synthesis Tool Support

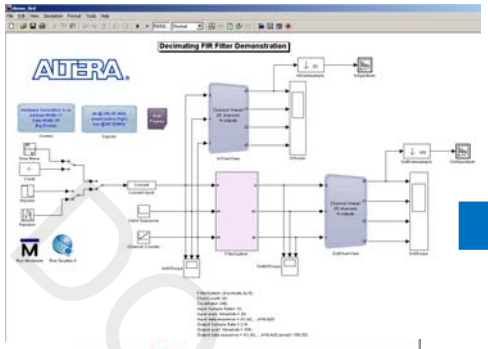
- Precision RTL
- Precision RTL Plus

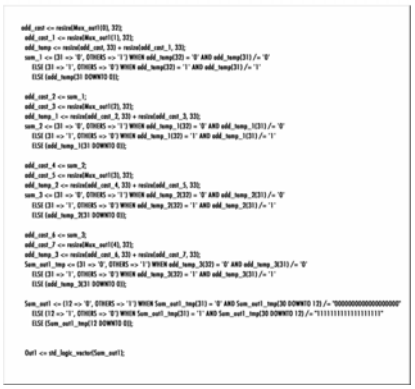
- Synplify
- Synplify Pro
- Synplify Premier

System Design Entry Tools

- Qsys system development tool *(discussed earlier)*
- DSP Builder Standard/Advanced Blocksets
- Altera SDK for OpenCL™ Compiler

DSP Builder (DSPB) Standard/Advanced Blocksets

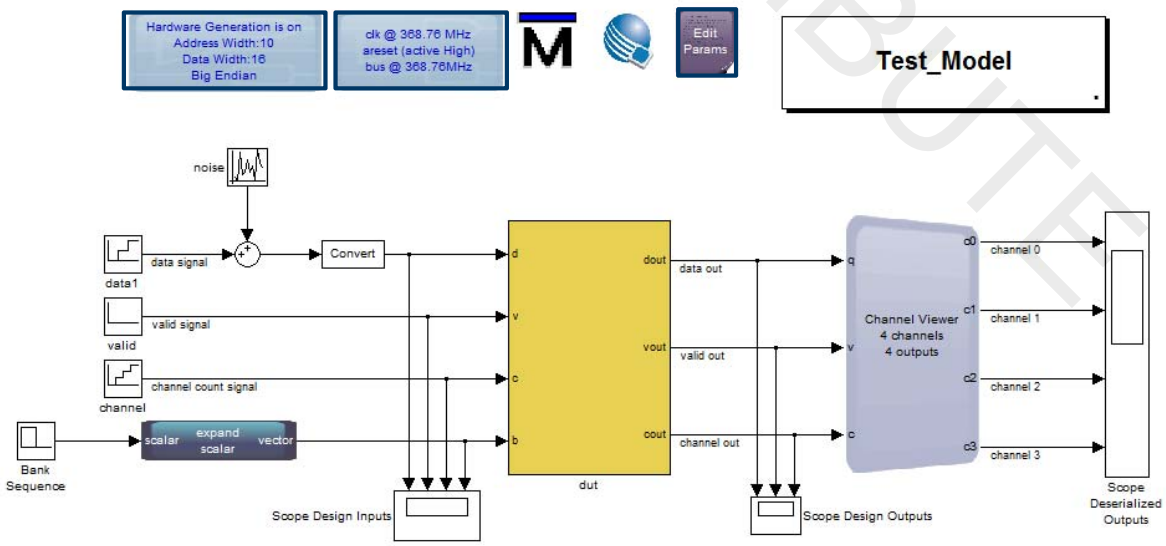




MATLAB SIMULINK

HDL automatically optimized for system clock frequency & latency

DSP Builder Advanced Blockset Example



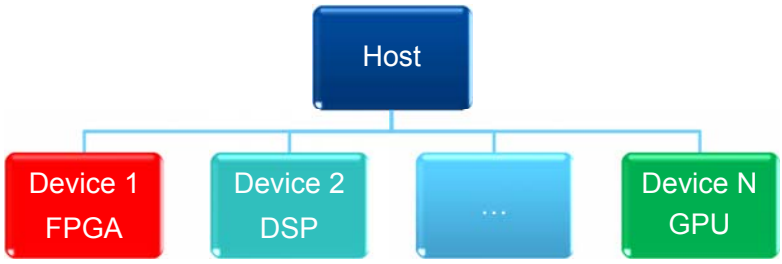
- Top-level of a DSP Builder design is a testbench

DSP Builder Design Standard and Advanced Blocksets

- DSP Builder Standard Blockset
  - Cycle accurate behavior (works the way you designed it)
  - Multiple clock domain design
  - Control rich with state machine or backpressure
  - Access detailed device features
  - Custom HDL code import support
  - Hardware-in-the-loop simulation support
- DSP Builder Advanced Blockset
  - Specification driven design with automatic pipeline and folding
  - Multichannel designs with automatic vectorized inputs
  - Single system clock for the main datapath logic
  - Design exploration
- DSP solutions web site
  - https://www.altera.com/solutions/technology/dsp/overview.html

Open Computing Language (OpenCL)

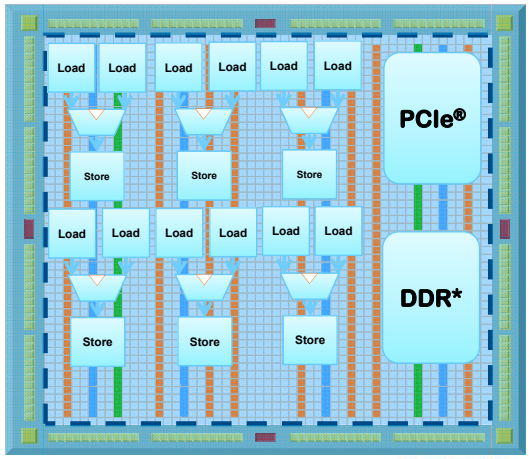
- Framework for heterogeneous programs
- Royalty-free
- Allows general purpose programs to run on multiple platforms
- Adaptable to obtain high performance



Altera SDK for OpenCL

- Translates OpenCL source file into custom-generated hardware image to be loaded onto FPGA
- Provides API for use by OpenCL host application to execute and communicate with hardware image
- Altera SDK for OpenCL
  - https://www.altera.com/products/design-software/embedded-software-developers/opencl/overview.html

```
// kernel.cl
__kernel void KernelName(…)
{
   int i = get_global_id(0);
   c[i] = a[i] + b[i];
}
```



Test Your Knowledge: Design Entry

1. IP cores in the IP Base Suite and IP MegaCores require additional licensing to generate unrestricted programming files.
   - ❑ True
   - ❑ False

2. The Memory Editor is used to _____. Its supported file formats are _____.
   a. change the contents of initialized RAMs and ROMs; **.mif** and **.hex**
   b. initialize on-chip RAMs and ROMs; **.mif** and **.hex**
   c. initialize on-chip RAMs and ROMs; **.mif** and **.qsys**
   d. view the contents of initialized RAMs and ROMs; **.hex** and **.emif**

Exercise 2

Design Entry Summary

- Multiple design entry methods supported
  - Text (Verilog, VHDL, State Machine Editor output)
  - 3rd-party netlist (VQM, EDIF)
  - Schematic
  - System (Qsys)
- IP Parameter Editor and MegaWizard Plug-In Manager parameterize IP cores
- Memory Editor allows generation of memory initialization files
- 3rd-party EDA tools supported for design entry & synthesis

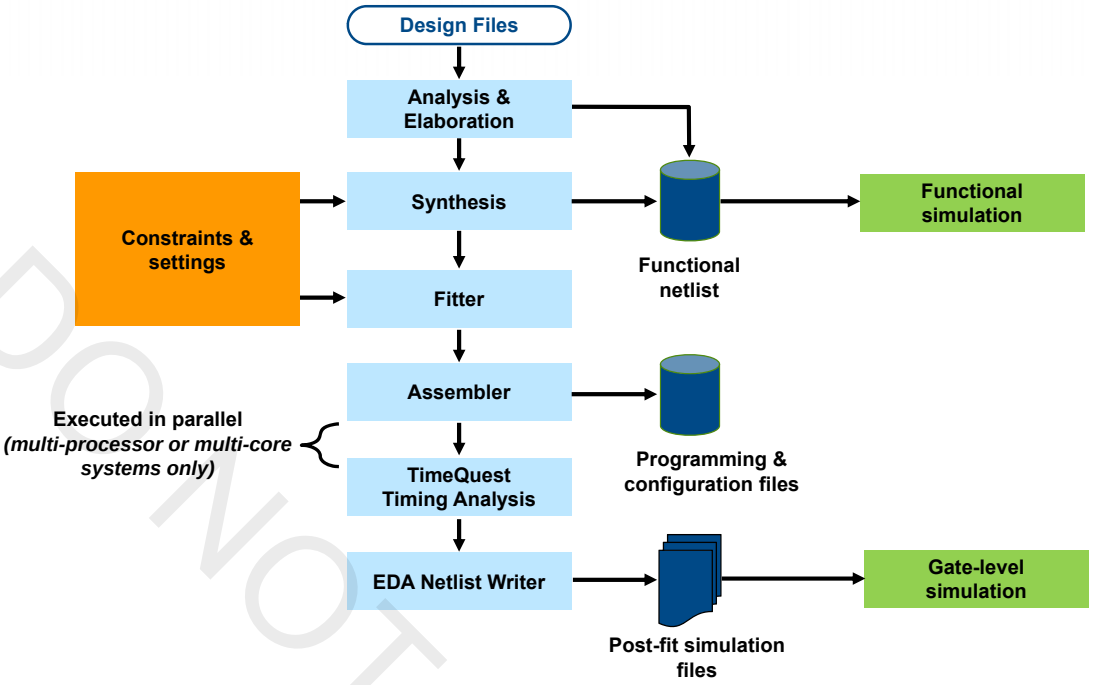
Design Entry Support Resources

- Quartus Prime Handbook chapters (Volume 1)
  - Recommended Design Practices
  - Recommended HDL Coding Styles
  - Qsys chapters
    - Creating a System With Qsys
    - Creating Qsys Components
    - Qsys Interconnect
  - 3rd-party EDA tool chapters
- Training courses & demonstrations
  - VHDL & Verilog HDL Basics (online courses)
  - Introduction to Verilog HDL & Advanced Verilog HDL Design Techniques courses
  - Introduction to VHDL & Advanced VHDL Design Techniques courses
  - SystemVerilog with the Quartus II Software (online)
  - Designing with the DSP Builder Advanced Blockset
  - Introduction to OpenCL for Altera FPGAs

The Quartus Prime Software: Foundation

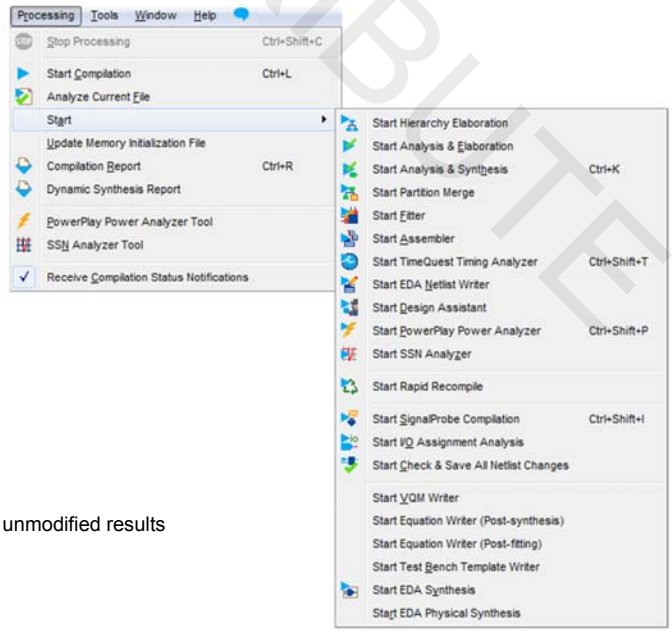
Quartus Prime Compilation

Quartus Prime Full Compilation Flow



Processing Options

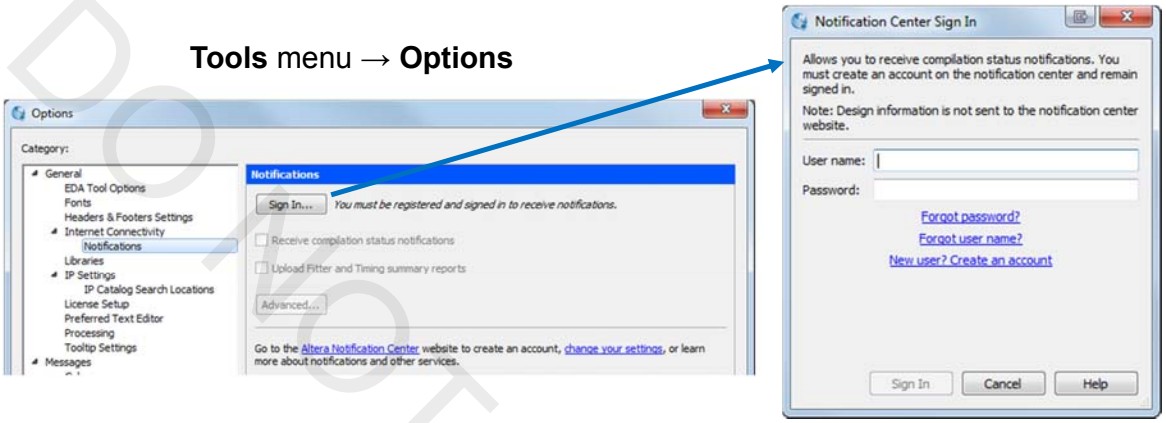
- **Start Compilation** (Full)
- **Start Hierarchy Elaboration**
  - Checks syntax & builds hierarchy
- **Start Analysis & Elaboration**
  - Checks syntax & builds hierarchy
  - Performs initial synthesis
- **Start Analysis & Synthesis**
  - Synthesizes & optimizes code
- **Start Fitter**
  - Places & routes design
  - Generates output netlists
- **Start Assembler**
  - Generate programming files
- **Start Rapid Recompile**
  - Run compilation preserving previous unmodified results
- Analyzers
  - I/O Assignment
  - PowerPlay
  - Design Assistant



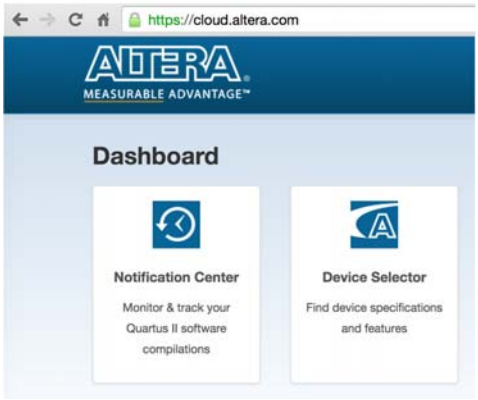
Notification Center

- Monitor compilations via the web
- Receive email notifications when compilations finish
- Compile is local only; no private design data or IP is transferred
- Receive error or critical warning messages *(optional)*

Tools menu → **Options**



Using the Notification Center

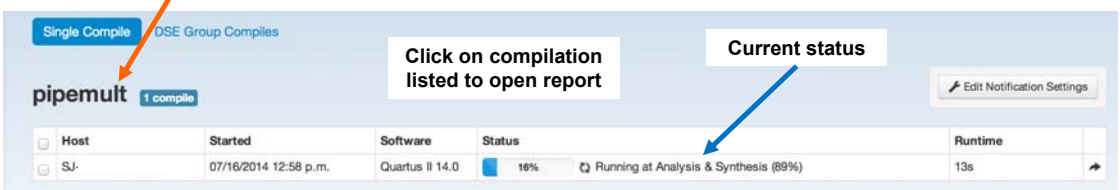


1. Sign up for Altera web account https://cloud.altera.com
   - Requires valid email address
2. Connect Quartus Prime installation to web account by signing in
3. Compile design from command-line or GUI
   - Compile status (only) reflects on web
   - https://cloud.altera.com/notify shows list of running or finished compiles
4. Receive email notifications and web links summarizing compile status
5. Delete records in web account at anytime

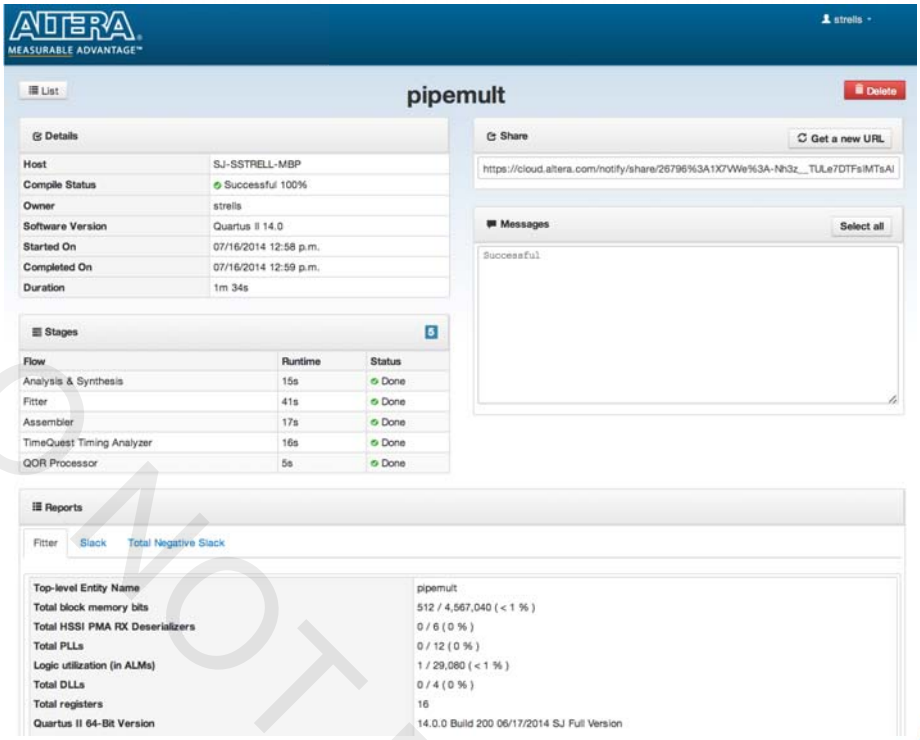
Project name

Click on compilation listed to open report

Current status



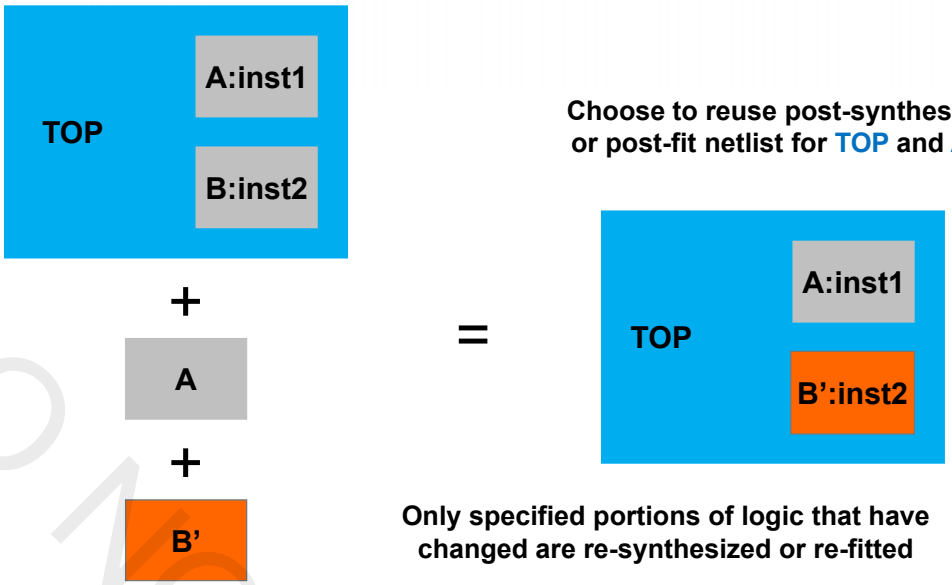
Notification Center Report



Compilation Design Flows

- Default “flat” compilation flow
  - Design compiled as a whole
  - Global optimizations performed
- Incremental flow (on by default for new projects in Lite & Standard Editions; not available in Pro Edition)
  - User assigns design partitions
  - Each partition processed separately; results merged
  - Post-synthesis or post-fit netlists for partitions are reused
  - Benefits
    - Decrease compilation time
    - Preserve compilation results and timing performance
    - Enable faster timing closure

Incremental Compilation Concept



Choose to reuse post-synthesis or post-fit netlist for TOP and A

Only specified portions of logic that have changed are re-synthesized or re-fitted

Note: For more details on using incremental compilation, please attend the course Design Optimization Using Quartus II Incremental Compilation or watch the online training Introduction to Incremental Compilation

Reducing Compile Times

- Incremental compilation
- Parallel compilation
  - Take advantage of multi-processor, multi-core machines

Assignments menu → **Settings**
(discussed in more detail later)

Rapid Recompilation

- Reuse previous compilation results to update design for *small* changes (affect less than 5% of design logic)
- Design changes supported by rapid recompilation
  - Node tapping with the SignalTap II embedded logic analyzer
  - Modify simple combinational logic
  - Modify state machine logic
  - Add pipeline registers
  - *See Quartus Prime Handbook for more*
- Works with and without incremental compilation
- "Disengages" if compiler determines netlist can't be preserved

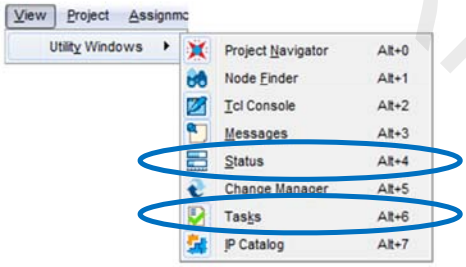
Processing menu → **Start** → **Start Rapid Recompile**

Partition Merge Rapid Recompile Summary

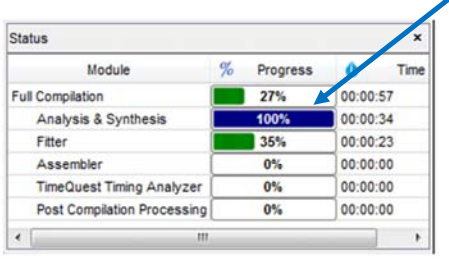
	Partition Name	Rapid Recompile Status	Netlist Preservation Requested	Netlist Preservation Achieved	N
1	Top	Engaged	100.00% (748 / 748)	100.00% (748 / 748)	
2	des_top:des_inst	Engaged	99.65% (4242 / 4257)	99.62% (4241 / 4257)	
3	fir_top:fir_inst	Disengaged	n/a	n/a	Rapid Recompile not attempted: incren
4	placeholder:ph_inst	Disengaged	n/a	n/a	Rapid Recompile not attempted: incren
5	hard_block:auto_generated_inst	Disengaged	n/a	n/a	Rapid Recompile not attempted: empty

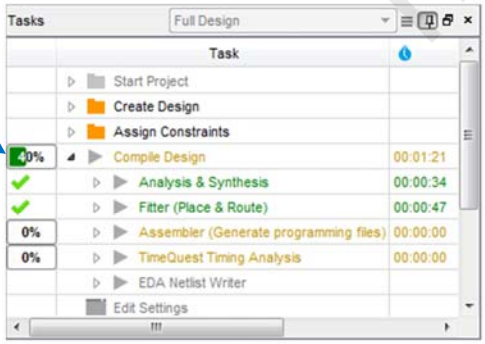
Partition Merge Rapid Recompile Summary
(Partition Merge folder of **Compilation Report)**

Status and Tasks Windows



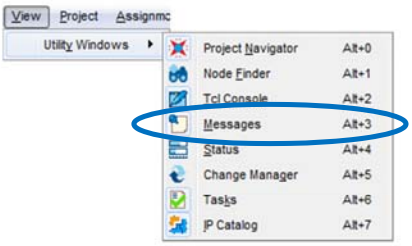
Status bars indicate compilation progress





Message Windows

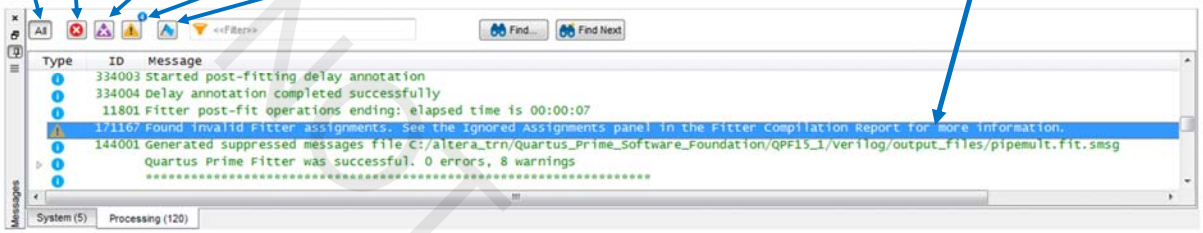
- Message window displays informational, warning, and error messages



Filter messages by type

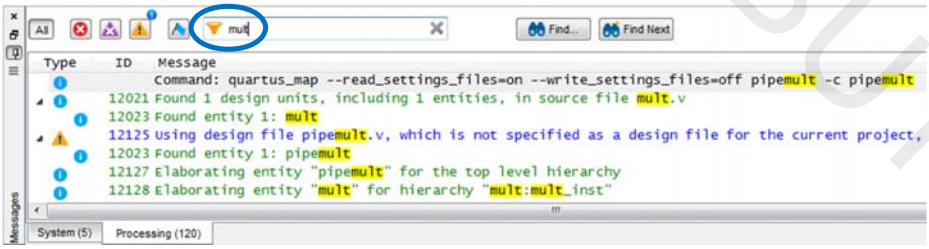
All **Error** **Critical Warning** **Warning** **Flagged**

Right-click to flag selected messages for later review

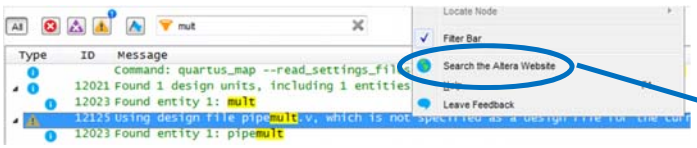


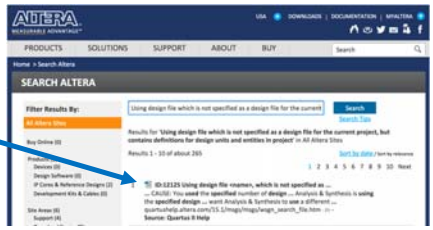
Message Searching & Help

- Search for keywords in the Messages window



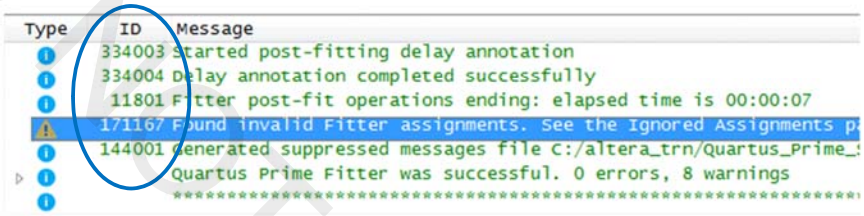
- Right-click to search message on altera.com or built-in help





Message IDs

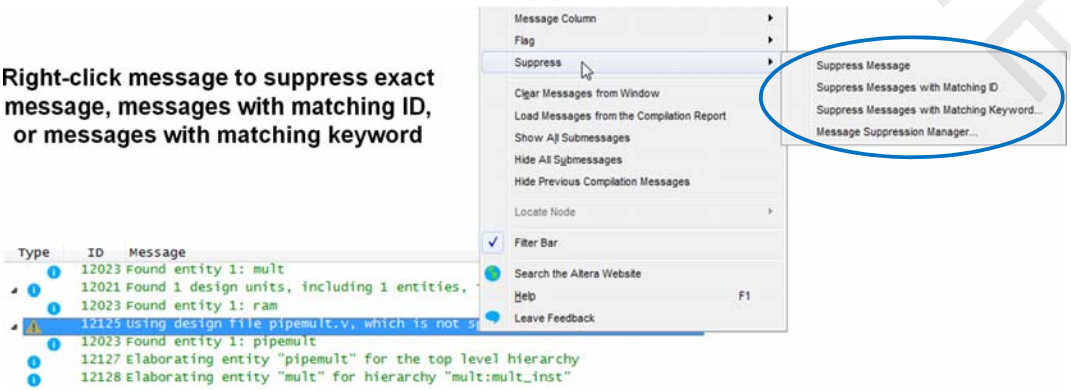
- Most messages have IDs
- Benefits
  - More easily reference messages in service requests
  - Search for collateral documentation (self-help)
  - Disable specific info or warning messages globally (`MESSAGE_DISABLE` assignment)
- Benefits for IP developers
  - Disable info or warning messages specifically for an IP core
  - Ship “warning-free” IP



Message Suppression

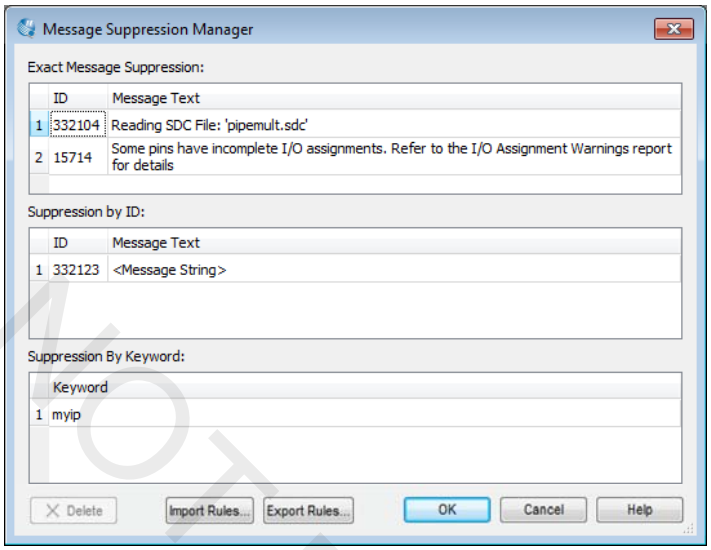
- Hide messages from current and future compiles
  - E.g. known synthesis warning message already investigated
- Store suppression rules in *<revision_name>**.srf*** file

Right-click message to suppress exact message, messages with matching ID, or messages with matching keyword



Message Suppression Manager Tool

- View/edit/remove suppression rules
- Import and export message suppression rules



Viewing Compilation Results

- Quartus Prime graphical tools available for
  - Understanding design processing
  - Verifying correct design results
  - Debugging incorrect results
- Compilation Report
- Viewers
  - RTL Viewer
  - Technology Map Viewer
  - State Machine Viewer
- Chip Planner

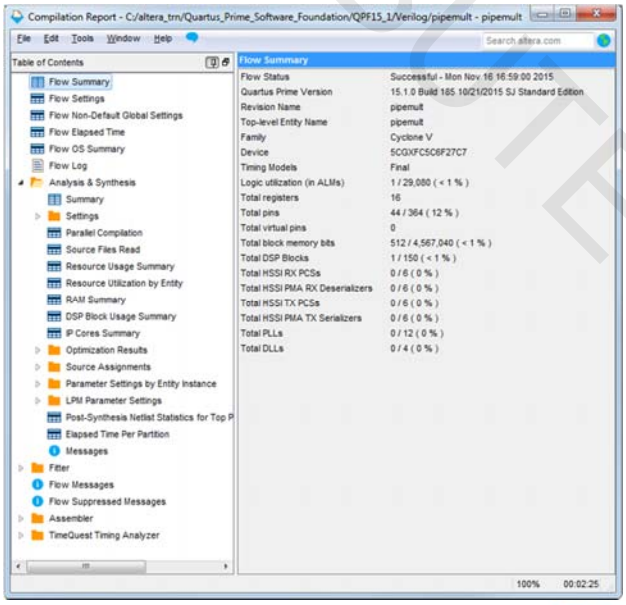
Compilation Report

- Contains all compilation processing information
  - Resource usage
  - Device pin-out
  - Settings and constraints applied
  - Messages
- Recommendation: Go through report for a design to get sense of information being provided
- Information also available as text files in **output_files** folder in project directory: *<revision_name>**.fit.rpt***, *<revision_name>**.map.rpt***, etc.

Compilation Report

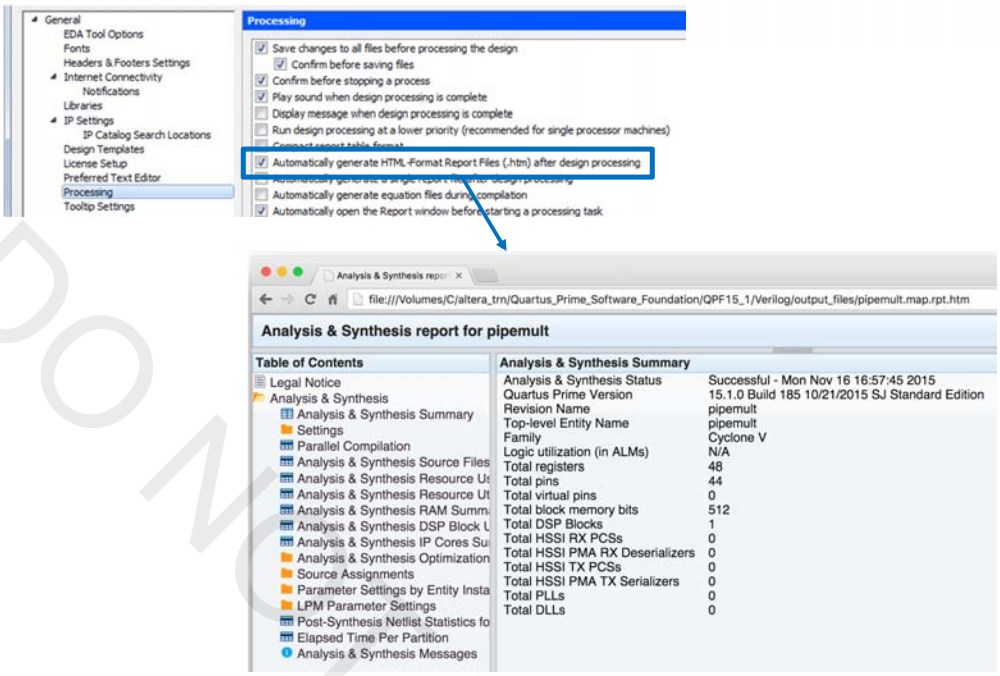
- Access from **Processing** menu or toolbar

Each compiler executable generates separate folder



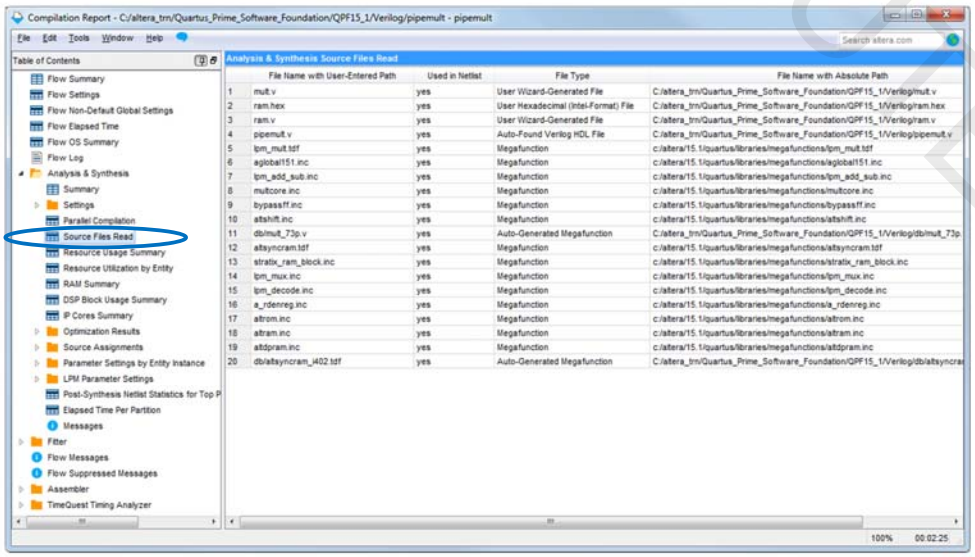
Compilation Report: HTML version

Tools menu → **Options**



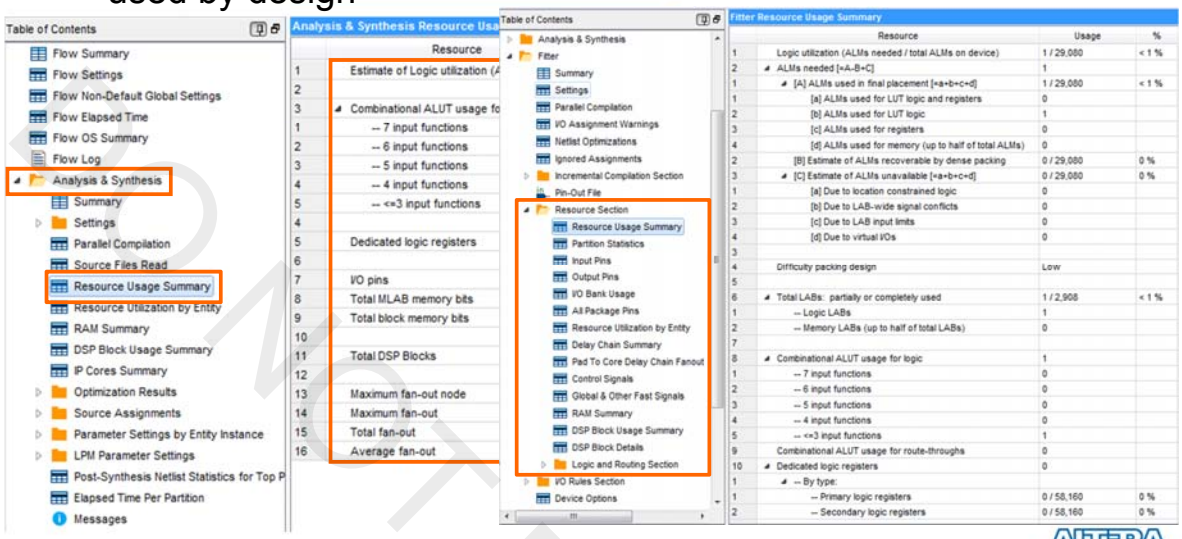
Example: Source Files Read

- List of all design files (user-coded & library) used during last compilation along with files' type and location



Example: Resource Usage

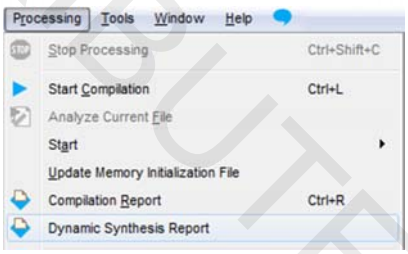
- Synthesis resource usage: estimates of FPGA resources required to implement design
- Fitter resource usage: Detailed information on all resources used by design

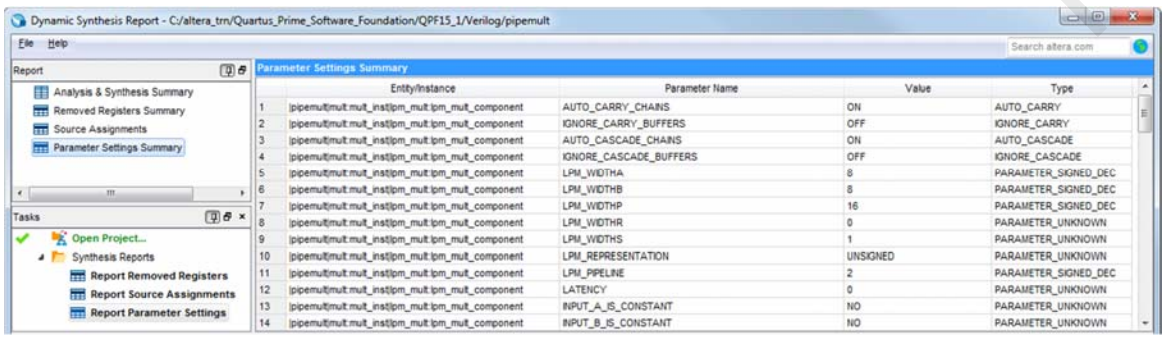


Dynamic Synthesis Report

- Task/report Model
  - Customized entity/node filtered reports
- Available tasks:
  - **Report Removed Registers**
  - **Report Source Assignments**
  - **Report Parameter Settings**

Processing menu → **Dynamic Synthesis Report**



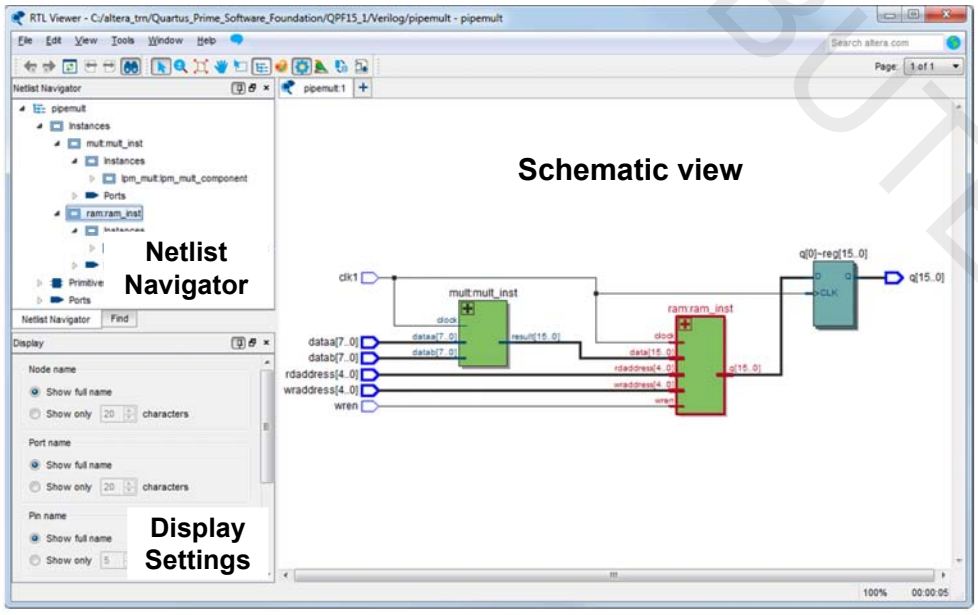


Netlist Viewers

- RTL Viewer
  - Schematic of design after Analysis & Elaboration
  - Visually check initial HDL before synthesis optimizations
  - Locate synthesized nodes for assigning constraints
  - Debug verification issues
- Technology Map Viewers (Post-Mapping or Post-Fitting)
  - Graphically represents results of mapping (post-synthesis) & fitting
  - Analyze critical timing paths graphically
  - Locate nodes & node names after optimizations (cross-probing)

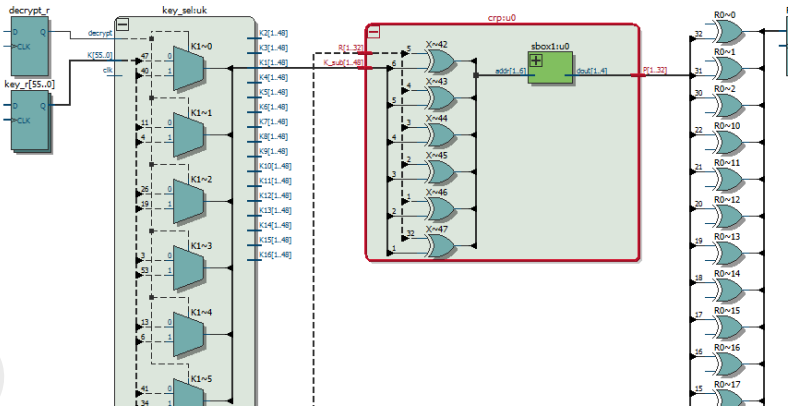
Netlist Viewers

Tools menu → **Netlist Viewers** or **Tasks** window **Compile Design** tasks



Note: Must perform elaboration first (e.g. Analysis & Elaboration OR Analysis & Synthesis)

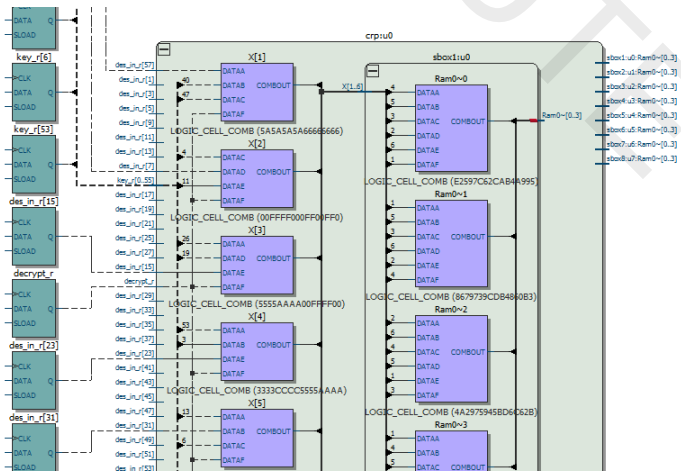
Schematic View (RTL Viewer)



- Represents design using logic blocks & nets
  - I/O pins
  - Registers
  - Muxes
  - Gates (AND, OR, etc.)
  - Operators (adders, multipliers, etc.)

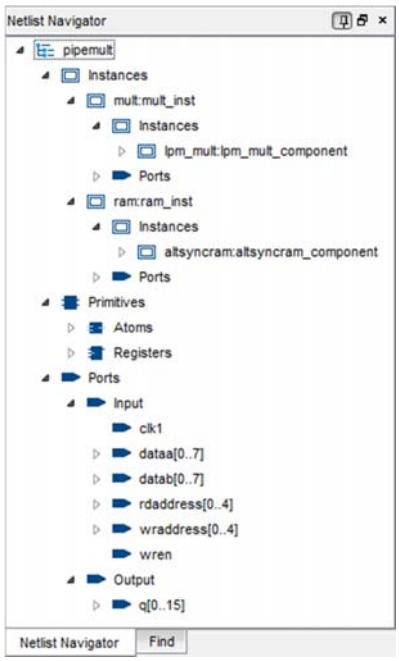
Schematic View (Technology Map Viewer)

- Represents design using atoms
  - I/O pins & cells
  - Logic cells (Lcells)
  - Memory blocks
  - MAC (DSP blocks)

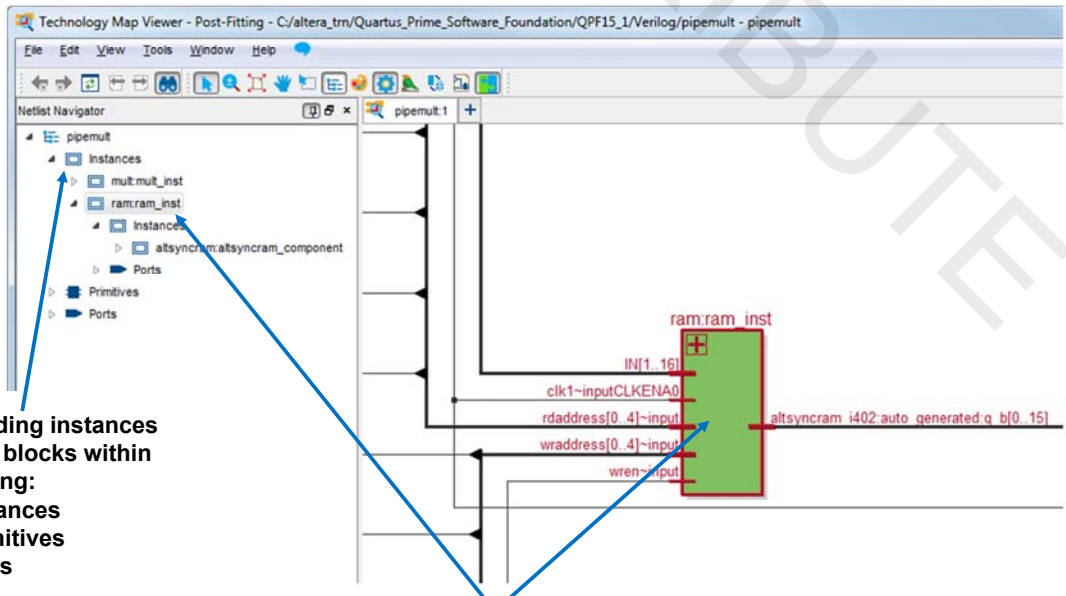


Netlist Navigator (Hierarchy List)

- Traverse between levels of design hierarchy
- View logic schematic for each hierarchical level
- Break down each hierarchical level into netlist elements or atoms
  - Instances
  - Primitives
    - Registers
    - Buffers
    - Logic
    - Operators
    - Atoms
    - I/O
  - Ports
  - State machines



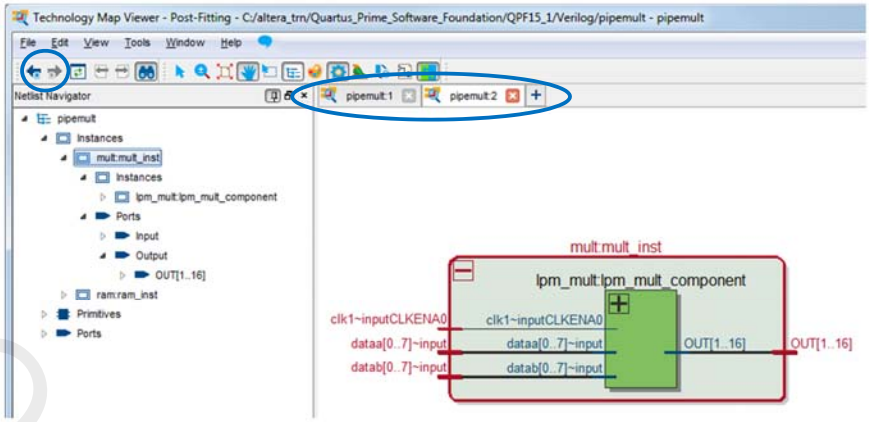
Using the Netlist Navigator



Expanding instances shows blocks within including:
- **Instances**
- **Primitives**
- **Ports**

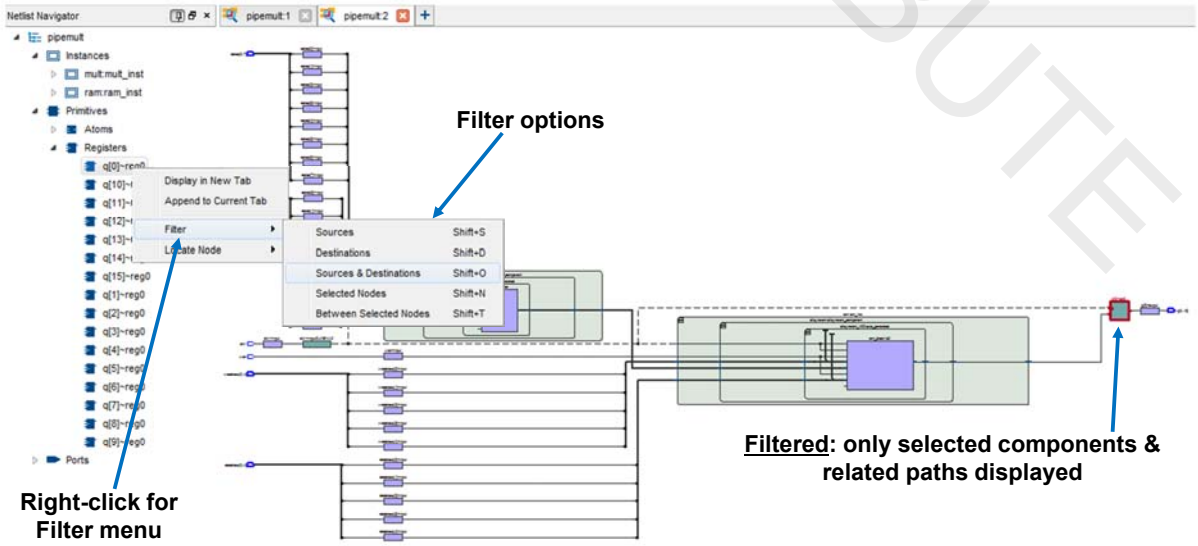
Highlighting netlist element in hierarchy list highlights that element in schematic view

Using View Tabs



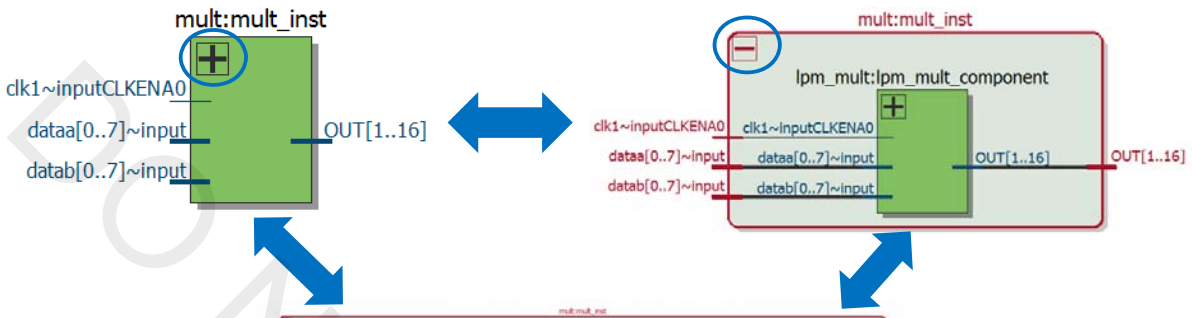
- Open multiple tabs to create and store particular views
- To create new tab:
  - Click **+**
  - Right-click on block in schematic or hierarchy and select **Display in New Tab**
- Tile or cascade tabs to see views simultaneously

Filter Schematic



Instance Unfold/Fold

- Unfold instance by clicking **+** to expand and see logic within
- Fold instance by clicking **-** to hide expanded logic

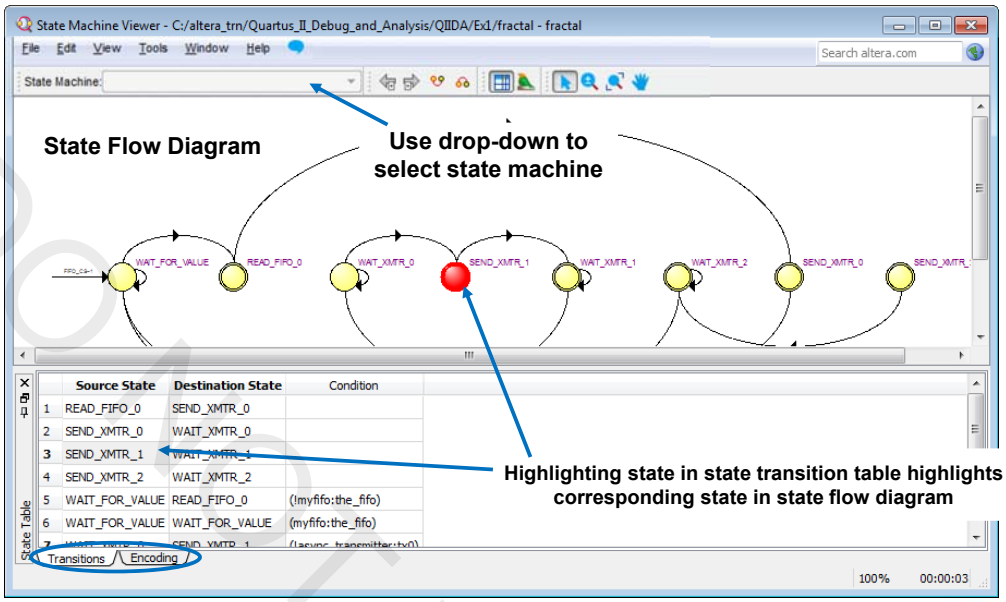


Other Features

- Bird's Eye View: overall panning view of design
- Block properties view (right-click)
  - Fan-in, fan-out, ports, parameters, logic diagram of atom
- Design search
- Lookup table (LUT) internal detail
- Cross-probing: locate nodes from/to
  - Design files
  - Assignment Editor
  - Chip Planner
  - Resource Property Editor
  - RTL/Technology Map Viewers
  - Pin Planner
  - TimeQuest timing reports
  - SignalTap II logic analyzer

State Machine Viewer

- **Tools** menu → **Netlist Viewers** or **Tasks** window **Compile Design** tasks

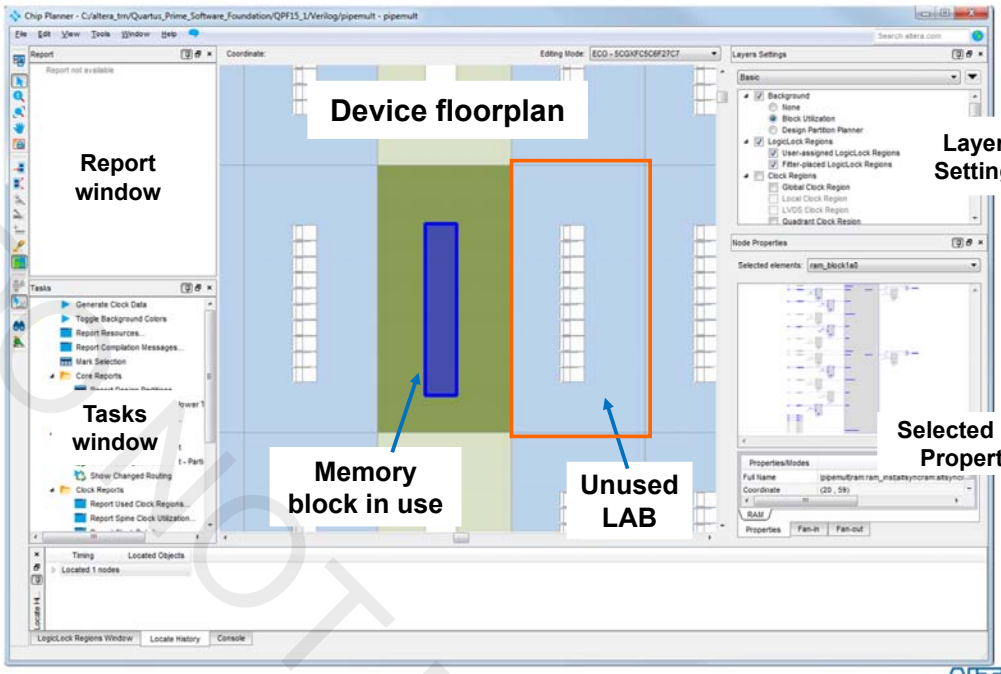


Chip Planner

- Graphical view of design resource usage in target device
- Displays
  - Graphical layout of device resources
  - Routing channels between device resources
  - Global clock regions
- Uses
  - View placement of design logic
  - View connectivity between resources used in design
  - Make placement assignments
  - Debugging placement-related issues

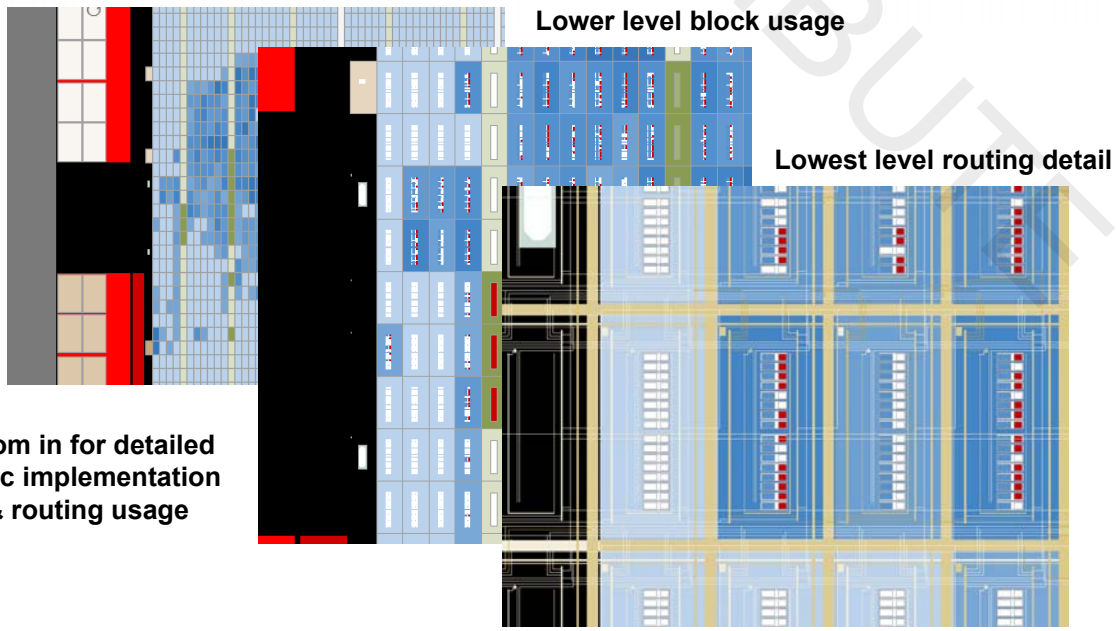
Chip Planner

Tools menu or
Tasks window **Compile Design** tasks



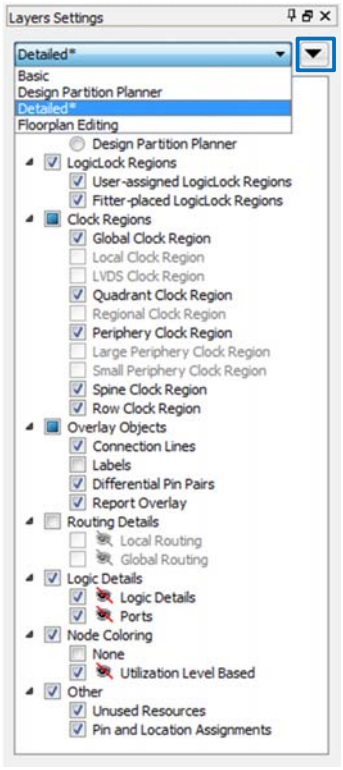
Floorplan Views

Overall device resource usage



Layers Settings

- Quickly change which device resources are displayed
  - e.g. logic details, routing channels
- Default layers
  - Basic
  - Design Partition Planner
  - Detailed
  - Floorplan Editing
- User-defined layers



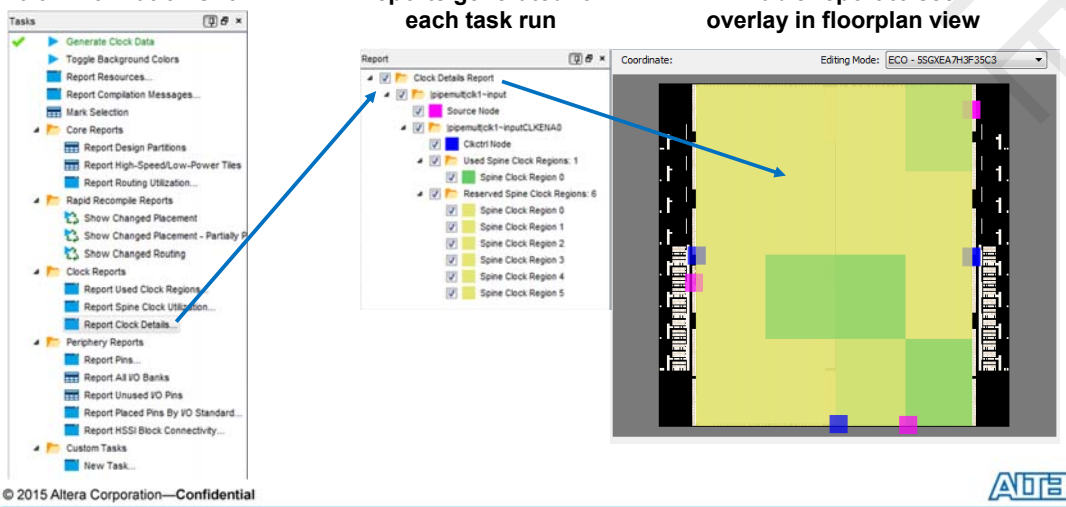
Report & Task Windows

- Run tasks to generate reports that display information about device and device usage
  - e.g. resource location, clock region usage, transceiver connectivity
- Reports displayed as graphical overlays on floorplan

Double-click to run tasks that control information shown

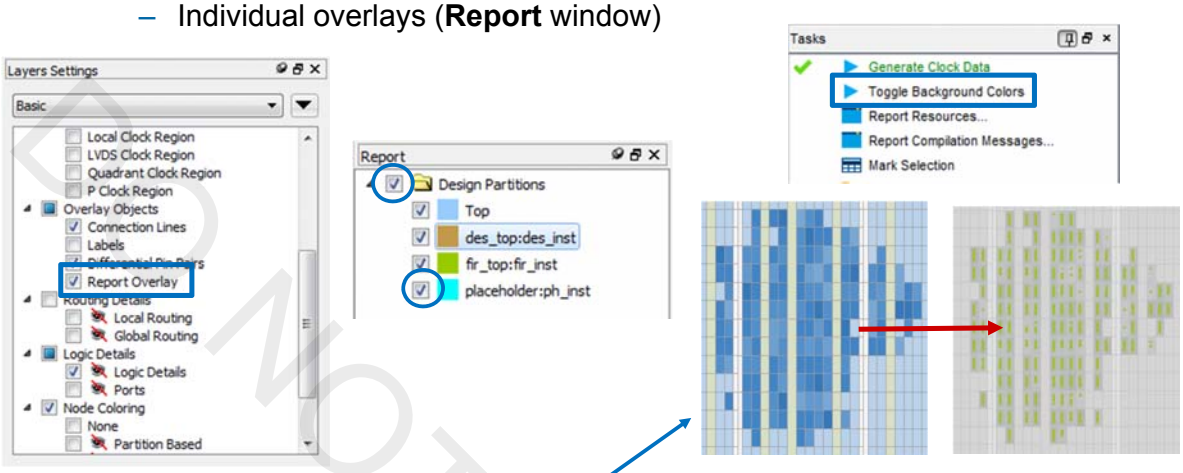
Reports generated for each task run

Enable report to see overlay in floorplan view



Controlling Report Overlay Visibility

- Quickly enable or disable report overlays as needed
  - All overlays (**Layers Settings**)
  - All overlays of a specific type (**Report** window)
  - Individual overlays (**Report** window)

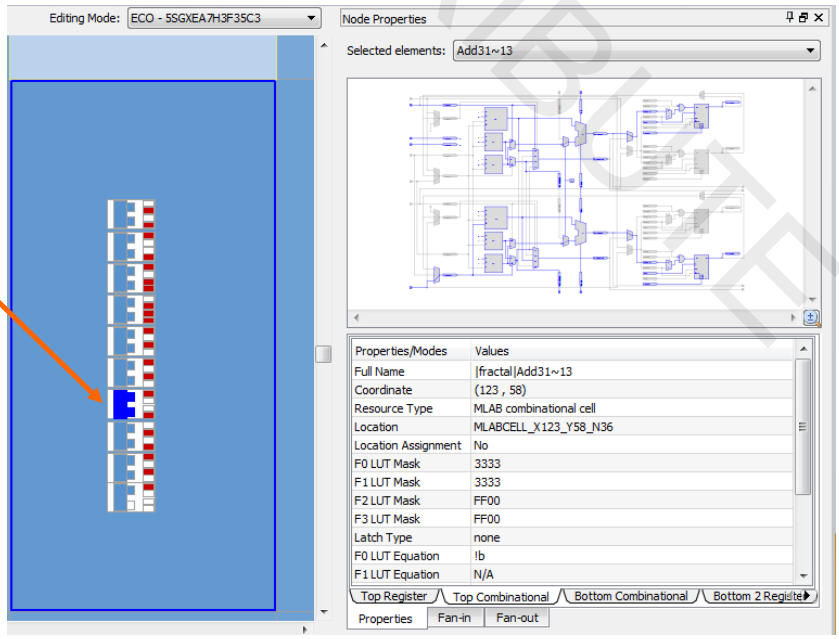


De-emphasize background colors to highlight report

Block Resource Properties

Click any block to view the internal resources detail

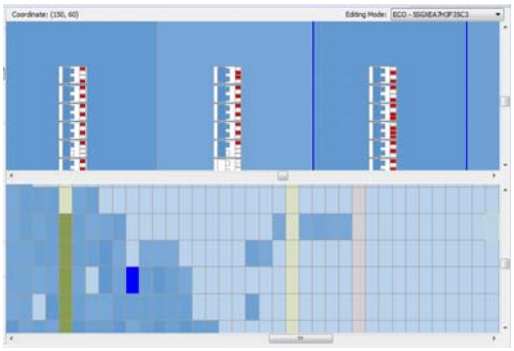
Double-clicking on the resource detail invokes the Resource Property Editor



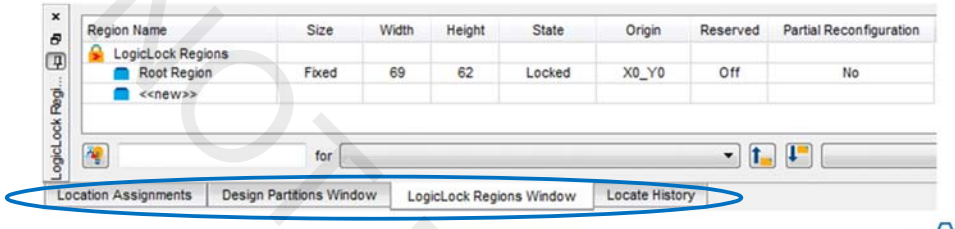
Splitter & Location Assignments

Window menu → **Split Window**

- Split the floorplan into two views
  - Allows for easier drag-and-drop operation
  - View at different zoom levels
- Ability to perform assignments directly in Chip Planner and see the effect in the floorplan

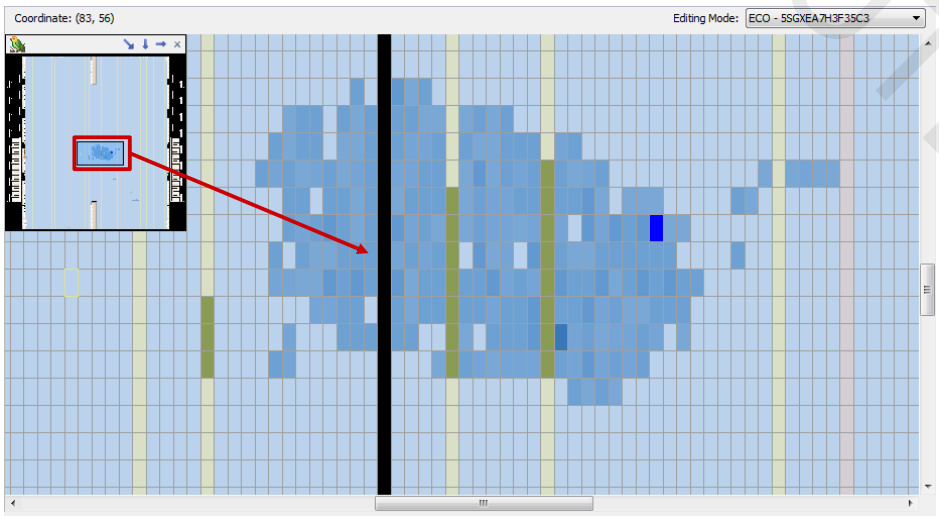


View menu → **Location Assignment Window, LogicLock Regions Window, Design Partitions Window**

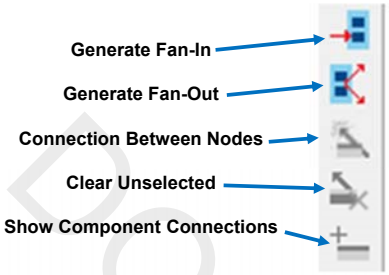


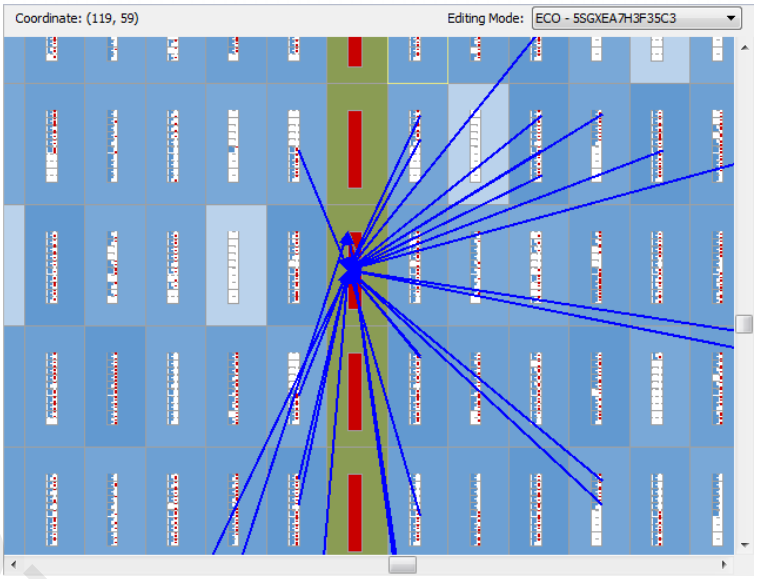
Bird's Eye View

- Provides view of the entire device
- Navigate quickly through the floorplan



Displaying Fan-In & Fan-Out



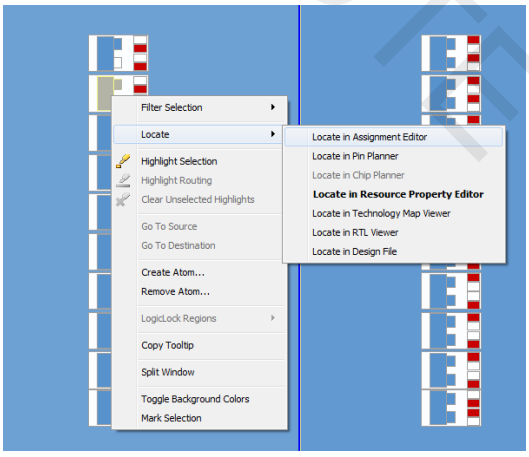


1. Highlight target blocks
2. Click fan-in toolbar button
3. Highlight target blocks again
4. Click fan-out toolbar button

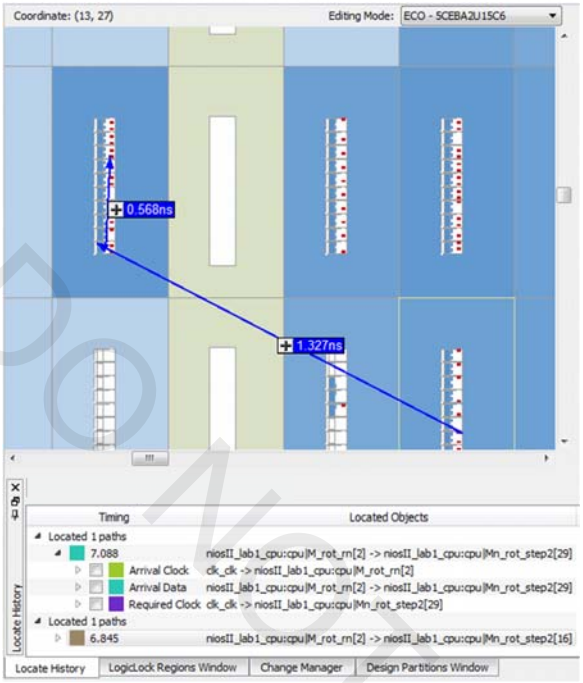
Cross-Probing from/to Chip Planner

- Locate hierarchy blocks or specific logic from/to other Quartus Prime tools
  - Project Navigator
  - Compilation Report
  - Design files
  - RTL Viewer
  - Technology Map Viewers
  - Messages window
  - Pin Planner
  - TimeQuest reports
  - Resource Property Editor

Right-click node & Locate in…



Cross-Probe Example



- Locate timing path from TimeQuest timing analyzer
  - Use placement to understand path timing
- Keep track of located paths using **Locate History** tab
- Quickly highlight (or re-highlight) entire or portion of timing path
- View actual signal routing path

Chip Planner View menu → **Locate History**

Why Did I Receive Undesired Results?

- Incorrect coding
- Non-recommended coding style
- Suboptimal synthesis & fitting settings/constraints (*discussed next*)

- Use tools described to find problems and help fix them

Note: For more details on optimizing designs based on undesired results, please attend the course
Timing Closure with the Quartus II Software

Test Your Knowledge: Compilation

1. The RTL Viewer displays elaboration results and functional blocks, while the Technology Map Viewer displays the actual hardware implementation.
   - ❑ True
   - ❑ False

2. Which of the following is *not* a method for reducing compilation time in the Quartus Prime software?
   a. Flat compilation
   b. Parallel (multi-processor) compilation
   c. Rapid recompilation
   d. Incremental compilation

Exercise 3

Compilation Summary

- Compilation includes synthesis & fitting
- Compilation Report contains detailed information on compilation results
- Use Quartus Prime software tools to understand how design was processed
  - Dynamic Synthesis Report
  - RTL Viewer
  - Technology Map Viewers
  - State Machine Viewer
  - Chip Planner

Compilation Support Resources

- Quartus Prime Handbook chapters
  - *Quartus Prime Incremental Compilation for Hierarchical & Team-Based Design* (Volume 1; Standard Edition Handbook only)
  - *Best Practices for Incremental Compilation and Floorplan Assignments* (Volume 1; Standard Edition Handbook only)
  - *Quartus Prime Integrated Synthesis* (Volume 1; Standard Edition Handbook only)
  - *Optimizing the Design Netlist* (Volume 1)
  - *Analyzing and Optimizing the Design Floorplan with the Chip Planner* (Volume 2)
  - *Engineering Change Orders with the Chip Planner* (Volume 2)
- Training courses
  - Introduction to Incremental Compilation (online)
  - Using the Quartus II Software: Chip Planner (online)
  - Design Optimization Using Quartus II Incremental Compilation (instructor-led)
  - Timing Closure with the Quartus II Software (instructor-led)

The Quartus Prime Software: Foundation

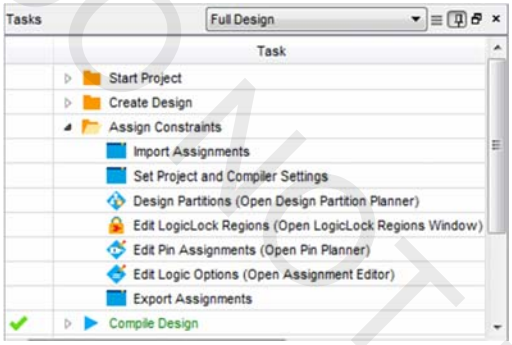
Settings & Assignments

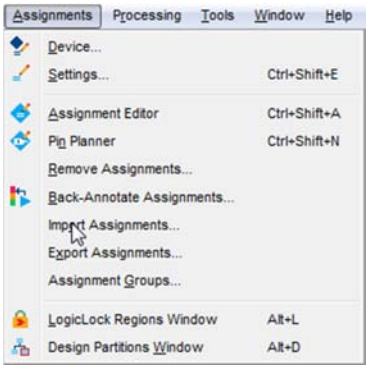
Setting & Assignments Objectives

- Define the difference between settings and assignments
- List some examples of settings and assignments that can appear in a Quartus Prime project
- Create settings and assignments using various methods in the Quartus Prime software

Synthesis & Fitting Control

- Controlled using two methods
  - **Settings**: project-wide switches
  - **Assignments**: individual entity/node controls
- Both accessed in **Assignments** menu or **Tasks** window
- Stored in **.qsf** file for project/revision
- Timing constraints stored in separate **.sdc** file
  - Discussed in online and instructor-led TimeQuest classes

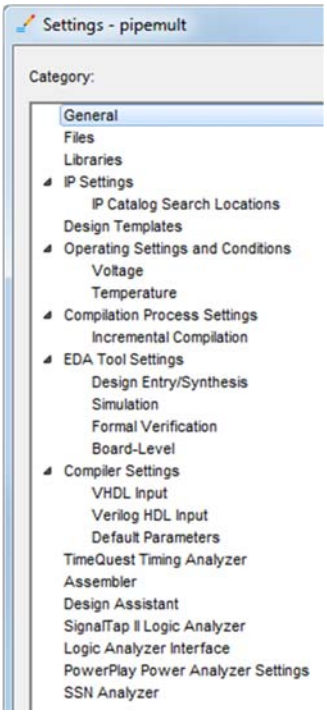




Settings

- Project-wide switches that affect entire design
- Examples
  - Device selection
  - Synthesis optimization
  - Fitter settings
  - Physical synthesis
  - Design Assistant
- Located in **Device** and **Settings** dialog boxes
  - **Assignments** menu
  - **Set Project and Compiler Settings** task in **Tasks** window
- Some options may be different or unavailable in the Pro Edition

Settings Dialog Box



Change settings

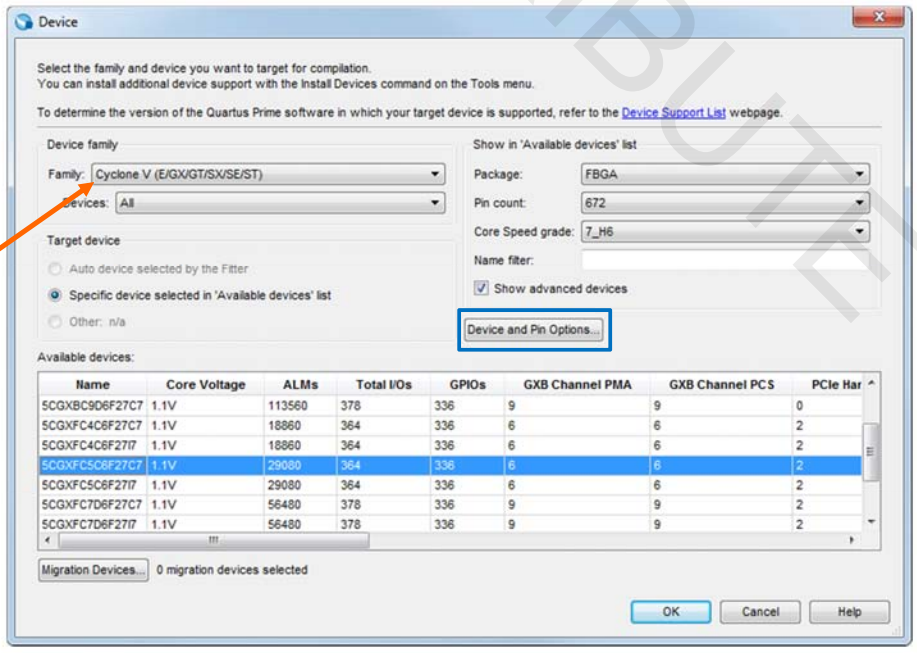
- Top-level entity
- Add/remove files
- Libraries
- Compiler settings
- EDA tool settings
- Synthesis settings
- Fitter settings
- Timing analyzer settings
- Power analysis settings

Tcl: set_global_assignment –name <assignment_name> <value>

Device Settings

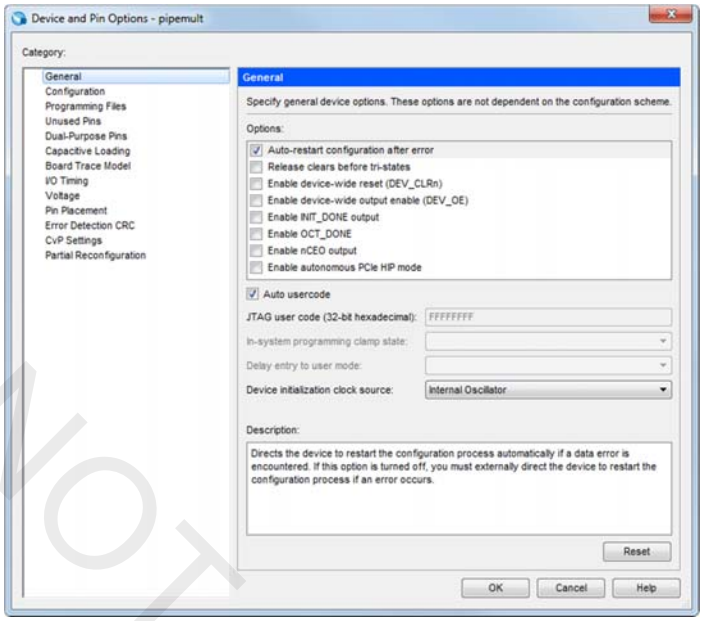
Assignments menu → **Device**

Device family must be installed along with Quartus Prime

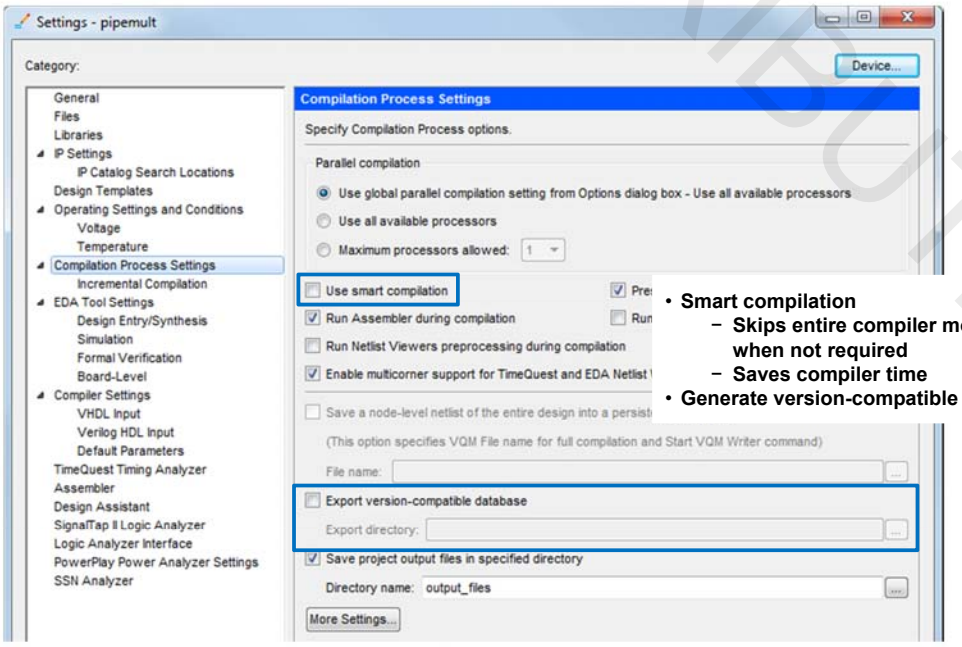


Device and Pin Options

- Hardware-specific feature settings



Compilation Process Setting Examples



- **Smart compilation**
  - **Skips entire compiler modules when not required**
  - **Saves compiler time**
- **Generate version-compatible database**

Tcl: set_global_assignment -name SMART_RECOMPILE ON

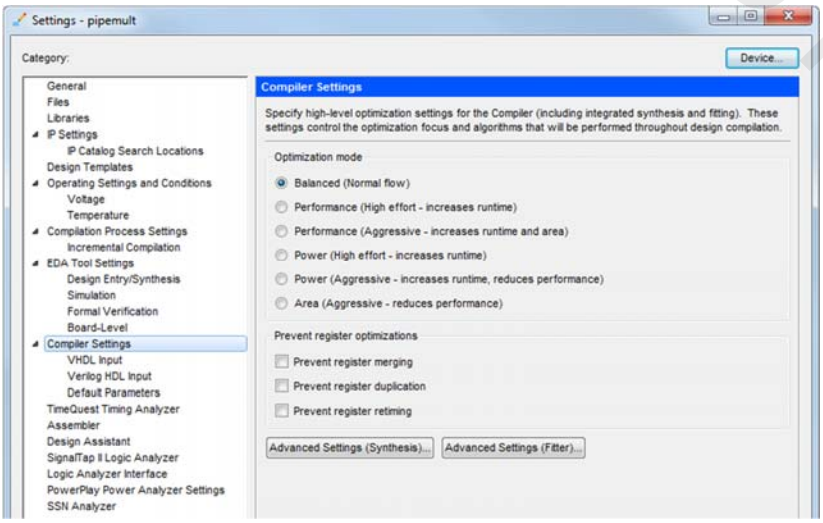
Version-Compatible Database

- For simplifying process of migrating design between versions of Quartus Prime software
- Exports special database (**export_db** folder)
- Preserve compilation results between Quartus Prime software versions
  - Re-run timing analysis or simulation with updated timing models
  - No need to fully recompile (until changes are made that require it)
- Two methods to create
  - **Settings** dialog box
  - **Project** menu (**Export Database**)

Tcl: set_global_assignment –name AUTO_EXPORT_VER_COMPATIBLE ON
Tcl: set_global_assignment –name VER_COMPATIBLE_DB_DIR <directory_name>

Consolidated Compiler Optimization Settings

- Choose general options for how compiler should optimize design
- Set specific optimization features under **Advanced Settings** buttons



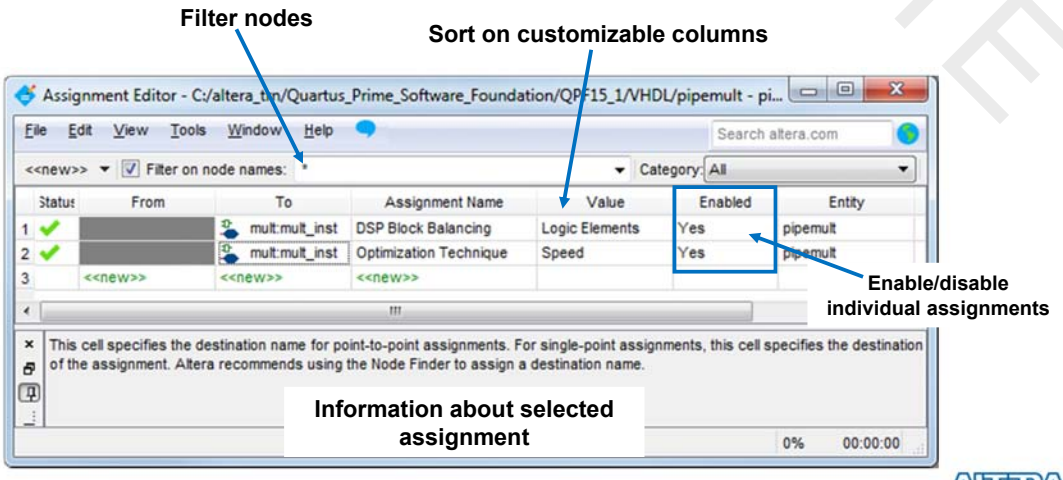
Assignments (aka Logic Options, Constraints)

- Individual switches applied
  - I/O
  - Internal nodes
  - Hierarchical blocks (design entities)
- Assignment Editor manages assignments
- Must perform analysis & elaboration
- Example assignments
  - Optimization Technique
  - PCI I/O

Assignment Editor

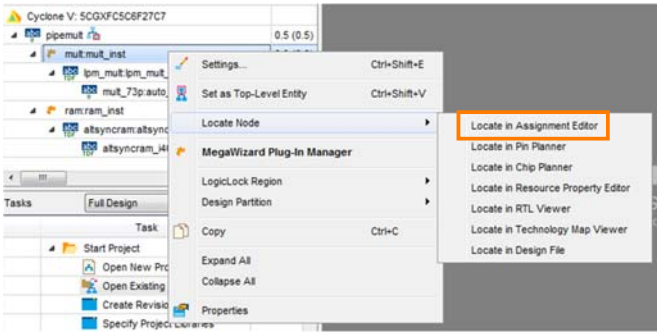
- Provides spreadsheet assignment entry & display
  - Copy & paste support
  - Multi-cell editing

Assignments menu or **Tasks** window



Creating Assignments: Cross-Probing

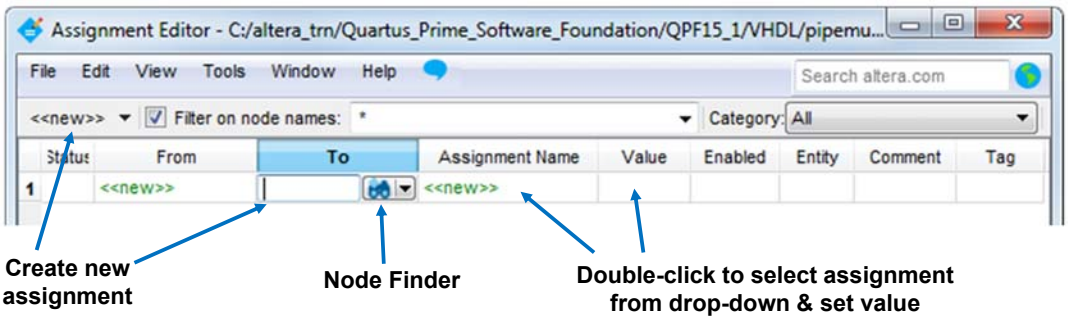
- Cross-probe (**Locate Node**) to Assignment Editor from:
  - Project Navigator
  - Message window
  - Compilation Report
  - Design files



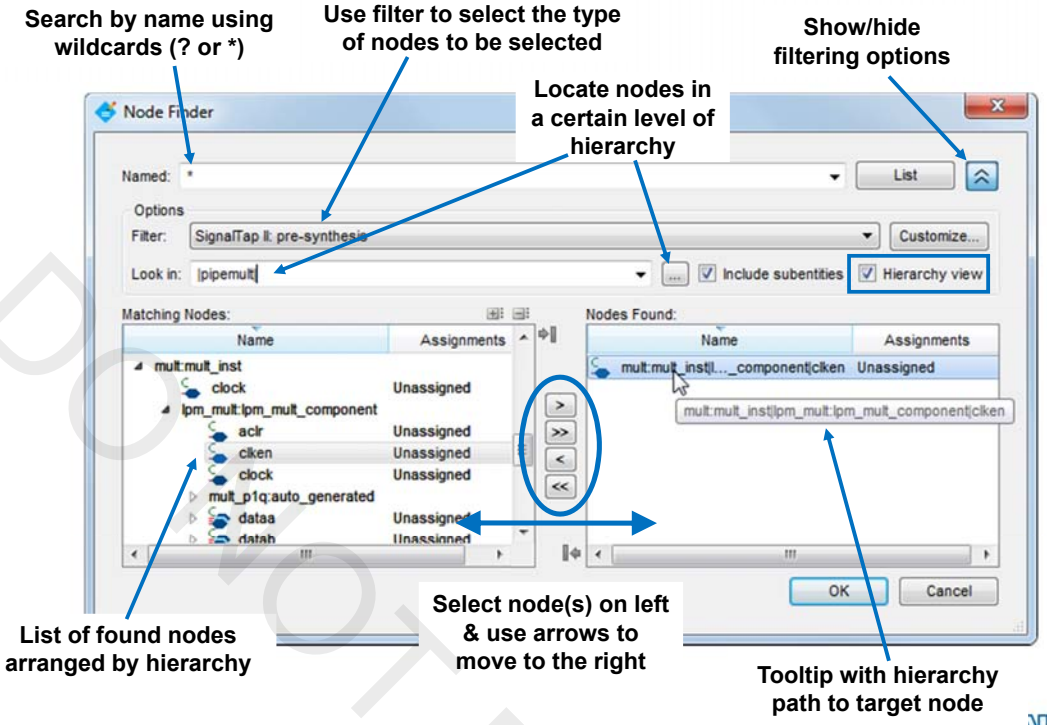
- Assignment Editor Node Filter automatically filled in with cross-probed node(s)
- Easiest method for creating assignments

Creating Assignments: Assignment Editor

- Cross-probe from other tools to fill in filter
- Click on **<<new>>** button to create assignment for cross-probed node or entity
- Double-click **<<new>>** in the **From** or **To** columns to create a new assignment from scratch
  - Manually type in object name
  - Or click **Node Finder** icon to search

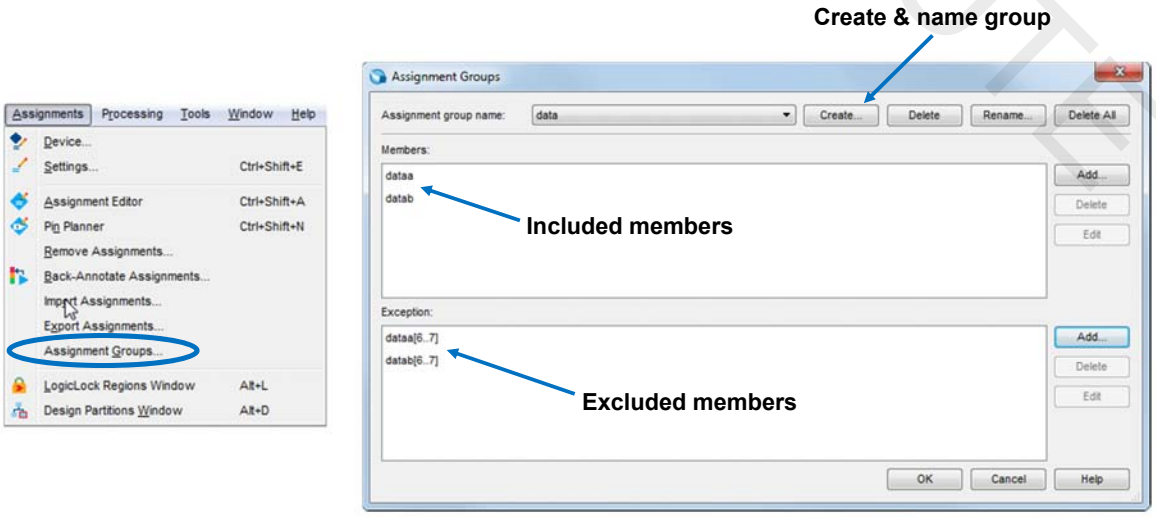


Node Finder



Assignment Groups

- Assign names to user-defined groups of nodes
- Allows single assignment to constrain entire group



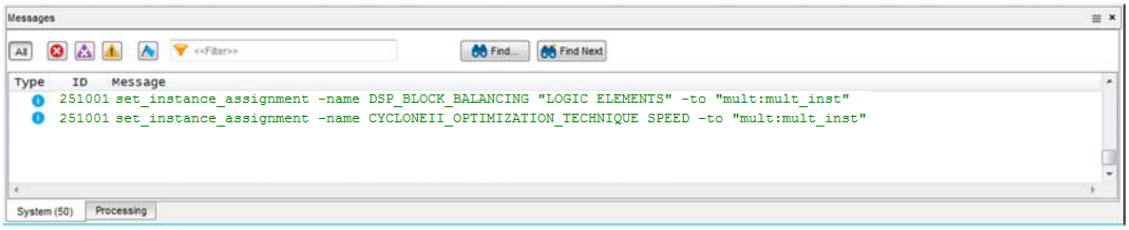
Updating **.qsf** File

- By default, **.qsf** not updated automatically when constraint entered
- **.qsf** updated only when
  - Project is saved or closed (**File** menu)
  - Assignment Editor is saved or closed
  - Beginning of any processing task (e.g. compilation)
  - Any IP parameter editing tool is launched
  - Changing revisions
- Change behavior to update assignments immediately (**Tools** menu → **Options** → **General** → **Processing**)
  - May impact software performance slightly due to file accesses

Assignment Tcl Commands

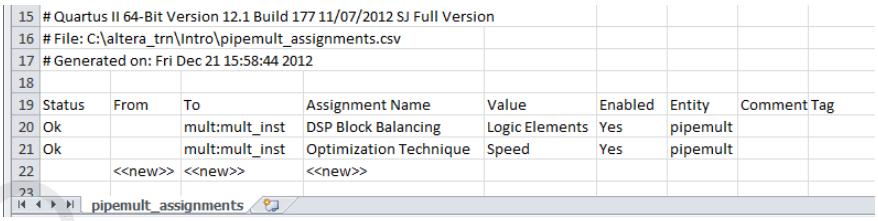
- Equivalent Tcl commands displayed as assignments are entered
  - Manually copy to create Tcl scripts
  - Export command (**File** menu) writes all assignments to a Tcl file

Messages window



Export Assignments

- Export to **.csv** file (**File** menu → **Export**)
  - Import and edit assignments in Excel



15	# Quartus II 64-Bit Version 12.1 Build 177 11/07/2012 SJ Full Version							
16	# File: C:\altera_trn\Intro\pipemult_assignments.csv							
17	# Generated on: Fri Dec 21 15:58:44 2012							
18								
19	Status	From	To	Assignment Name	Value	Enabled	Entity	Comment Tag
20	Ok		mult:mult_inst	DSP Block Balancing	Logic Elements	Yes	pipemult	
21	Ok		mult:mult_inst	Optimization Technique	Speed	Yes	pipemult	
22		<<new>>	<<new>>	<<new>>				
23								

pipemult_assignments

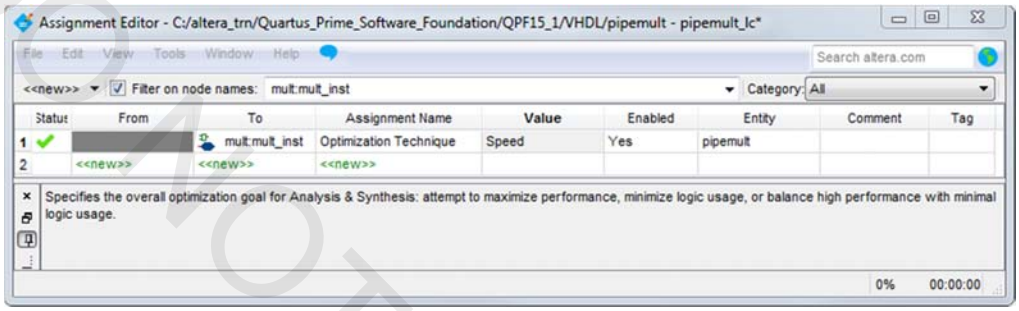
- Export to new **.qsf** (**Assignments** menu → **Export Assignments**)
  - Migrate assignments to another project

Example Assignments

- Optimization Technique
- PCI I/O

Optimization Technique

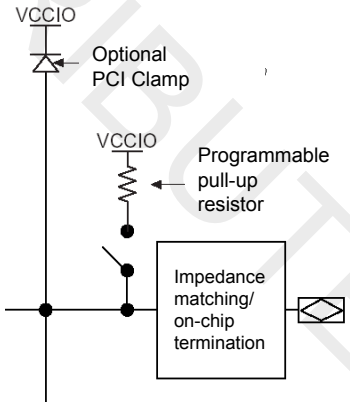
- Selects synthesis optimization goal (***Speed***, ***Balanced***, ***Area***)
- Apply to hierarchical entities
  - Locate (cross-probe) from Project Navigator
  - Or drag and drop into Assignment Editor
- May also apply project-wide in Advanced Synthesis Settings
- Affects synthesis & logic mapping; Quartus Prime synthesis only

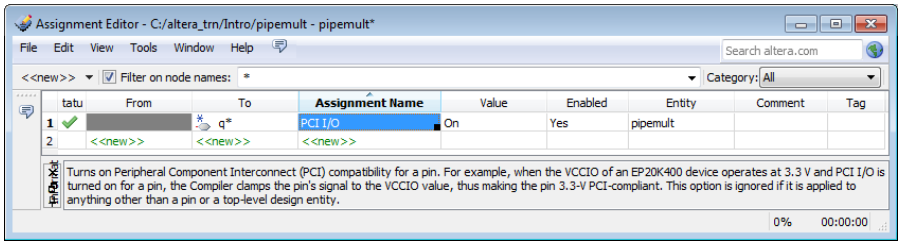


Tcl: set_instance_assignment –name OPTIMIZATION_TECHNIQUE SPEED –to <node name>

PCI I/O

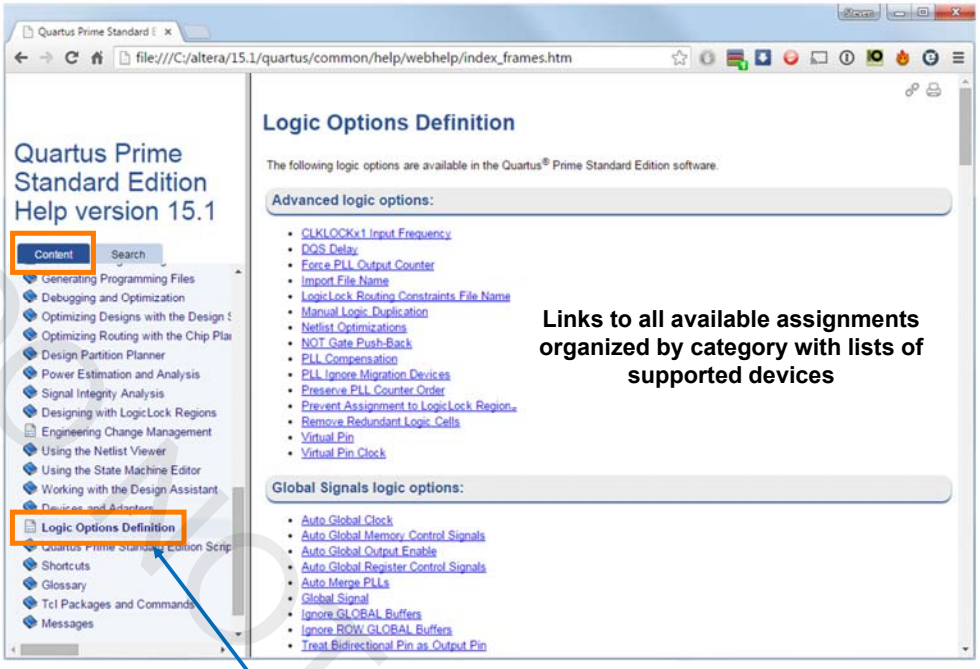
- Turns on PCI™ compatibility for pins
  - Ignored if applied to anything other than a pin or a top-level design entity
- Controls clamping diode located in the I/O elements
  - For applicable families





Tcl: set_instance_assignment –name PCI_IO ON –to <pin name>

Available Logic Options (Assignments)



Links to all available assignments organized by category with lists of supported devices

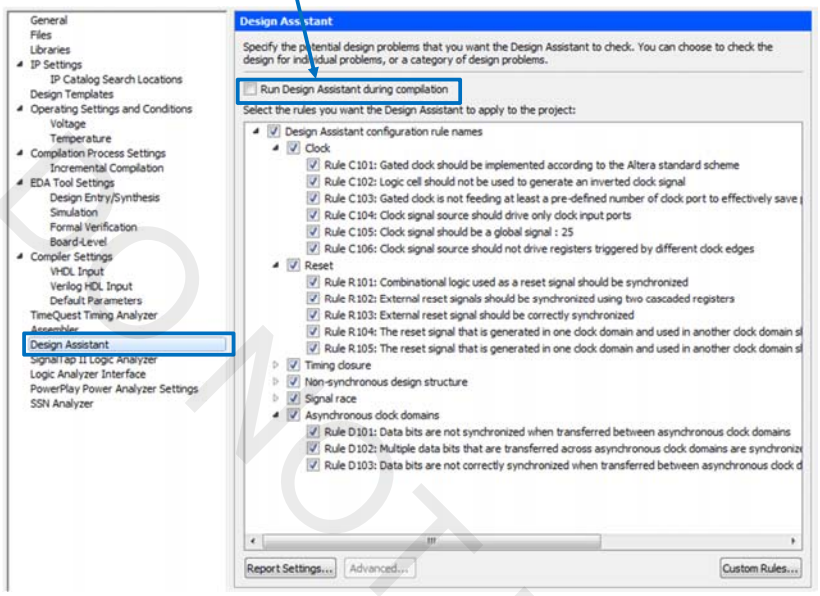
Select "Logic Options Definition" in Quartus Prime Help Contents

Design Assistance

- Design Assistant
  - Check design against sets of good design practice rules
- Advisors
  - Check current settings and assignments
  - Make recommendations for different optimization goals

Design Assistant

- Run from **Processing** menu or **Tasks** window
- Enable to execute automatically during compilation

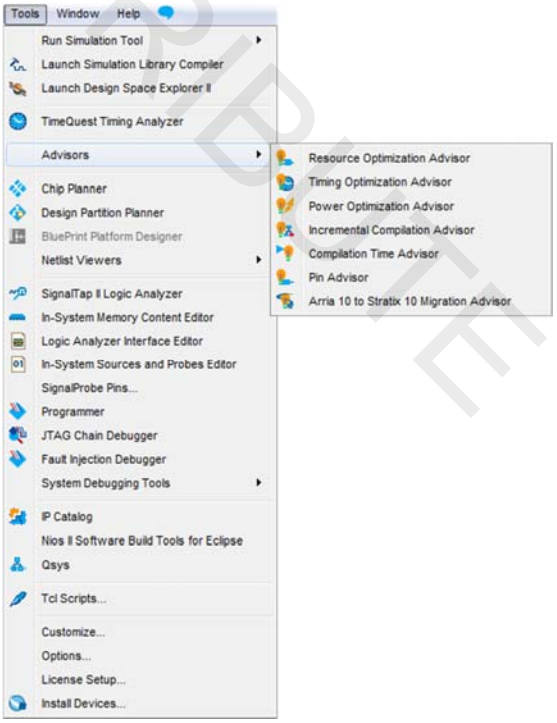


Design rule areas:

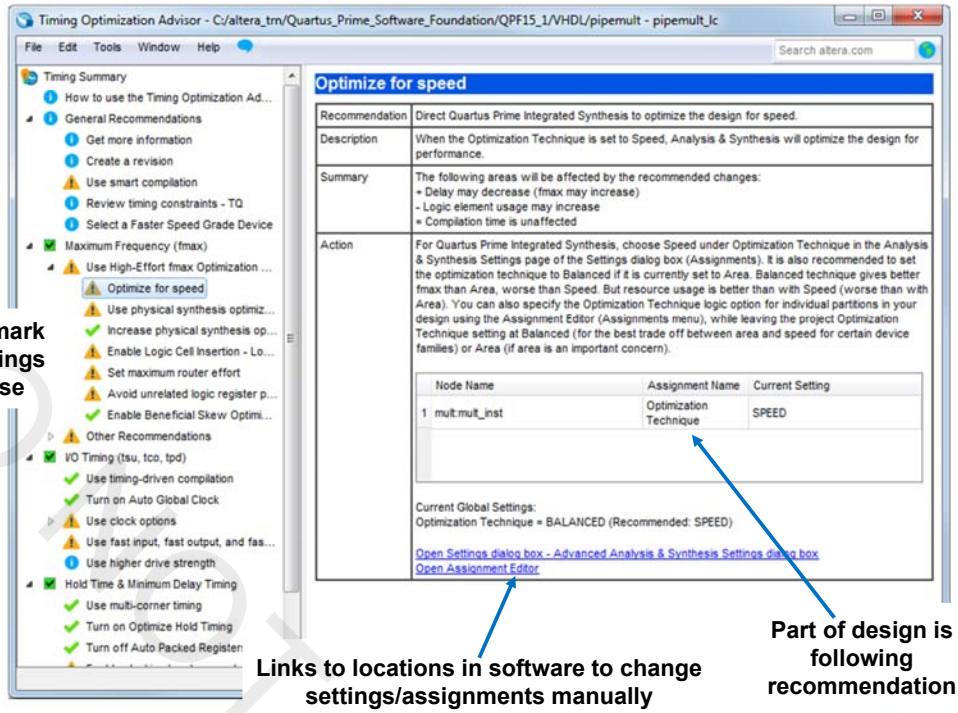
- Clocks
- Reset
- Timing closure
- Non-synchronous design structure
- Signal race conditions
- Asynchronous clock domain data transfers

Advisors

- Provide project-specific recommendations (feedback) on optimizing designs
- Access through **Tools** menu or **Tasks** window
- Seven types
  - Resource usage optimization
  - Timing (performance) optimization
  - Power optimization
  - Incremental compilation suggestions
  - Compilation time reduction
  - I/O-related recommendations
  - Arria 10 to Stratix 10 migration
- Recommendations can contradict each other between different advisors



Advisor Example



Green checkmark indicates settings already in use

Links to locations in software to change settings/assignments manually

Part of design is following recommendation

Test Your Knowledge: Settings & Assignments

1. Settings are individual switches that are applied to I/O, internal nodes, or hierarchical blocks (design entities), while assignments affect the entire design.
   - ❑ True
   - ❑ False

2. Optimization technique is a(n):
   a. Assignment
   b. Setting
   c. Both an assignment and a setting
   d. Neither an assignment nor a setting

3. The selection of a target device is a(n):
   a. Assignment
   b. Setting
   c. Both an assignment and a setting
   d. Neither an assignment nor a setting

Exercise 4

Settings & Assignments Summary

- Settings & assignments allow a designer to control how a design is synthesized and placed & routed
- Use the Settings dialog box to adjust project-wide settings
- Use the Assignment Editor to enable/disable individual assignments targeting hierarchical blocks, internal nodes, or I/O
- Design Assistant & Optimization Advisors help improve design results through design rule checking and settings recommendations

Design Analysis Support Resources

- Quartus Prime Handbook chapters (all Volume 2)
  - *Constraining Designs*
  - *Design Optimization Overview*
  - *Reducing Compilation Time*
  - *Timing Closure & Optimization*
  - *Power Optimization*
  - *Area Optimization*
  - *Netlist Optimizations & Physical Synthesis*
- Training courses
  - The Quartus II Software Design Series: Timing Analysis (instructor-led)
  - Advanced Timing Analysis with TimeQuest (instructor-led)
  - Timing Closure with the Quartus II Software (instructor-led)
  - TimeQuest Timing Analyzer
  - Timing Closure Using Quartus Advisors and Design Space Explorer
  - Timing Closure Using Quartus II Physical Synthesis Optimizations
  - Resource Optimization
  - Power Optimization and Analysis

The Quartus Prime Software: Foundation

I/O Planning

I/O Planning Need

- I/O standards increasing in complexity
- FPGA/CPLD I/O structure increasing in complexity
  - Results in increased pin placement guidelines
- PCB development performed simultaneously with FPGA design
- Pin assignments need to be verified earlier in design cycle

Creating I/O-Related Assignments

- Pin Planner
- BluePrint Platform Designer
  - For Arria 10 and newer devices
- Import from spreadsheet in **.csv** format
- Type directly into **.qsf** file
- Directly in HDL code
- Scripting

***Note:** Other methods/tools are available in the Quartus Prime software to make I/O assignments. The above are the most common or recommended.*

The Quartus Prime Software: Foundation

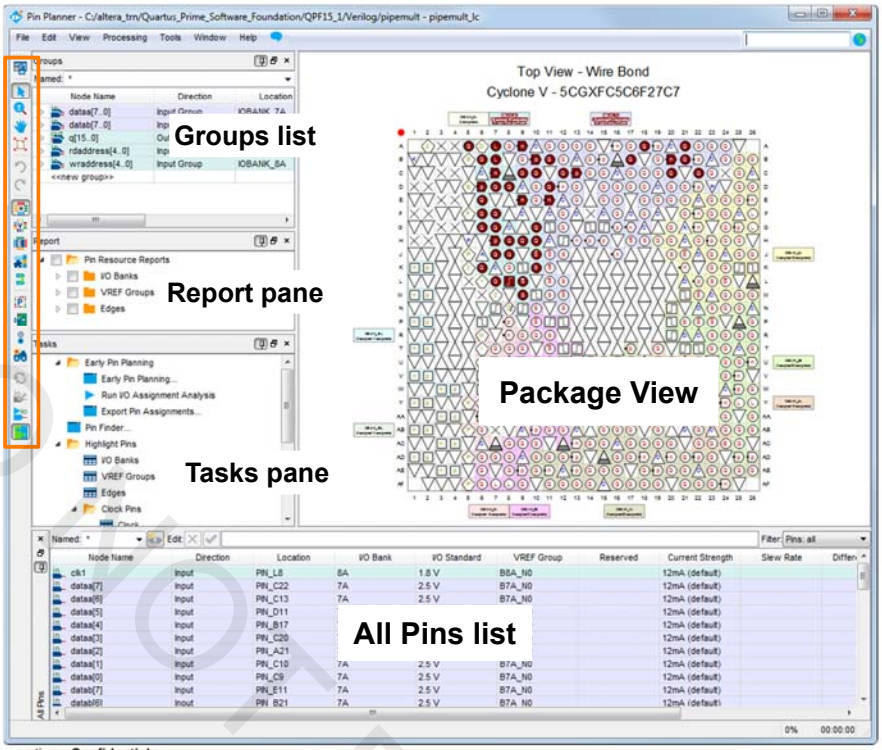
I/O Planning with the Pin Planner

Pin Planner

- Interactive graphical tool for assigning pins
  - Drag & drop pin assignments
  - Set pin I/O standards
  - Reserve future I/O locations
- Default window panes
  - Package View
  - All Pins list
  - Groups list
  - Tasks window
  - Report window

Assignments menu → **Pin Planner** or
Assign Constraints folder in **Tasks** window

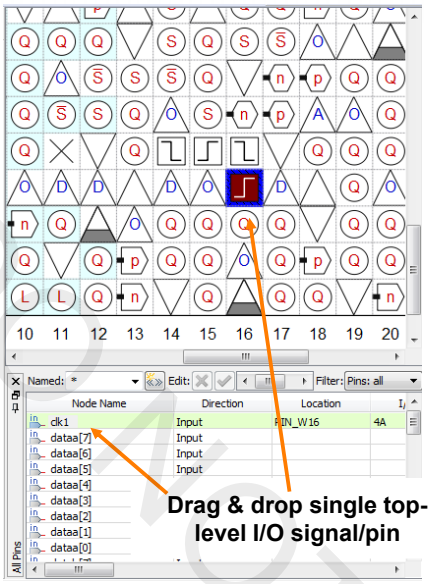
Pin Planner Window



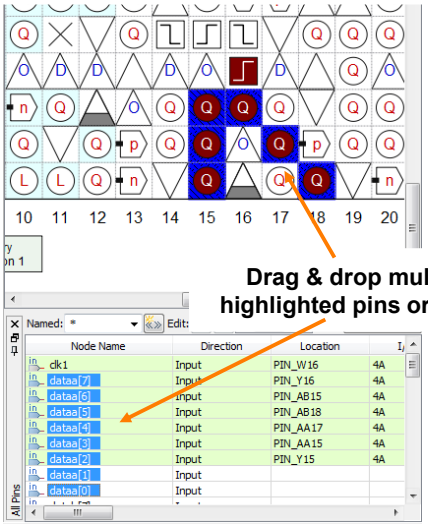
Pin Planner Window Panes

- Package View
  - Displays graphical representation of chip package
  - Locate, make, or edit I/O assignments
- All Pins list
  - Displays I/O pins (signals) in design
  - Edit pin assignments
- Groups list
  - Similar to All Pins list displaying only groups & buses
  - Make bus and group assignments
  - Create new user-defined groups
- Tasks pane
  - Perform tasks such as Early Pin Planning and pin highlight reports
- Report Pane
  - Quickly enable/disable reports generated in the Tasks pane

Assigning Pin Locations Using Pin Planner



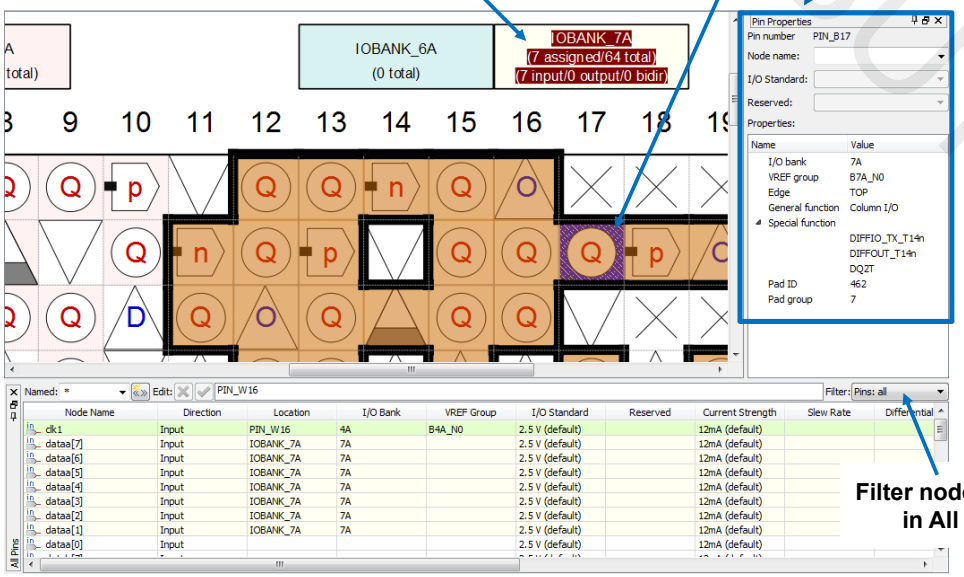
Choose one-by-one or pin alignment direction (Edit menu)



Assigning Pin Locations Using Pin Planner (2)

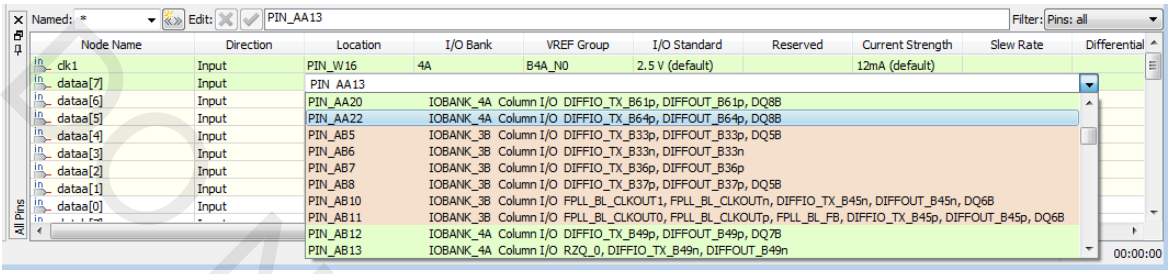
Drag & drop to I/O bank, VREF block, or device edge

Click pin or I/O bank to display pin properties



Assigning Pin Locations Using Pin Planner (3)

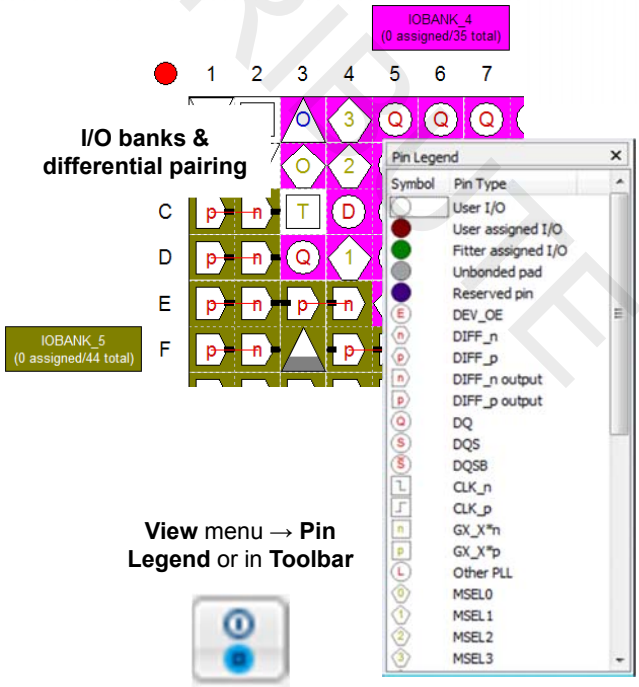
- Select available locations from list of pins color-coded by I/O bank



Pin Legend and Pin Planner Views

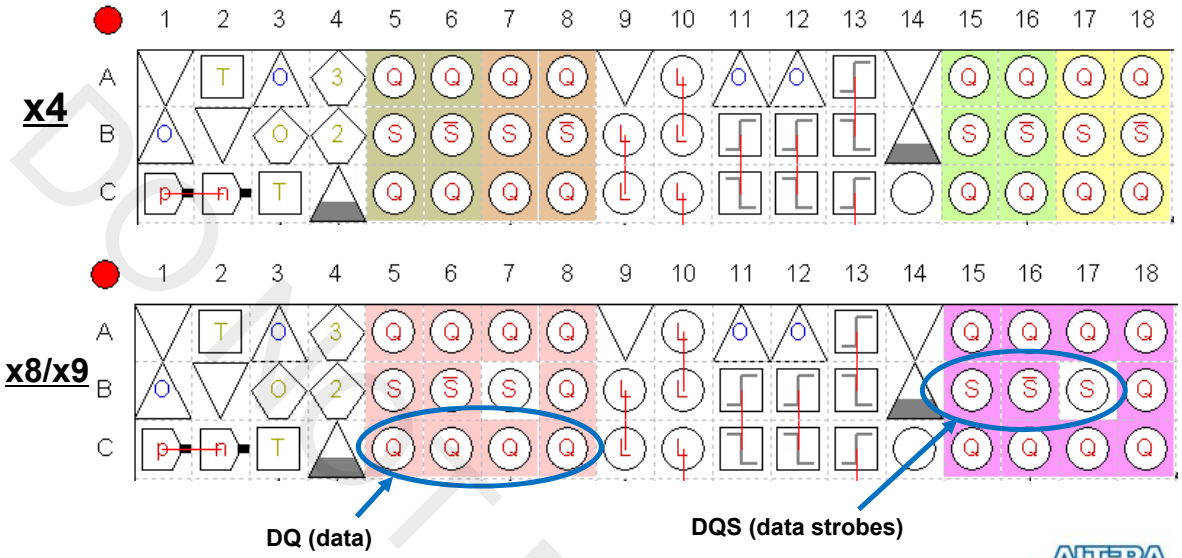
- Displays (**View** menu → **Show**, Toolbar buttons, or right-click in Package View)
  - Device edges
  - I/O banks
  - VREF groups
  - Differential pin pairing
  - DQ/DQS and hard memory interface pins
  - PCIe hard IP pins
  - Clock region inputs

- Easy-to-read pin legend

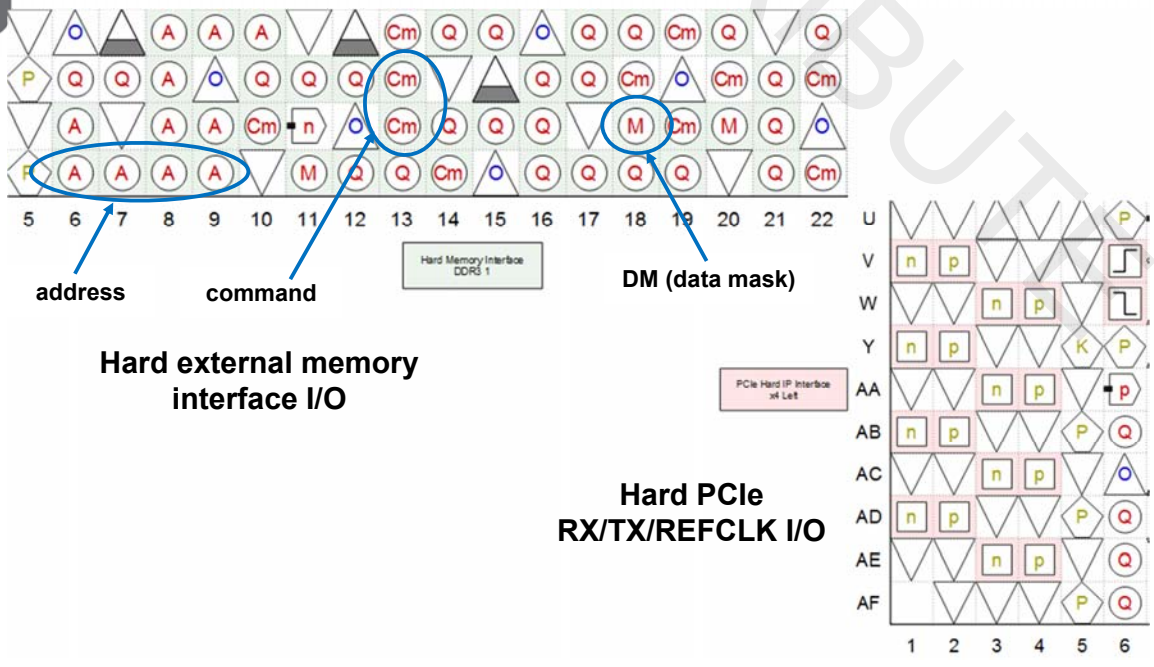


Example View: Show DQ/DQS Pins

- Show color-coded DQ/DQS sets in x4, x8/x9, x16/x18, or x32/x36 modes in the Package View for DDR interfaces



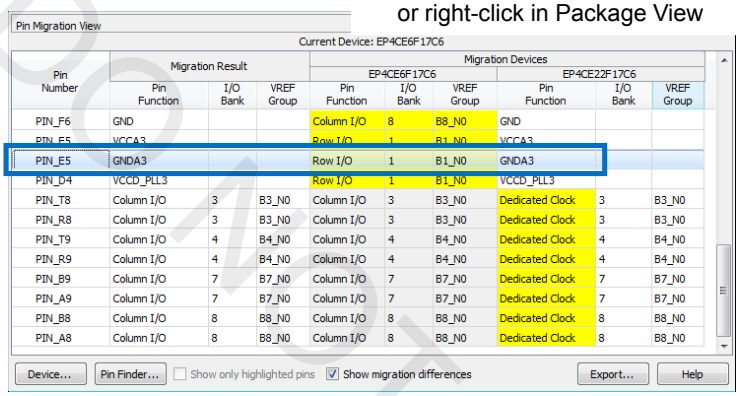
Example Views: Show Dedicated Hard Resources I/O



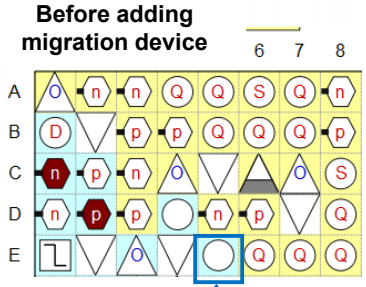
Pin Migration View

- Select migration devices in Device settings
- View & compare pin function differences between migration devices
- Package View adjusts to prevent non-migratable assignments

View menu → **Pin Migration View**
or right-click in Package View



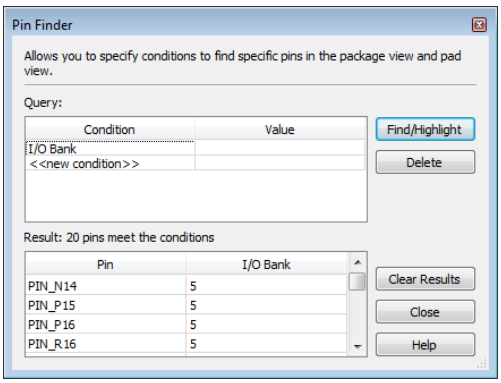
Before adding migration device



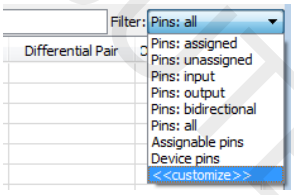
After adding migration device

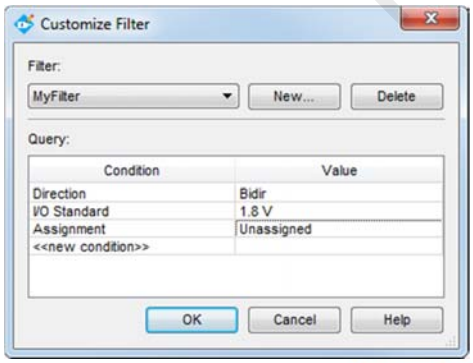
More Pin Planner Features (1)

- Pin Finder
  - Locate pins meeting user-defined criteria
  - Use to find compatible pin locations
  - Pins highlighted in Package View



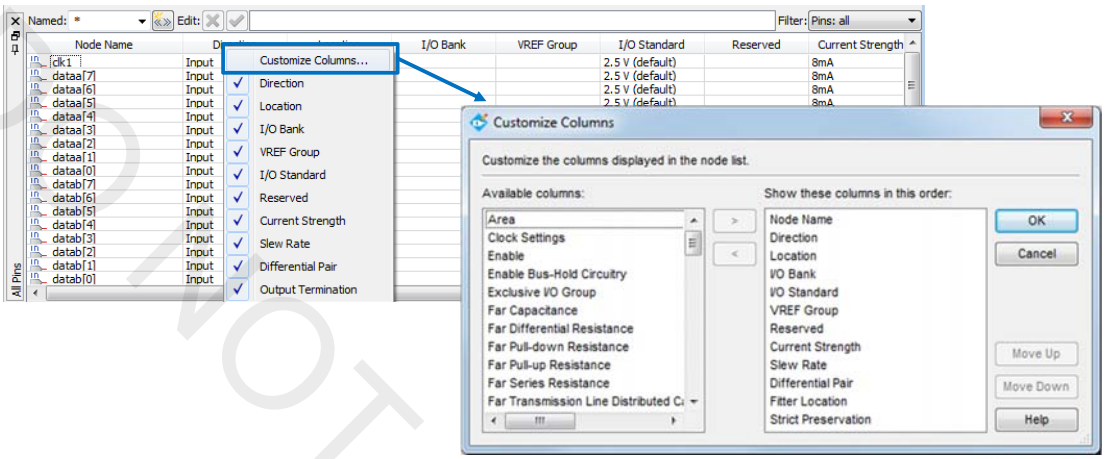
- Custom Filters
  - Create custom filters for All Pins list





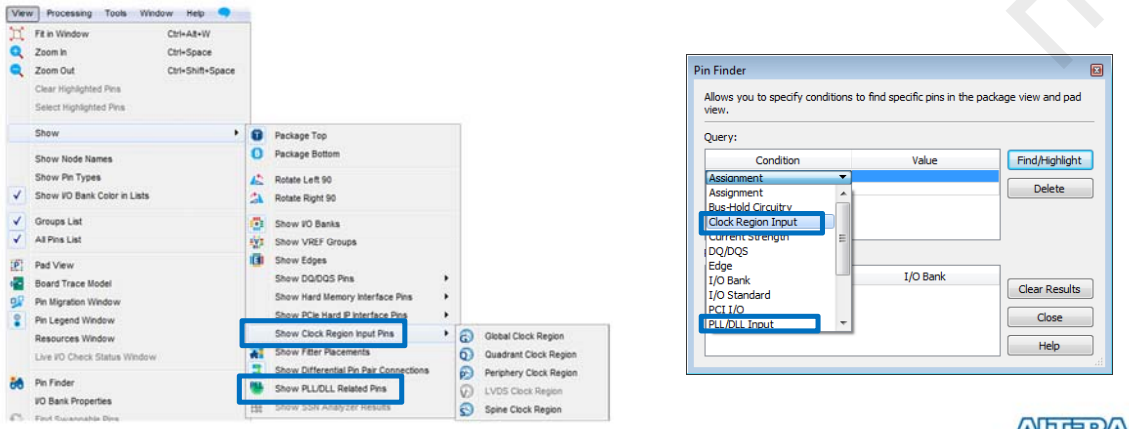
More Pin Planner Features (2)

- Customizable columns
  - Select the I/O-related assignment columns to be shown in All Pins list for easy management
  - Right-click on column heading to select which columns will be displayed



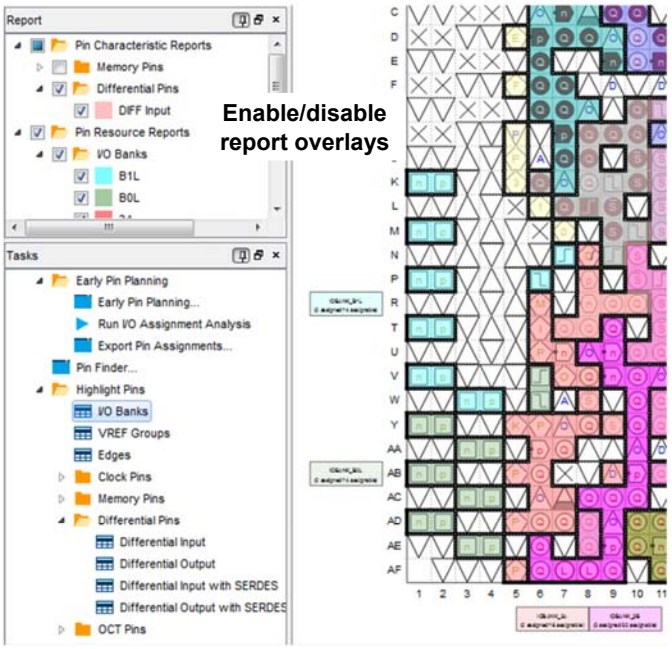
Clock/PLL Views and Support

- **Show Clock Region Input Pins** view
  - Highlights input pins into each type of clock region (global, region, etc.)
- Show PLL/DLL related pins
- Property windows show clock region information
- Pin Finder allows searching of clock region input pins, PLL input/output pins



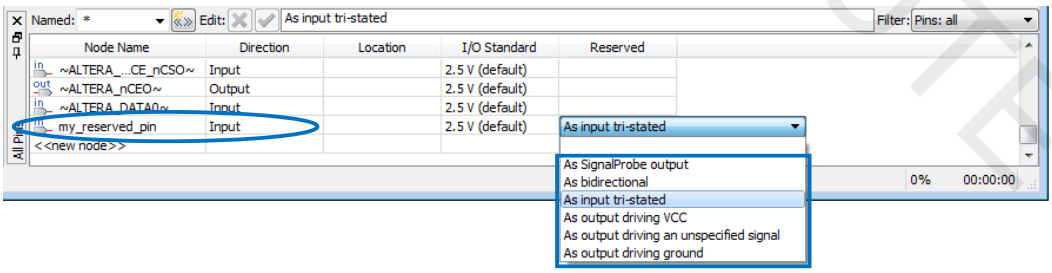
Pin Planner Tasks & Report Windows

- Use **Tasks** window to access common tasks
- **Highlight Pins** tasks will generate colored overlays similar to the Chip Planner
  - Enable/disable reports in Report pane



Reserved and Unused I/O Pins

- Type reserved I/O name into All Pins list & select reserved setting
- Prevents Fitter from placing unassigned signal on pin *(discussed next)*

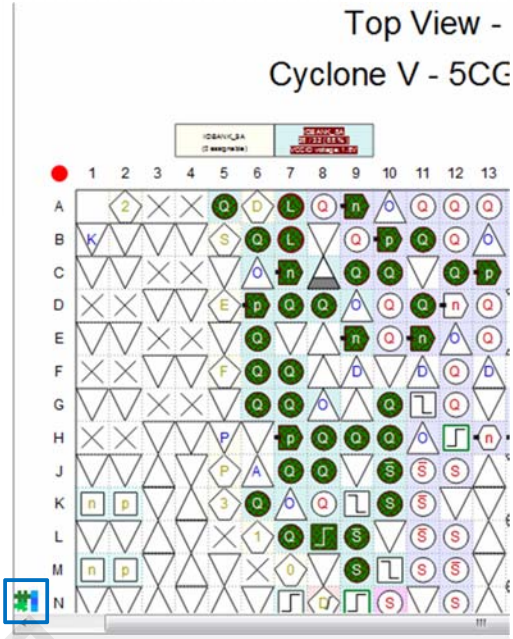


- Or right-click on pin in Package View and click **Reserve Pins → As…**
  - Pin name set to **user_reserve_**<*pin_number*>
- Set initial state of other unused pins in **Device** settings dialog box *(shown later)*

Show Fitter Placements

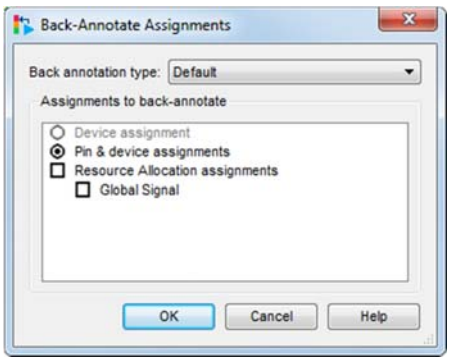
- View I/O locations selected by the Fitter

View → Show or in Toolbar

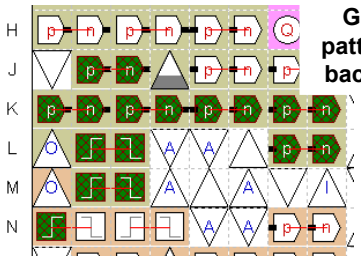


Back-Annotation

Assignments menu in main Quartus Prime window



- Use to lock Fitter-chosen pin assignments for future compilations
  - Copies device & resource locations chosen by fitter into **.qsf** file
    - Pins
    - Logic
    - Routing
- "Locks down" locations in Pin Planner



Green/brown pattern indicates back-annotation

The Quartus Prime Software: Foundation

Verify I/O Assignments with the Pin Planner

Verifying I/O Assignments

- I/O Assignment Analysis
  - Checks legality of all I/O assignments without full compilation
- Minimal requirements for running
  - I/O declaration
    - HDL port declaration
    - Reserved pin
  - Pin-related assignments
    - I/O standard
    - Current strength
    - Pin location (pin, bank, edge)
    - PCI clamping diode
    - Toggle rate

Run from Pin Planner toolbar

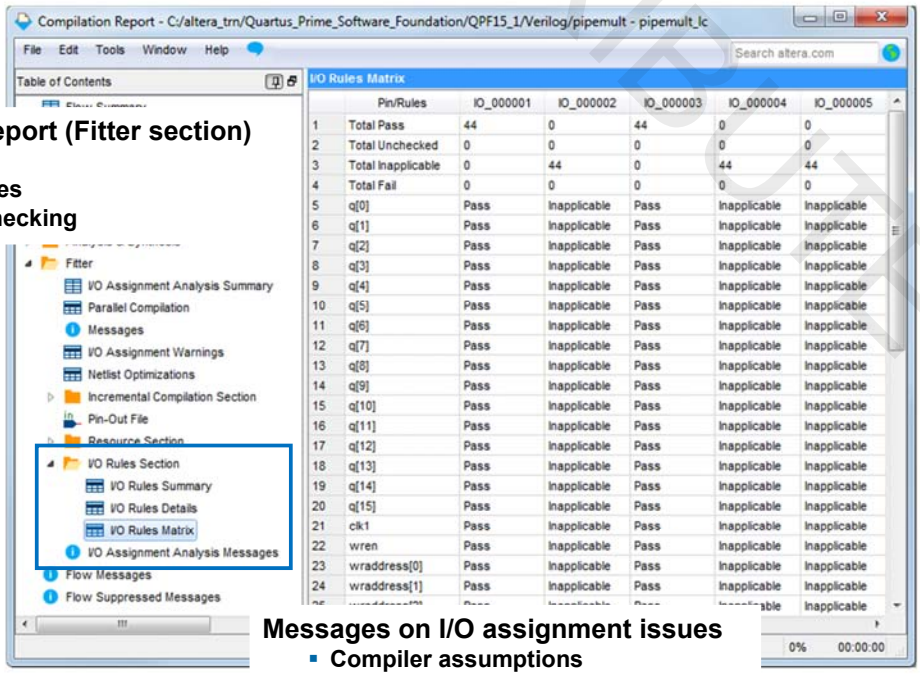
Processing menu → **Start** → **Start I/O Assignment Analysis** or **Tasks** window

I/O Rules Checked

- No internal logic
  - Checks I/O locations & constraints with respect to other I/O & I/O banks
  - e.g. Each I/O bank supports a single V_{CCIO}
- I/O connected to logic
  - Checks I/O locations & constraints with respect to other I/O, I/O banks, & internal resources
  - e.g. PLL must be driven by a dedicated clock input pin

Note: When working with design files, synthesize design before running I/O Assignment Analysis

I/O Assignment Analysis Output



Compilation Report (Fitter section)

- **Pin-out file**
- **I/O pin tables**
- **I/O rules checking**

Messages on I/O assignment issues

- **Compiler assumptions**
- **Device & pin migration issues**
- **I/O bank voltages & standards**

Validating I/O Pin-out

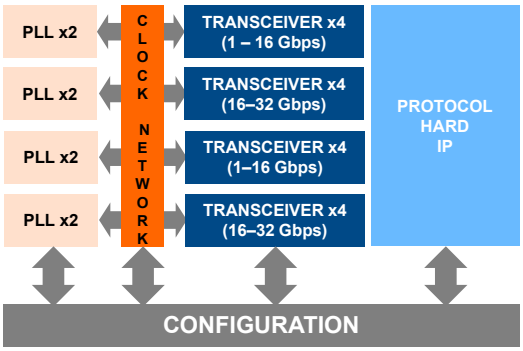
- Completed design
  - Run full compilation
  - Enable option to **Run I/O Assignment Analysis before compilation** (**Settings** dialog box → **Compilation Process Settings**)
- Incomplete design with completed top-level design file
  - Run I/O Assignment Analysis on design
- Incomplete or no design files
  - Use early I/O planning methodology (see I/O System Design online training)

The Quartus Prime Software: Foundation

I/O Planning with the BluePrint Platform Designer

Problem: I/O Interfaces are Complex

- FPGAs support hundreds of I/O protocols, dozens of memory standards
- Protocols have thousands of configurations specified by standards



- Example rules
  - Neighbor I/O to 28 Gbps I/O restricted to 14 Gbps
  - PCIe® master channel restricted to specific transceiver lanes
  - DDR3-144 can only exist in Banks A to C if PLL #21 needs to be used by fabric logic

Problem: Periphery Placement Rules are Complex

PCI EXPRESS

DDR3

PLL

GPIOs

Input design with illegal or incomplete assignments

Quartus Prime Design Software

No fit!!

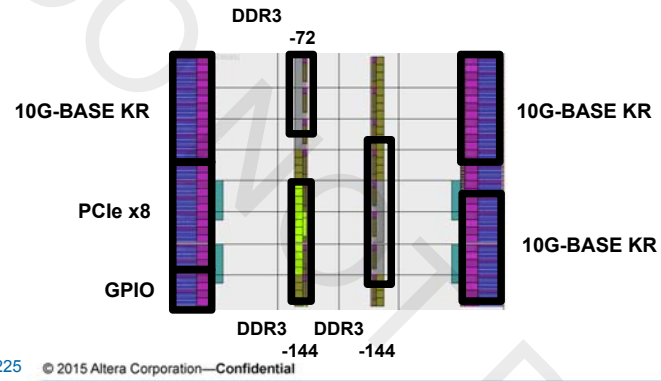
```
Error (175022): The auto-promoted clock driver could not be placed in any
                location to satisfy its connectivity requirements
```

???

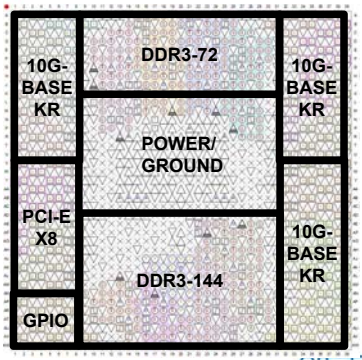
Better Solution: Interfaced-Based Assignment

- Instead of pin-by-pin or bus-by-bus, interface-by-interface!
- Choose valid locations for interfaces and clock domains graphically
- Verify assignments and update valid locations as assignments are made

By device resources

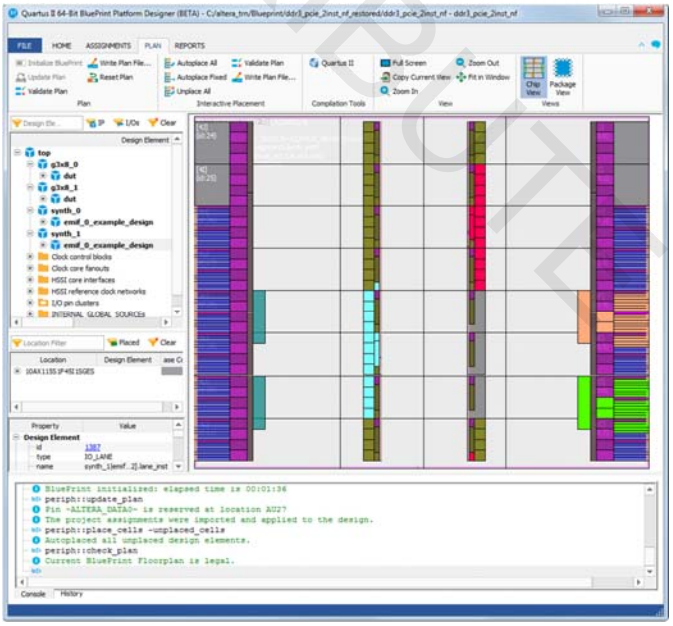


By I/O pins



Introducing the BluePrint Platform Designer

- Create legal assignments using the power of the Fitter
- Verify and validate in real-time
- High impact productivity tool for faster design closure
- Legal mapping of package footprint to I/O protocols within minutes
- Develop board layouts with confidence
- *Available only in the Quartus Prime Pro Edition software*
- *For Arria 10 and newer devices*



Drag and Drop Design Element Assignment & Checking

- Click and drag post-synthesis design elements (parts of design requiring placement in device) to the device's **Chip View**

Source Assignments Into Project & Compile

- Source generated Tcl script with command line or **Tcl Scripts** command in **Tools** menu
- Script provides command for sourcing
- Script organized into sections; pick and choose types of assignments to use

```
C:/altera_trn/Blueprint/ddr3_pcie_2inst_nf_restored/ddr3_pcie_2inst_nf.pdp_assignments.tcl*
26
27  ## To apply these settings to your project run the command "quartus_sh -t ddr3_pcie_2inst_nf.pdp_assignments.tcl" in your project directory.
28
29
30
31  project_open ddr3_pcie_2inst_nf -revision ddr3_pcie_2inst_nf
32
33
34
35  ## Disabled/Removed assignments
36
37
38
39  ## Pin location assignments
40  # Assignment for I/O pad emif_ref_clk_1_in_clk_clk
41  set_location_assignment PIN_R14 -to "emif_ref_clk_1_in_clk_clk" -tag "BluePrint IO Location Assignment"
42
43  # Assignment for I/O pad emif_ref_clk_in_clk_clk
44  set_location_assignment PIN_AG31 -to "emif_ref_clk_in_clk_clk" -tag "BluePrint IO Location Assignment"
45
46  # Assignment for I/O pad emif_reset_1_in_reset_reset_n
47  set_location_assignment PIN_AU10 -to "emif_reset_1_in_reset_reset_n" -tag "BluePrint IO Location Assignment"
48
49  # Assignment for I/O pad emif_reset_in_reset_reset_n
50  set_location_assignment PIN_AV10 -to "emif_reset_in_reset_reset_n" -tag "BluePrint IO Location Assignment"
51
52  # Assignment for I/O pad g3x8_0_currentspeed_currentspeed[0]
53  set_location_assignment PIN_AT22 -to "g3x8_0_currentspeed_currentspeed[0]" -tag "BluePrint IO Location Assignment"
54
55  # Assignment for I/O pad g3x8_0_currentspeed_currentspeed[1]
56  set_location_assignment PIN_AT17 -to "g3x8_0_currentspeed_currentspeed[1]" -tag "BluePrint IO Location Assignment"
```

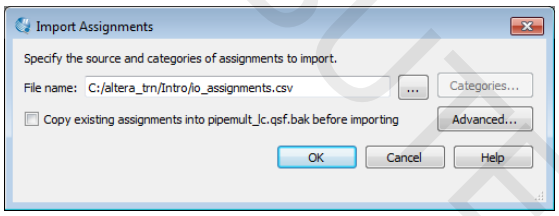

The Quartus Prime Software: Foundation

Other Methods of Creating I/O-Related Assignments

Import/Export via CSV

- Use spreadsheet Comma Separated Value (**.csv**) file to enter or edit I/O locations
- Convenient for transferring assignments between project revisions
- CSV column names must match Pin Planner column headings

Pin Planner **File** menu or Quartus Prime **Assignments** menu

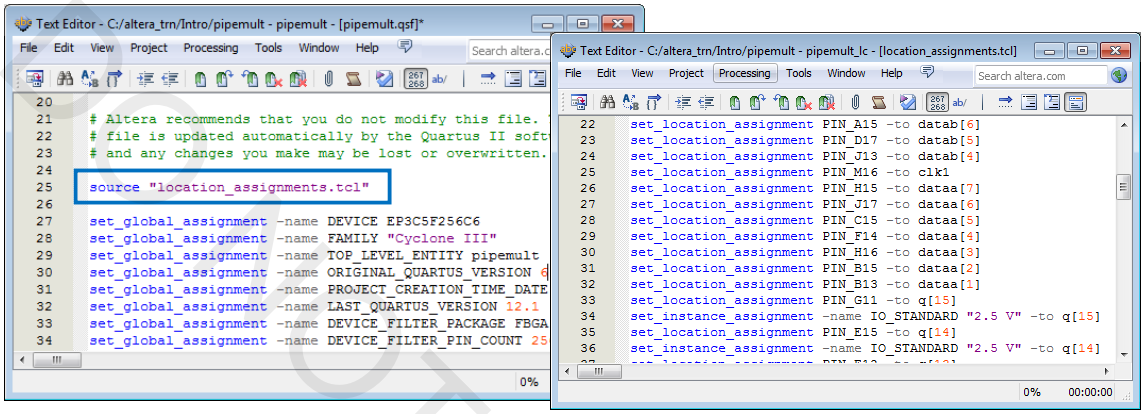


	A	B	C	D	E	F
1	To	Direction	Location	I/O Bank	VREF Group	I/O Standard
2	clk1	Input	PIN_E15	6	B6_N0	1.8 V
3	dataa[7]	Input	PIN_B14	7	B7_N0	2.5 V
4	dataa[6]	Input	PIN_A14	7	B7_N0	2.5 V
5	dataa[5]	Input	PIN_B13	7	B7_N0	2.5 V

pipemult

.qsf Editing & Scripting

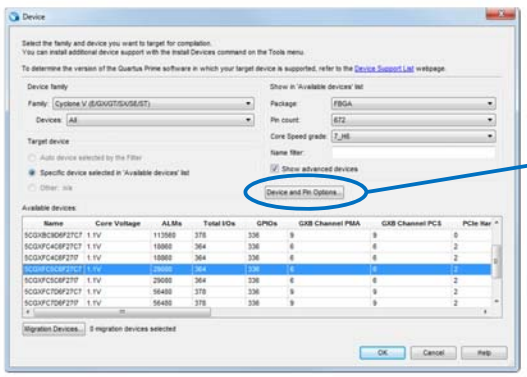
- Type pin-related assignments directly into **.qsf**
- Type pin-related assignments into separate Tcl
  - Source Tcl file in project **.qsf**
  - Execute Tcl file to write assignments into **.qsf**



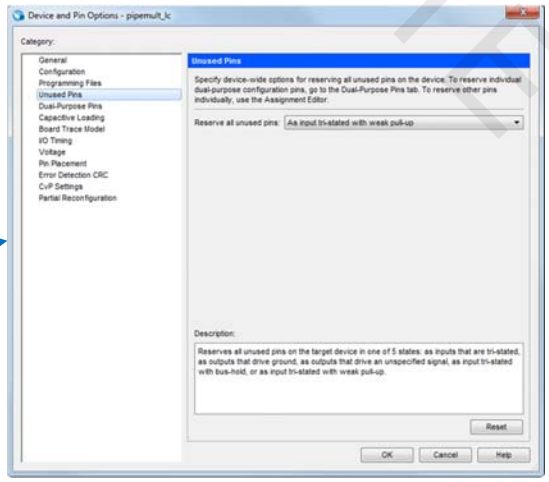
Global Pin Settings

- Select global settings for pins
  - e.g. unused I/O configuration, dual-purpose configuration pins, device CRC checking
- Assignments in Pin Planner or other tools have precedence over global settings

Device settings (Assignments menu)



Device and Pin Options



Test Your Knowledge: I/O Planning

1. Which of the following is *not* a method of creating pin location assignments using the Pin Planner?
   a. Location column in the All Pins list
   b. Drag and drop interfaces from the Design Elements window
   c. Location list in the Node Properties window
   d. Drag and drop pins from the All Pins list (or the Groups list) to a pin
   e. Drag and drop pins from the All Pins list (or the Groups list) to a bank

2. I/O Assignment Analysis should be run after creating assignments using BluePrint.
   - ❑ True
   - ❑ False

I/O Planning Summary

- Pin assignments can be performed in many ways, graphically & by means of text files
- The Pin Planner provides an easy-to-use graphical method for creating and managing pin assignments
- The BluePrint Platform Designer combines I/O interface resource assignment with on-the-fly legality checking
- I/O Assignment Analysis helps validate a device pin-out without performing a full compilation
- Pin validation can be completed during any point in design development

I/O Planning Support Resources

- Quartus Prime Handbook chapters (all Volume 2)
  - *Managing Device I/O Pins*
  - *BluePrint Design Planning* (**Pro Edition Handbook only**)
  - *Reviewing Printed Circuit Board Schematics with the Quartus Prime Software*
  - *Signal Integrity Analysis with Third-Party Tools*
  - *Mentor Graphics® PCB Design Tools Support*
  - *Cadence® PCB Design Tools Support*
- Training courses and demonstrations
  - I/O System Design
  - Advanced I/O System Design
  - Fast & Easy I/O System Design with BluePrint

The Quartus Prime Software: Foundation

A Taste of Programming & Configuration

Programming Files

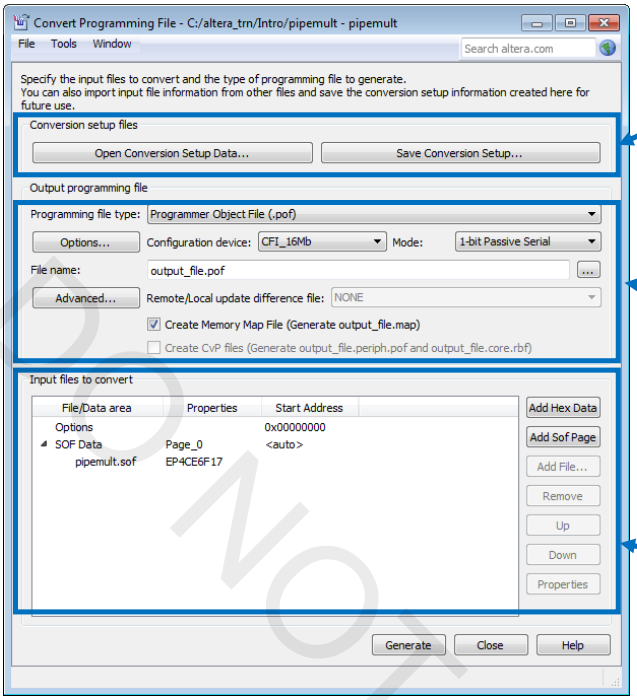
- **.sof** (SRAM Object File)
  - Used to configure FPGAs directly from Quartus Prime software through download cable
  - Always generated by default during a full compilation by the Assembler
- **.pof** (Programming Object File)
  - Used to program CPLDs and configuration devices
- **.jam/.jbc**
  - ASCII file used by processors and test equipment to program devices via JTAG
- **.jic** (JTAG Indirect Configuration File)
  - Contains programming data for target FPGA
  - Used to program EPCS (Altera serial configuration) devices through their dedicated configuration interface connection to an FPGA
  - Configuration devices do not have JTAG interfaces!

Programming File Conversion

- Quartus Prime Assembler by default automatically generates **.sof** files for any FPGA
  - May generate additional single-device programming file types from Device settings dialog box (**Device and Pin Options** → **Programming Files**)
- **.sof** files can only be used by the Quartus Prime Programmer to directly configure FPGA
- Other configuration solutions may require converting **.sof** file(s) into other file formats

Convert Programming Files GUI

File menu → **Convert Programming Files**



Load or save file conversion setup (.cof)

Select an optional programming file format

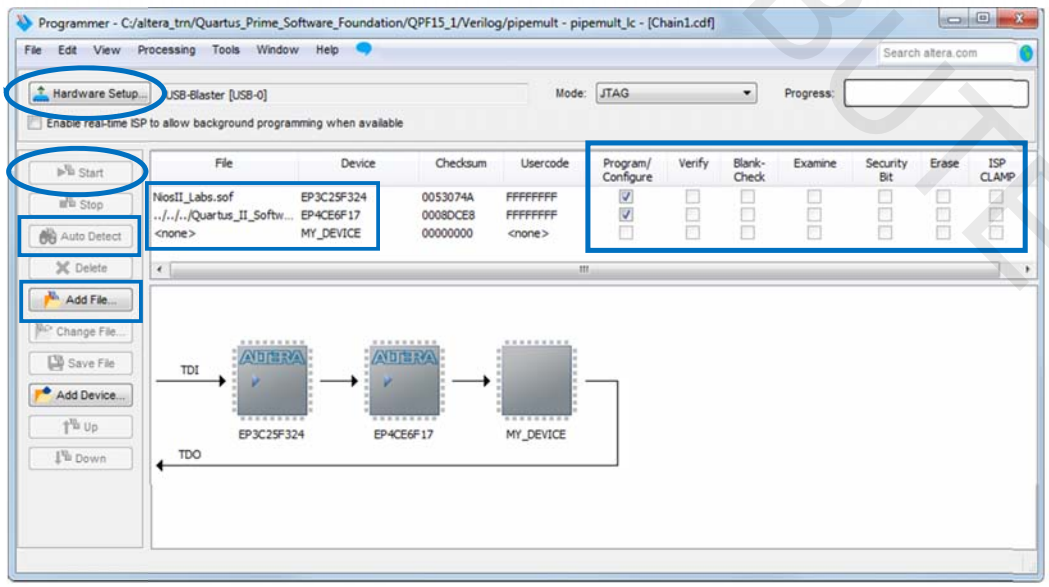
Select a configuration device and its supported mode

Select more options for the configuration device

Add input configuration/ programming file(s)

Quartus Prime Programmer

Tools menu → **Programmer**



Test Your Knowledge: Programming

1. Which of the following file types would you use to program an Altera configuration device via the JTAG interface of an FPGA configured with a flash loader IP?
   a. .pof
   b. .sof
   c. .jic
   d. .jam

2. The Quartus Prime Assembler creates, by default, both **.sof** and **.pof** files during a full compilation.
   - ❑ True
   - ❑ False

Programming/Configuration Support Resources

- Quartus Prime Handbook chapters
  - *Quartus Prime Programmer* (Volume 3)
- Training & Demonstrations
  - Introduction to Configuring Altera FPGAs
  - Configuration Schemes for Altera FPGAs
  - Configuration Solutions for Altera FPGAs
  - Debugging JTAG Chain Integrity

Exercise 5

Class Summary

- Quartus Prime Projects
- Design Entry
- Quartus Prime Compilation
- Settings & Assignments
- I/O Planning
- Programming/Configuration

Additional Quartus Instructor-Led Courses

- Quartus II Software Debug and Analysis Tools
  - Debugging solutions
    - SignalProbe incremental routing
    - SignalTap II Embedded Logic Analyzer
    - In-System Sources & Probes
    - In-System Memory Content Editor
    - Chip Planner & Resource Property Editor
    - System Console and related toolkits
- Quartus II Software Design Series: Timing Analysis
  - Create timing constraints to meet and optimize timing using the TimeQuest timing analyzer
  - Perform detailed timing analysis on an Altera device with the TimeQuest timing analyzer
- Advanced Timing Analysis with TimeQuest
  - Automate constraining and analysis of FPGA designs using Tcl
  - Constrain advanced types of interfaces and blocks
- Timing Closure with the Quartus II Software
  - Employ best practices to close timing using the tools available
- Introduction to the Qsys System Integration Tool
  - Integrate IP and custom logic into a Qsys system

Many Ways to Learn!

FREE
Always available
~4 minutes long
YouTube videos

Videos

FREE
Always available
~30 minutes long
>200 topics available

Online Training

Live over Webex
Ask questions to Altera expert
Taught in ½ day sessions
Class schedules at
www.altera.com/training

Virtual Classes

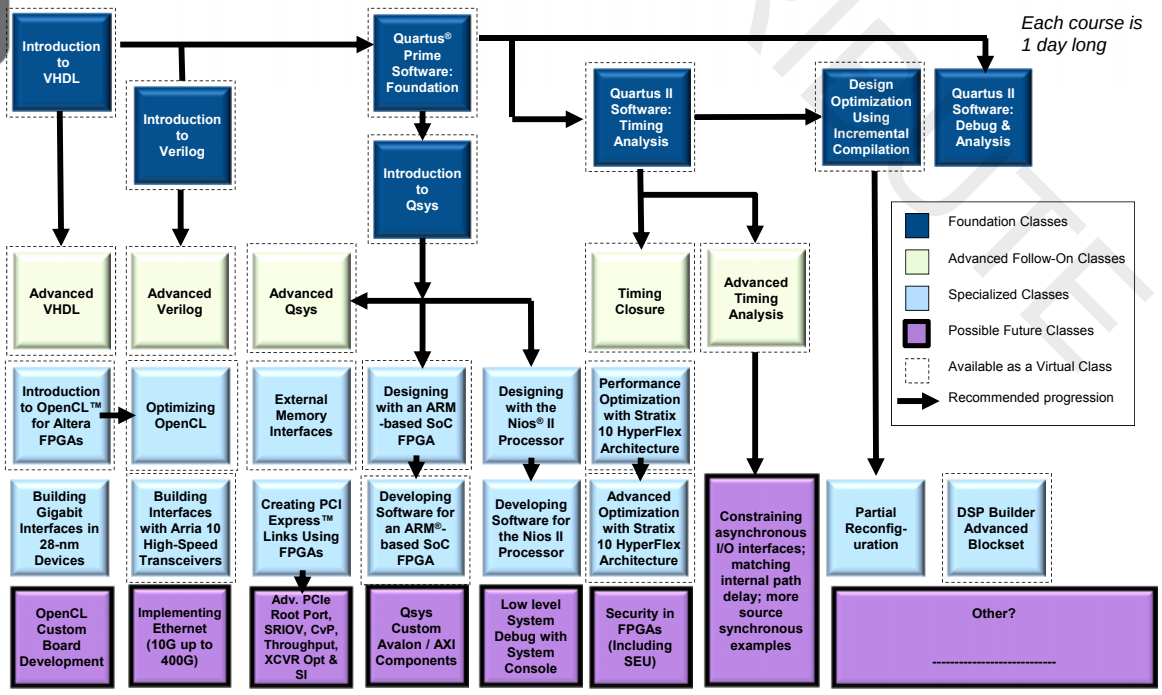
In-person
Ask questions to Altera expert
1 day long
Class schedules at
www.altera.com/training

Instructor-led Training

Altera Technical Support

- Reference Quartus Prime software on-line help
- Quartus Prime Handbook (separate Standard & Pro Editions)
- World-wide web: http://www.altera.com
  - Search for answers to problems with Knowledge Database
  - Download literature
  - View design examples
  - View online trainings
- MySupport: http://www.altera.com/mysupport
- Field applications engineers: contact your local Altera sales office
- Altera Wiki: www.alterawiki.com
- Altera Forum: www.alteraforum.com
- Intellectual Property Support
  - https://www.altera.com/support/support-resources/support-centers/interface-protocols.html

Instructor-Led and Virtual Training Curriculum

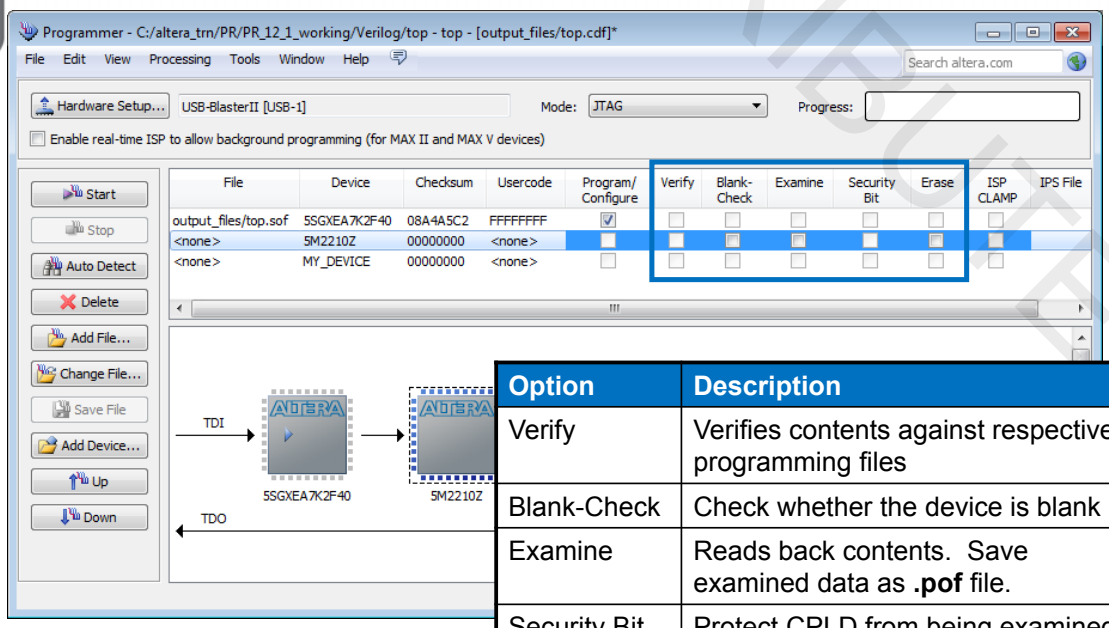


http://www.altera.com/training

The Quartus Prime Software: Foundation

Device Programming Appendix

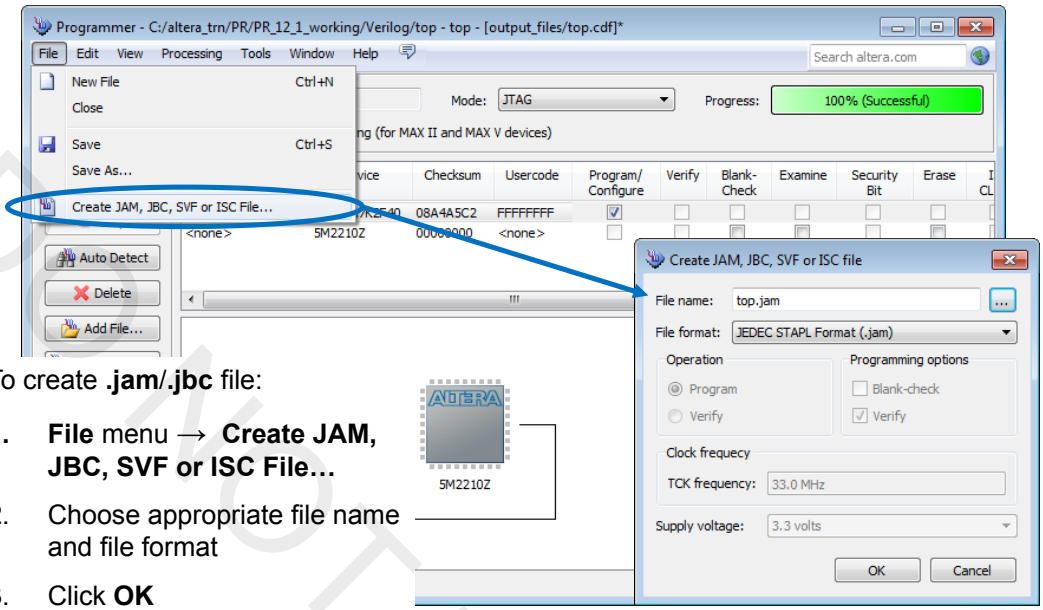
Programming Options for CPLDs/Config. Devices



Option	Description
Verify	Verifies contents against respective programming files
Blank-Check	Check whether the device is blank
Examine	Reads back contents. Save examined data as **.pof** file.
Security Bit	Protect CPLD from being examined.
Erase	Erase the contents of devices

Creating **.jam/.jbc** file

- Allows external processors or test equipment to program devices

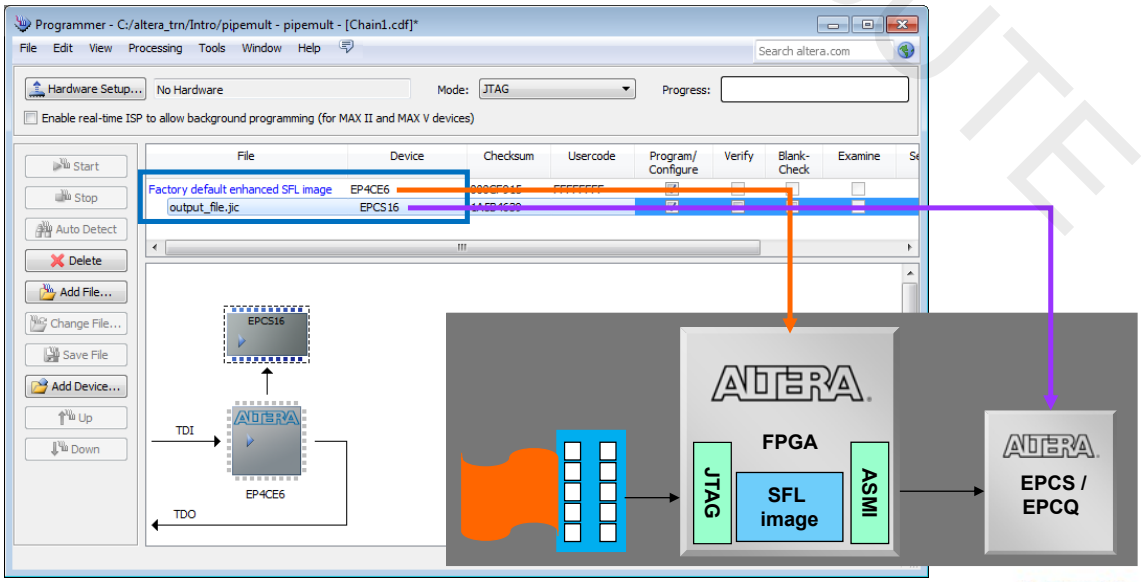


To create **.jam/.jbc** file:

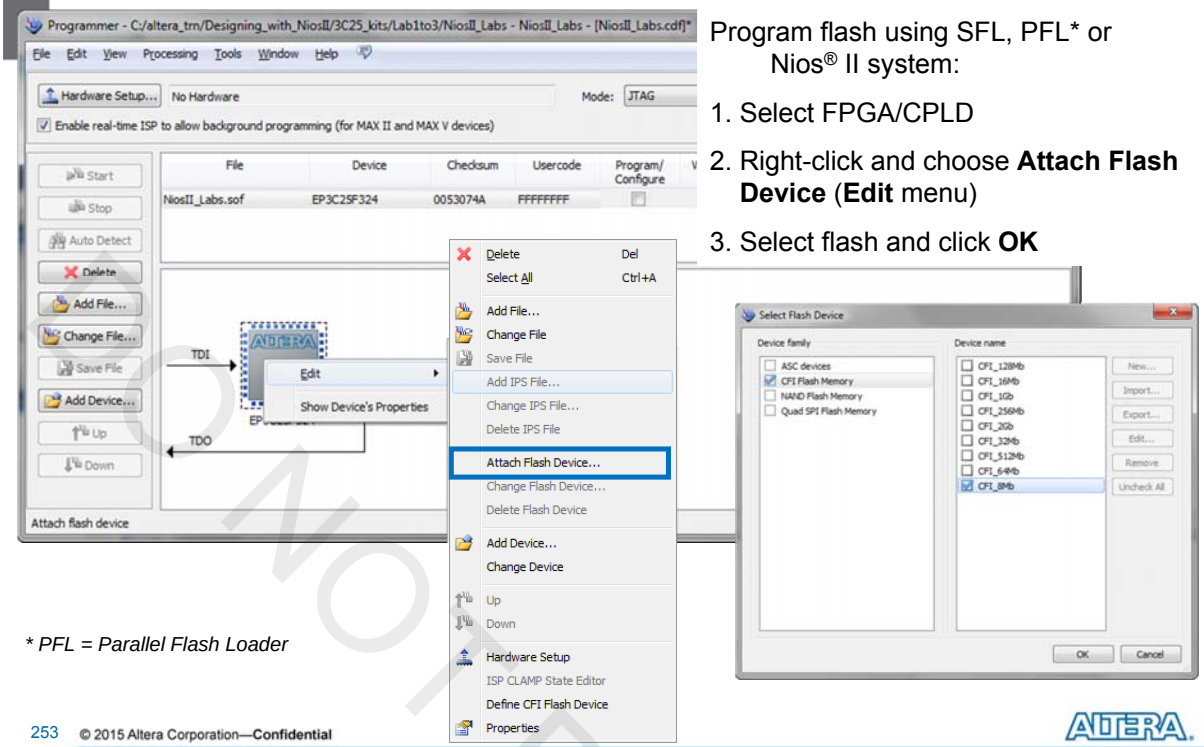
1. **File** menu → **Create JAM, JBC, SVF or ISC File…**
2. Choose appropriate file name and file format
3. Click **OK**

Programming Configuration Device with **.jic** File

- EPCS configuration devices have no JTAG interface
- Pass **.jic** through serial flash loader (SFL) image on FPGA



Quartus II Programmer: Flash Programming

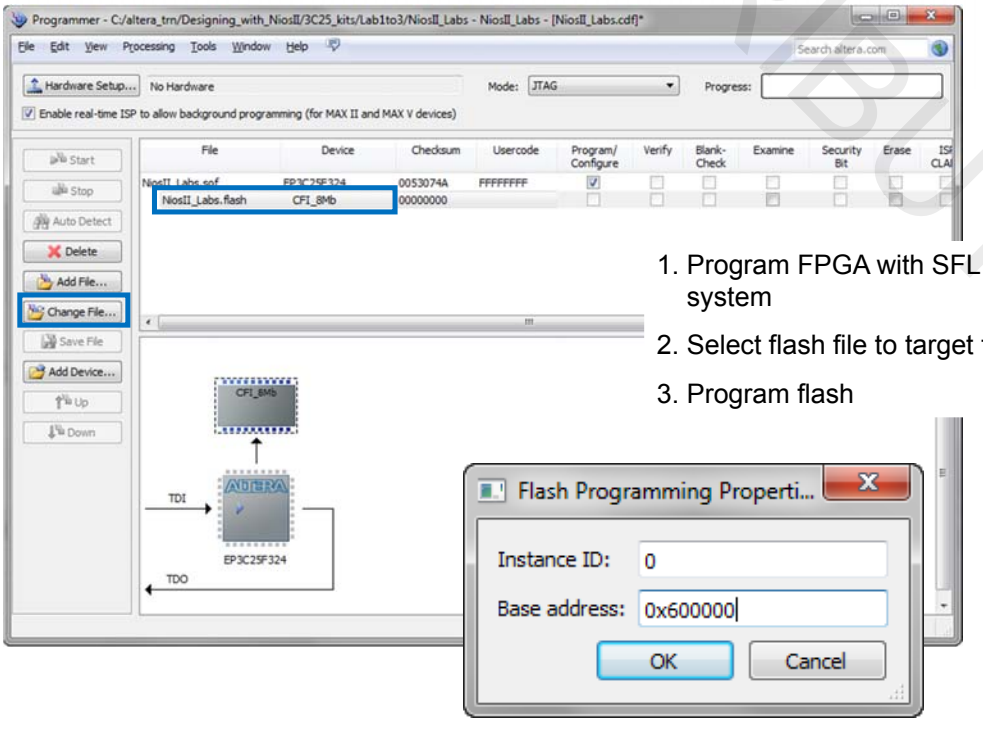


Program flash using SFL, PFL* or Nios® II system:

1. Select FPGA/CPLD
2. Right-click and choose **Attach Flash Device** (**Edit** menu)
3. Select flash and click **OK**

** PFL = Parallel Flash Loader*

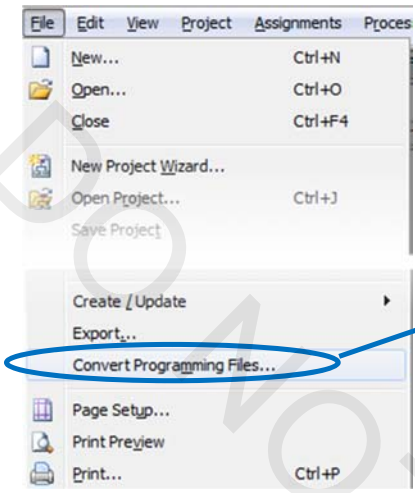
Quartus II Programmer: Flash Programming Example

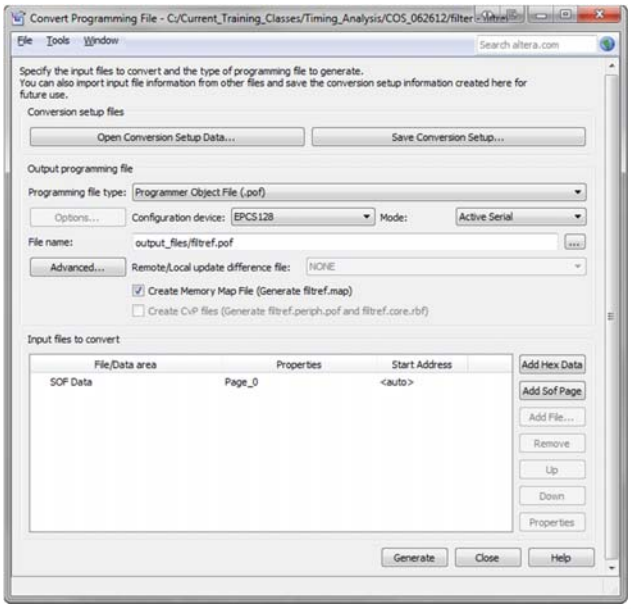


1. Program FPGA with SFL or Nios II system
2. Select flash file to target flash device
3. Program flash

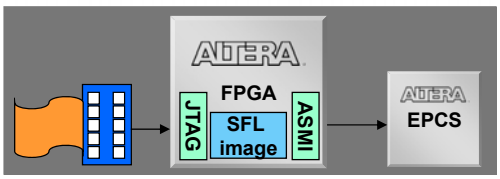
Converting Programming Files

- Convert **.sof** and **.pof** files to other supported programming files



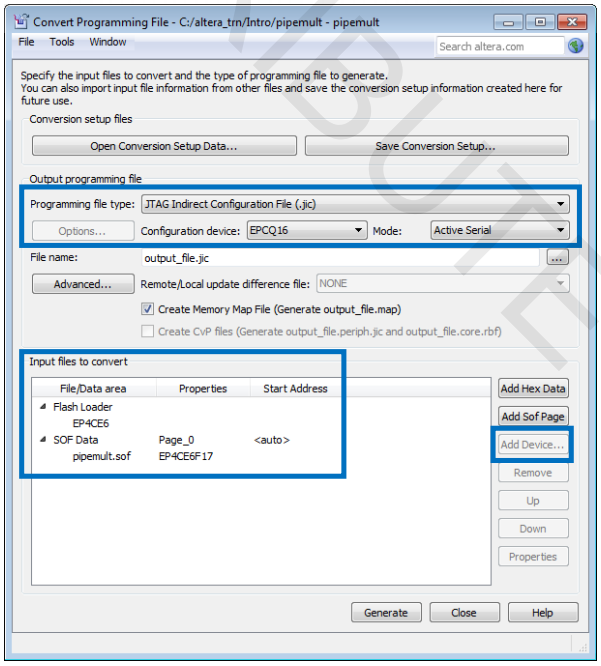


EPCS Programming Example: **.jic** Generation



To create the **.jic** file:

1. Select **.jic** as output **Programming file type**
2. Select **Flash Loader** in **Input files to convert list** and click **Add Device**
3. Select **SOF Data** and click **Add File**
4. Click **Generate**



Exercise Manual

for

The Quartus Prime Software: Foundation

Software Requirements to complete all exercises

Quartus® Prime Standard Edition software version 15.1 with Cyclone® V device family installed

Link to the Quartus Prime Handbook:

https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/hb/qts/qts-qps-handbook.pdf

Use the link below to download the design files for the exercises:

http://www.altera.com/customertraining/ILT/Quartus_Prime_Foundation_15_1_v1.zip

Exercise 1

Exercise 1

Objectives:

- *Create a project using the New Project Wizard*
- *Name the project*
- *Pick a device*

Note: In these exercises, you'll create a brand new project and complete an existing design. You'll have the choice of creating the design using three different types of design entry: Verilog, VHDL, or as a Quartus Prime schematic. Where noted, be sure to only follow the instructions appropriate for your choice of design entry method. By the end of the class, you'll have a final, optimized design, ready for programming into a Cyclone® V GX FPGA device.

Be sure to completely read the instructions for each step and sub-step in this lab manual. Each step first summarizes what you'll be doing in that step before providing complete instructions. Use the lines next to each step (____) to keep track of your progress or to check off completed steps in the exercises.

If you have any questions or problems, please ask the instructor for assistance.

Step 1: Create new project for use in the lab exercises

____ 1. Before starting the Quartus Prime software, unzip the lab project files:

a. If you are using an Altera® training laptop (either locally or remotely through the Virtual Classroom), go to the **C:\altera_trn\Quartus_Prime_Software_Foundation** folder in a Windows Explorer window.

b. **Important: *delete*** any old lab file folders that may already exist there labeled **QIIF*** or **QPF***.

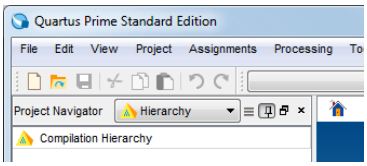
c. Double-click the executable file (**Quartus_Prime_Software_Foundation_15_1_v1.exe**) found in that location. If you cannot find this file, please ask your instructor for assistance.

d. In the WinZip dialog box, simply click **Unzip** to automatically extract the files in place to a new folder named **QPF15_1** in the directory mentioned above.

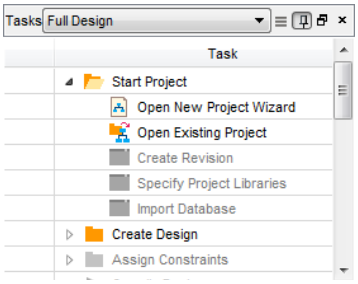
e. Clock **OK** then **Close** to close WinZip.

____ 2. Start the Quartus Prime Standard Edition software, version 15.1, from the **Start** menu (**All Programs → Altera 15.1.0.185 Standard Edition → Quartus Prime Standard Edition 15.1.0.185 → Quartus Prime 15.1**) or from a shortcut on the desktop.

IMPORTANT: *Make sure you are using the* ***Standard Edition*** *and* ***not*** *the* ***Pro Edition****! You can check in the title bar or go to the* ***Help*** *menu and select* ***About Quartus Prime****.*



____ 3. Start the New Project Wizard. You can find the New Project Wizard button on the **Home** tab that appears when you start the Quartus Prime software under **Start Designing**. If you've closed this tab, in the **Tasks** window on the left side of the Quartus Prime interface, switch to the **Full Design** task if necessary, expand the **Start Project** folder, and double-click **Open New Project Wizard**. You can also select **New Project Wizard…** from the **File** menu. The New Project Wizard appears. If the **Introduction** screen appears, click **Next**.



____ 4. Select one of the working directories shown below depending on the type of design entry you want to perform. Once you've made the choice of design entry method, it should be used as your entry method for **all** of the exercises. (The **<*lab_install_directory*>** is **C:\altera_trn\Quartus_Prime_Software_Foundation**.)

<*lab_install_directory*>\QPF15_1\VHDL

<*lab_install_directory*>\QPF15_1\Verilog

<*lab_install_directory*>\QPF15_1\Schematic

____ 5. Set the **Name** of the project to **pipemult** and leave the top-level entity name as **pipemult.**

____ 6. Click **Next**.

____ 7. On the **Project Type** page, leave the type set to the default: **Empty project**. Click **Next**.

*If you'd like, you can click the **Design Store** link to see the project templates that are currently available (requires an Internet connection).*

____ 8. On the **Add Files** page, add the top-level design file to the project.

a. Click the browse button  . Navigate to the project directory selected earlier as the **Select File** dialog box may not automatically point there.

b. Select the top-level file **pipemult** (**.v**, **.vhd**, or **.bdf**, depending on the design entry method you chose in #4).

c. Click **Open.**

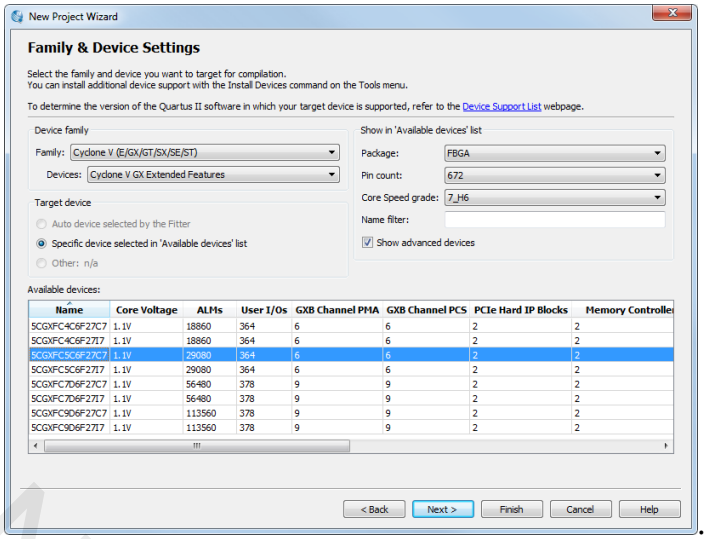
d. Click **Add** to actually add the file to the project *(easy to miss!).*

e. Click **Next**.

Note that this step isn't really necessary since the design file is already located in the project working directory. The new project would automatically include the design file as part of the project. Files or file directories (libraries) only need to be added in the New Project Wizard if they are not located in the project directory. Adding the file to the project removes a warning message that indicates the file has not been added.

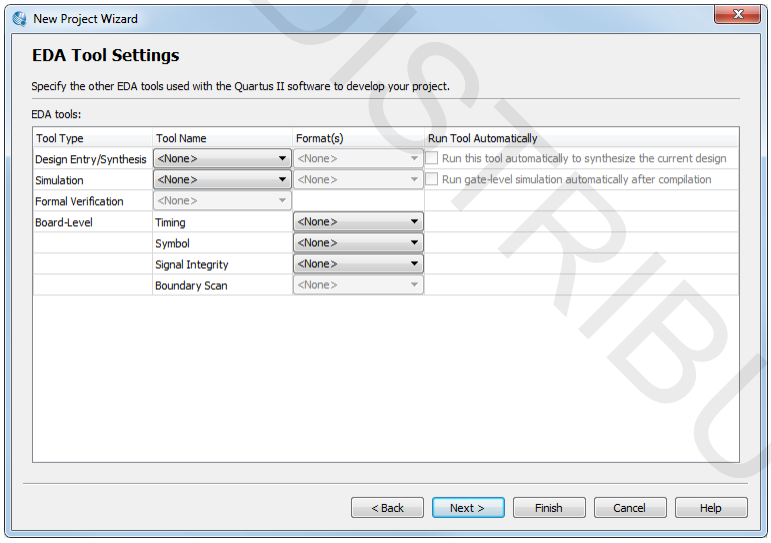
____ 9. On the **Family & Device Settings** page, select **Cyclone V (E/GX/GT/SX/SE/ST)** as the **Family**. In the **Devices** drop-down, choose the **Cyclone V GX Extended Features** variation.

____ 10. In the **Show in 'Available device' list** section, set **Package** to **FBGA**, **Pin count** to **672**, and **Core Speed grade** to **7_H6**. This filters the list of available devices. Select the **5CGXFC5C6F27C7** device from the **Available devices:** list. It should be the third device in the filtered list.

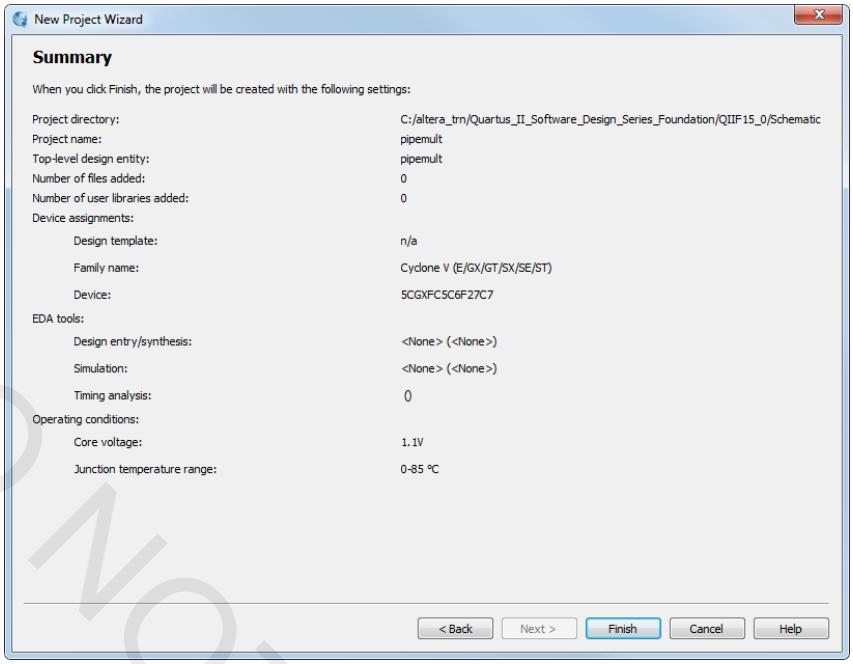


____ 11. Click **Next**.

____ 12. On the **EDA Tool Settings** page, you can specify third-party EDA tools you may want to use along with the Quartus Prime software for different functions such as simulation or timing analysis. Since these exercises will be done entirely within the Quartus Prime software without any other tools, set all the selections in the **Tool Name** column to **<None>** to save some compile time later. Click **Next**.



The summary screen appears.



____ 13. Click **Finish**.

The project is now created.

*Keep the project open as you continue through the rest of the exercises. There is no need to close the project. If you do close the project for some reason, be sure to select **Open Project** instead of just **Open** from the **File** menu (or **Open Existing Project** from the **Tasks** window). The **Open** command is used to simply open a single file instead of a project, preventing the ability to perform many project-based operations, such as compilation.*

Exercise Summary

- Created a project using the New Project Wizard
  - *Named the project*
  - *Picked a device*

END OF EXERCISE 1

Exercise 2

Exercise 2

Objectives:

- *Create a multiplier and RAM block using IP from the IP Catalog to complete the design*
- *Create a **.hex** file to initialize the RAM block using the Memory Editor*
- *Analyze and elaborate the design to check for errors*

Pipelined Multiplier Design

Figure 1 shows a schematic representation of the top-level design file you will be using today. It consists of a multiplier and a RAM block. Data is fed to the multiplier from an external source and stored in the RAM block, which is also controlled externally. The data is then read out of the RAM block by a separate address control.

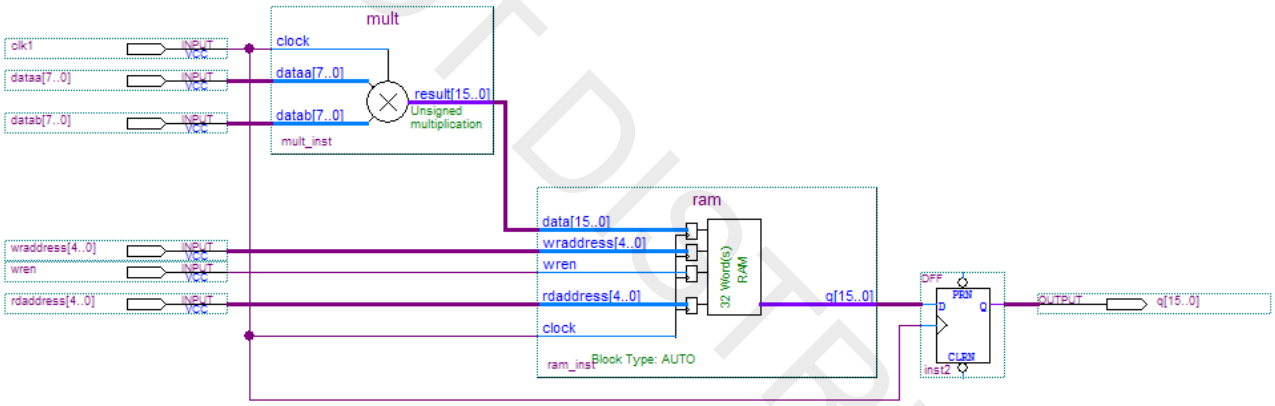


Figure 1

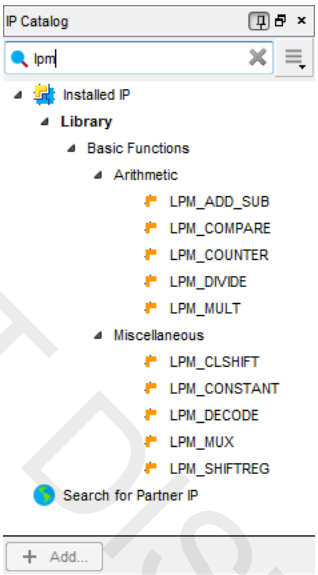
***IMPORTANT NOTE:** For exercises 2-5, you should continue working from the files of the previous exercise. The **Solutions** directory contains a Word document with the answers to questions asked in the exercises as well as an archive of the final project as it would be set up at the end of exercise 5.*

Step 1: Build an 8x8 multiplier using the IP Catalog

____ 1. If the **IP Catalog** is not visible, select it from the **Tools** menu, the toolbar  , or from the **View** menu → **Utility Windows**).

____ 2. Type **lpm** into the search field of the **IP Catalog**.

This filters the list of installed IP to only those that are part of the Library of Parameterized Modules (LPM) discussed in the presentation. These are basic logic functions that can be used in any design on any device.



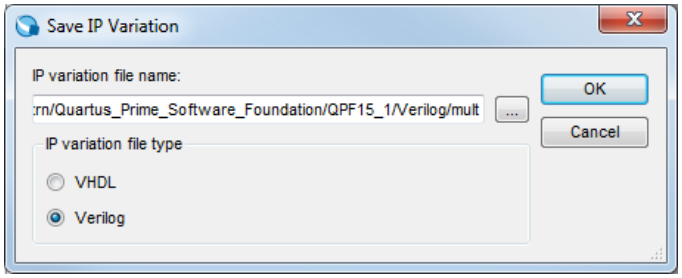
____ 3. Right-click the IP named **LPM_MULT** and hover over the **Details** submenu.

Here you can get some basic information about the selected IP, as well as where the files for the IP are located. Sometimes, datasheets or web links for more information can be found here. In this case, the IP is part of the Quartus Prime Standard Edition installation, so its files are located in the Quartus Prime installation directory.

____ 4. Create an instance of the **LPM_MULT** IP by either double-clicking it in the IP Catalog or clicking it once to select it and then click **Add**. You can also do this by right-clicking the IP and selecting **Add component**.

____ 5. In the **Save IP Variation** dialog that appears, set the **IP variation file name** to **mult** by adding **mult** to the end of the file path, which should match the project directory.

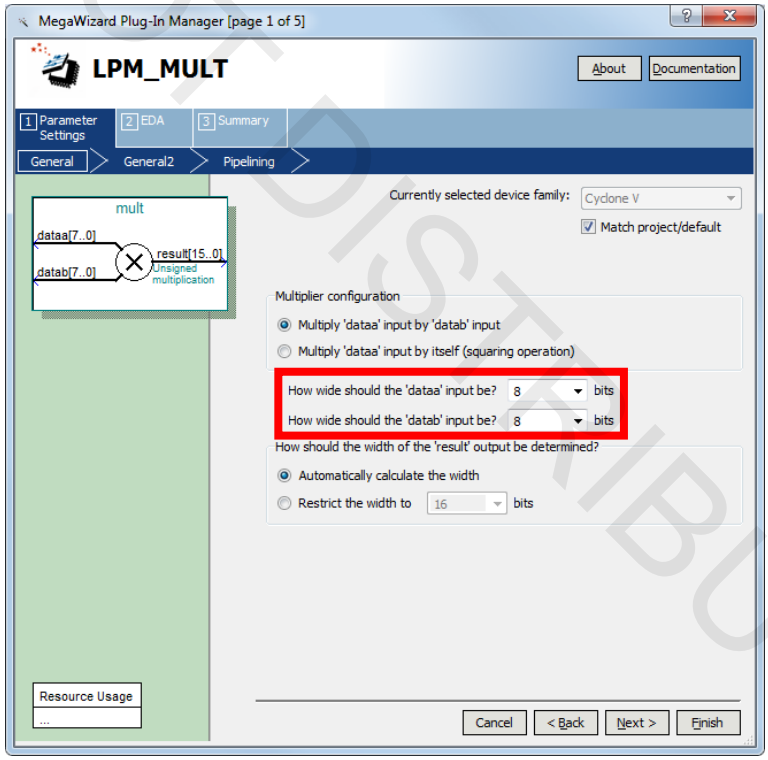
____ 6. For the **IP variation file type**, choose either **VHDL** or **Verilog** depending on your choice of HDL and exercise directory. If you are using the **Schematic** version of the design, you can choose either HDL.



____ 7. Click **OK**.

The MegaWizard™ Plug-In Manager for this IP will appear. Most older IP, such as LPM_MULT, use the MegaWizard interface for parameterization. Newer IP use the Qsys-based IP Parameter Editor. To keep things simple, the IP used in today's class all use the MegaWizard interface.

____ 8. On **page 1** of the MegaWizard, set the width of the **dataa** and **datab** buses to **8** bits if they are not already set. For the remaining settings in this window, use the default settings.

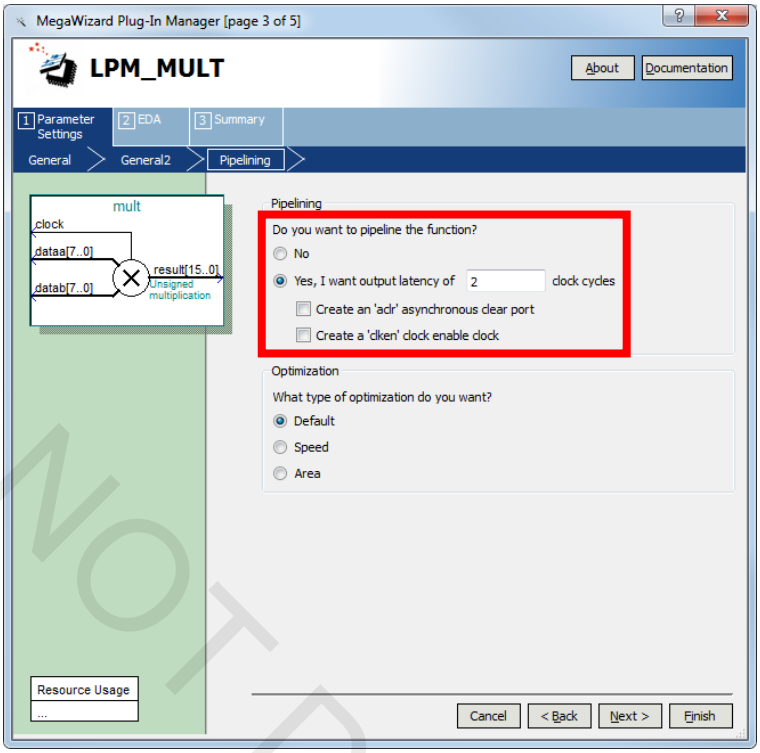


____ 9. Click **Next**.

____ 10. On **page 2**, use all the default settings (i.e. **datab** input does NOT have a constant value, use unsigned multiplication, and select the default multiplier implementation).

____ 11. Click **Next**.

____ 12. On **page 3**, choose **Yes, I want an output latency of 2 clock cycles**.

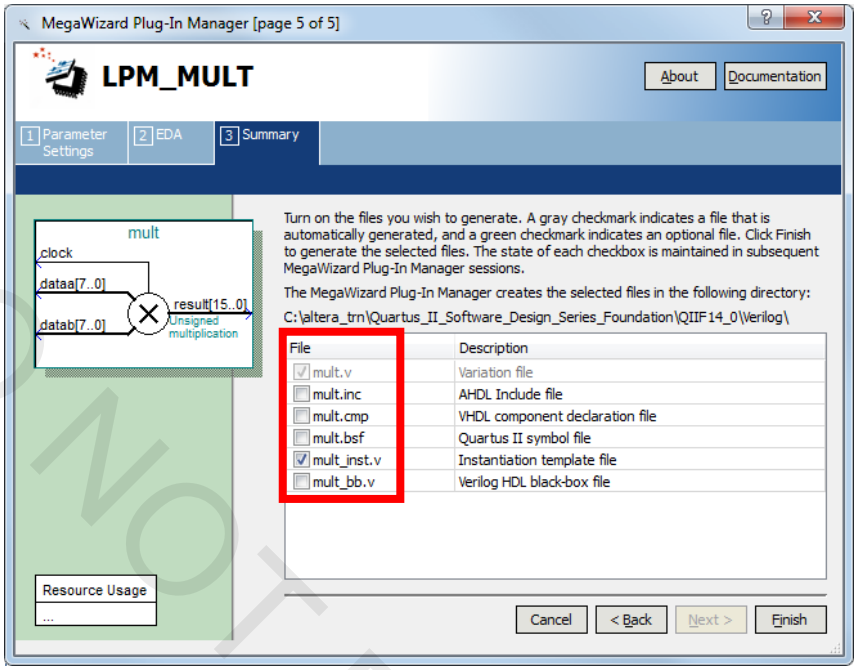


____ 13. Click **Next**.

*You should now be on **page 4** (tab 2 of the **MegaWizard Plug-In Manager**) named **EDA**. This tab indicates the simulation model file needed to simulate **LPM_MULT** in an EDA simulation tool (e.g. the **ModelSim®-Altera Edition** simulator or some other 3rd-party simulation tool). As indicated, the **lpm** simulation model file would need to be used. Also on this page, you have the option of generating a timing and resource estimation netlist for use by a 3rd-party synthesis tool.*

____ 14. We are not using any third-party tools, so just click **Next**.

____ 15. On **page 5**, using the table below, check the appropriate boxes depending on the design entry method you've selected.



Design entry method	Files to enable in MegaWizard Plug-In Manager
VHDL	**mult_inst.vhd** & **mult.cmp**
Verilog	**mult_inst.v**
Schematic	**mult.bsf**

____ 16. Click **Finish** to create the IP. If a dialog box appears asking if you want to add the **.qip** file to the Quartus Prime project, click **Yes**.

*The multiplier IP has been created. If you look in the project directory, you'll see the generated files (**.v**, **.vhd**, **.bsf**, **.qip**).*

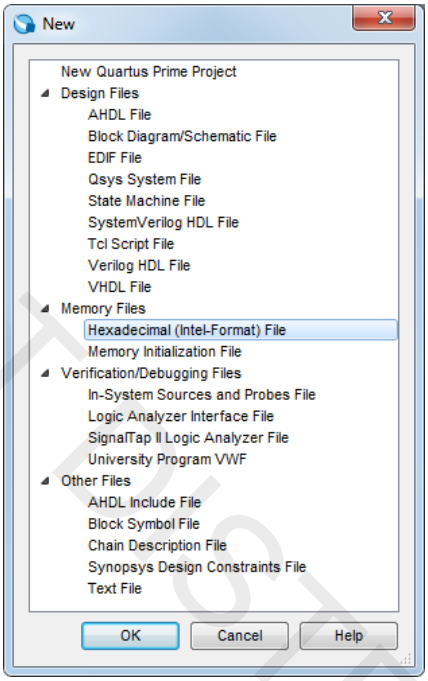
*If for some reason later on you find that the IP core you created is incorrect or you forgot or missed a checkbox for generating all the required output files, select **IP Components** in the pop-up list in the Project Navigator, and double-click the IP core you would like to edit. Go through the pages of the MegaWizard again (skip around with the tabs at the top) to fix your mistakes or generate the missing file(s). Click **Finish** to update the generated files.*

Step 2: Create .hex file using the Memory Editor

____ 1. From the **Tasks** window, in the **Create Design** folder, double-click **Create New Design File**. (*You may have to change the Tasks Flow to **Full Design** if you haven't already.)* You could also go to the **File** menu and select **New** or click in the toolbar.

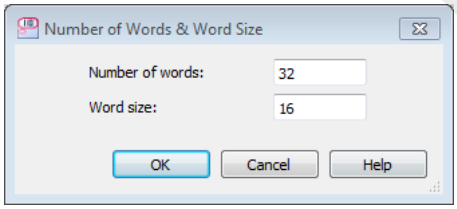
____ 2. In the **New** dialog box, expand the **Memory Files** category and select **Hexadecimal (Intel-Format) File**.

This hex file will be used to initialize a dual-port RAM you will create in the next step.

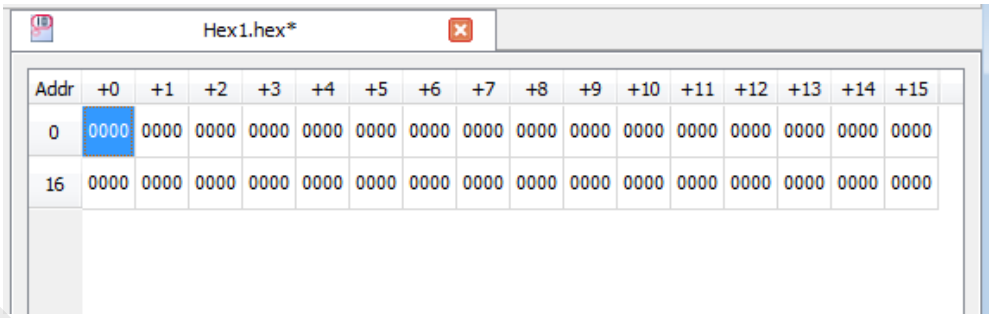


____ 3. Click **OK**.

____ 4. In the memory size dialog box, enter **32** as the **Number of words** and **16** as the **Word size**.



____ 5. Click **OK**.



*The **Memory Editor** now displays the memory space. If it is not displayed exactly as shown above, you can change the number of cells per row (**View** menu) to **16**, the memory radix (**View** menu) to **Hexadecimal**, and the address radix to **Decimal**.*

____ 6. Select all of the memory locations. Right-click and select **Custom Fill Cells**.

____ 7. Use the **Custom Fill Cells** dialog box to enter your own values to initialize your memory. You can enter any values you want. Select one of the following:

a. Repeating sequence: Enter a series of numbers separated by commas or spaces to be repeated in memory.

b. Incrementing/decrementing: Enter a start value and another value by which to increment or decrement the start value.

Addr	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+10	+11	+12	+13	+14	+15
0	0101	0202	0303	0404	0505	0606	0707	0808	0909	0A0A	0B0B	0C0C	0D0D	0E0E	0F0F	1010
16	1111	1212	1313	1414	1515	1616	1717	1818	1919	1A1A	1B1B	1C1C	1D1D	1E1E	1F1F	2020

____ 8. Click **OK** when complete to close the dialog box.

____ 9. Save the file as **ram.hex** in the project directory (you may have to navigate to the project directory again before you save). Make sure the option to **Add file to current project** is enabled.

____ 10. Close **ram.hex**.

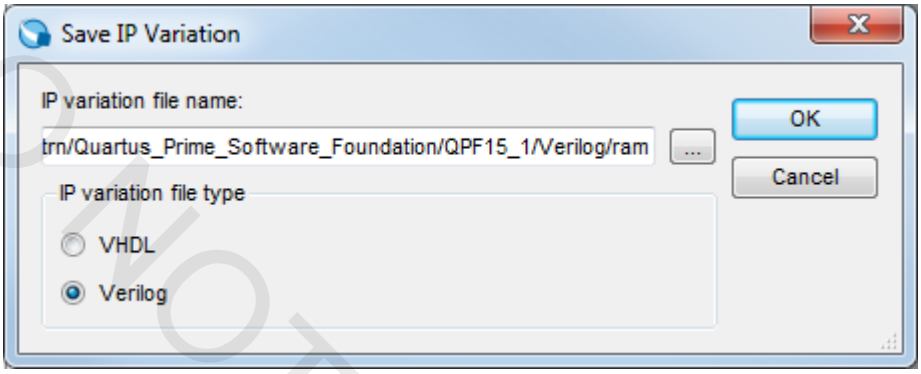
Step 3: Create a 32x16 RAM from the IP Catalog

____ 1. In the IP Catalog search field, enter **ram:** *(be sure to include the colon).*

____ 2. Add the IP named **RAM: 2-PORT** by double-clicking it or selecting it and clicking **Add**.

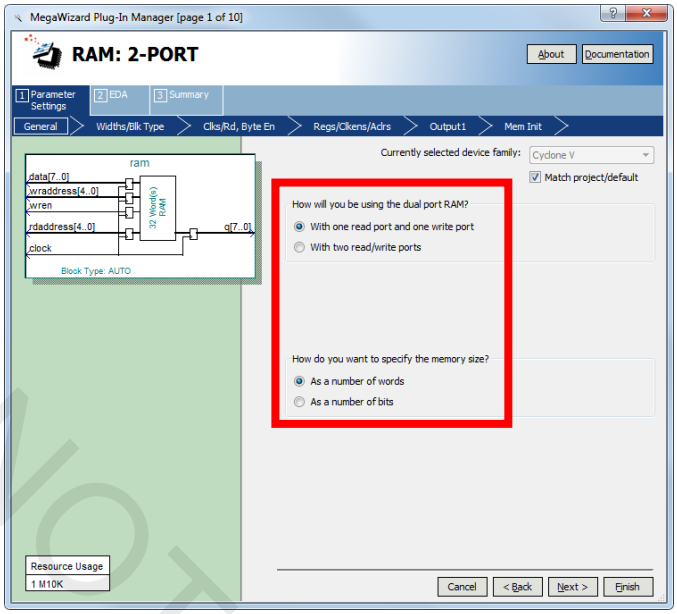
____ 3. For the **IP variation file name**, enter **ram**.

____ 4. Set the **IP variation file type** to the same HDL as before.



____ 5. Click **OK**.

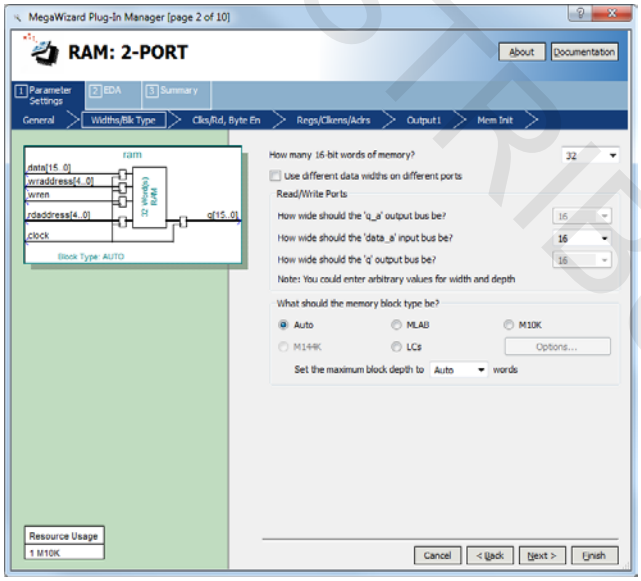
____ 6. On page 1, select **With one read port and one write port** for the dual port RAM mode. Select memory size **As a number of words**.



____ 7. Click **Next**.

____ 8. On **page 2** (**Widths/Blk Type**), set the width of the **data_a** bus to **16** and the number of 16-bit words to **32.**

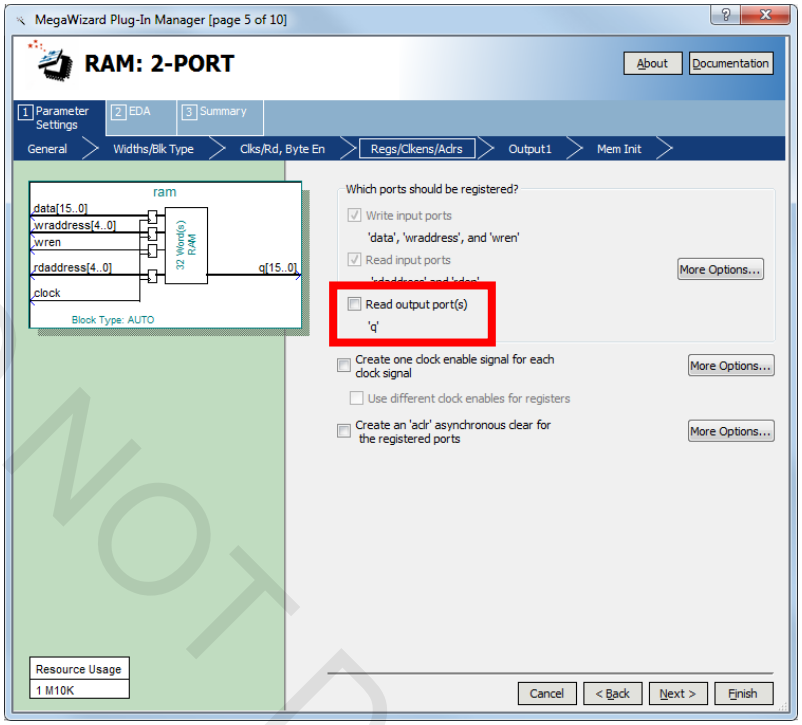
____ 9. Set the **memory block type** to **Auto** and the **maximum block depth** to **Auto**.



*Notice that the **Resource Usage** section in the lower left indicates that it will require 1 **M10K** memory block resource to implement this memory.*

____ 10. Click **Next** twice.

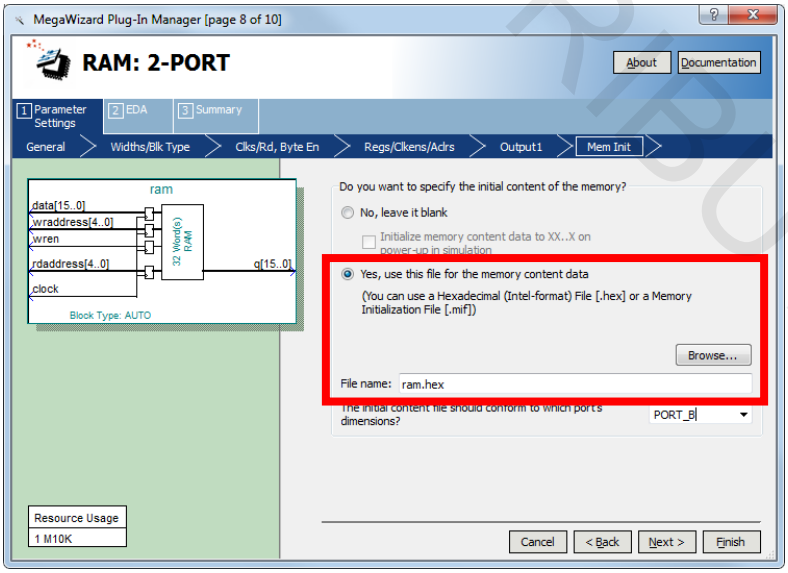
____ 11. On **page 5** (**Regs/Clkens/Aclrs**), disable the option to register the **Read output port(s) 'q'**. Accept the remaining default settings.



____ 12. Click **Next** 2 times.

____ 13. On **page 8** (**Mem Init**), select **Yes, use this file for the memory content data.**

____ 14. Once enabled, type in the file name **ram.hex**. This is the file you created in the previous step.



____ 15. Click **Next**.

____ 16. On **page 9**, the **altera_mf** simulation model file is displayed as being needed to simulate this IP in a 3rd-party EDA simulation tool. Click **Next**.

____ 17. On **page 10**, choose the same files to generate for **ram** as you selected for **mult** earlier (Step 1, #15).

____ 18. Click **Finish** to close the wizard.

____ 19. If asked, click **Yes** to add the Quartus IP file to the project.

You have now created the two components needed for this design.

Step 4: Instantiate and connect design blocks according to design entry method

*Choose **ONE** of the following procedures based on your design entry method (VHDL, Verilog, or Schematic). Follow **only** the directions for your selected design entry method and then proceed to **Step 5** on page 24.*

VHDL

Perform these instructions only if you are using VHDL.

____ 1. Open **pipemult.vhd**. You can use the **Open** command from the **File** menu, click the  toolbar button, or double-click the top-level entity in the Project Navigator. You can also double-click **Open Existing Design File** in the **Create Design** folder of the **Tasks** window.

*This is the top-level file for the design. Normally, you would have to instantiate both **ram** and **mult** and connect them together. In the interest of time, the file has been almost completed for you by including the **ram** component declaration and instantiation. However, it is missing the instantiation of the multiplier.*

____ 2. Open the file **mult.cmp**. Copy the **component** declaration from **mult.cmp** and paste it into the architecture declaration section of **pipemult.vhd** where indicated.

____ 3. Close **mult.cmp**.

____ 4. Open the file **mult_inst.vhd**.

____ 5. Copy the contents of **mult_inst.vhd** (the component instantiation) and paste into the architecture body of **pipemult.vhd** where indicated.

____ 6. Change the following signal names in the instantiation: (Note: While typing, the auto-complete pop up window may appear and show you a list of the available signal names which have been previously defined in the design file. You can use the **cursor keys**, *not the mouse*, to select a name from the list and press **Enter** to accept it.)

clock_sig	to	**clk1**
dataa_sig	to	**dataa**
datab_sig	to	**datab**
result_sig	to	**mult_to_ram**

____ 7. While you have this file open, try double-clicking any signal or port name to select it.

Notice how all instances of the selected signal or port name are highlighted in the file. This makes it very easy when editing HDL code to see where nodes are used or how they are connected together.

____ 8. Save **pipemult.vhd**.

____ 9. Close **mult_inst.vhd**.

____ 10. Continue to **Step 5: Check the design**.

Verilog

Perform these instructions only if you are using Verilog entry.

____ 1. Open **pipemult.v**. You can use the **Open** command from the **File** menu, click the toolbar button, or double-click the top-level entity in **Hierarchy** category of the Project Navigator. You can also double-click **Open Existing Design File** in the **Create Design** folder of the **Tasks** window.

*This is the top-level file for the design. Normally, you would have to instantiate both **ram** and **mult** and connect them together. In the interest of time, the file has been almost completed for you by including the **ram** instantiation. However, it is missing the instantiation of the multiplier.*

____ 2. Open the file **mult_inst.v**.

This is the instantiation template generated by the MegaWizard. If you do not see the file, return to the MegaWizard as mentioned earlier to generate the file.

____ 3. Copy the contents of **mult_inst.v** (the component instantiation) and paste into the body of **pipemult.v** where indicated.

____ 4. Change the following signal names in the instantiation. (Note: While typing, the auto-complete pop up window may appear and show you a list of the available signal names which have been previously defined in the design file. You can use the **cursor keys**, *not the mouse*, to select a name from the list and press **Enter** to accept it.)

clock_sig	to	**clk1**
dataa_sig	to	**dataa**
datab_sig	to	**datab**
result_sig	to	**mult_to_ram**

____ 5. While you have this file open, try double-clicking any signal or port name to select it.

Notice how all instances of the selected signal or port name are highlighted in the file. This makes it very easy when editing HDL code to see where nodes are used or how they are connected together.

____ 6. Save **pipemult.v**.

____ 7. Close **mult_inst.v**.

____ 8. Continue to **Step 5: Check the design**.

Schematic

Perform these instructions only if you are using schematic entry.

____ 1. Open **pipemult.bdf**. You can use the **Open** command from the **File** menu, click the toolbar button, or double-click the entity in the Project Navigator. You can also double-click **Open Existing Design File** in the **Create Design** folder of the **Tasks** window.

*This is the top-level schematic file for the design. Normally, you would have to instantiate both **ram** and **mult** sub-designs and connect them together manually. In the interest of time, the schematic file has been almost completed for you with the connections between the blocks already created. However, the design is missing the **ram** and **mult** blocks themselves and the output pins **q[15..0]**.*

____ 2. In the schematic file, double-click any empty space to open the **Symbol** window.

a. In the **Symbol** window, expand the **Project** folder.

b. Select the **mult** symbol and click **OK**.

c. Click the left mouse button to place the symbol inside the schematic file where indicated.

*Note: The three ports on the left side of the multiplier should line up exactly with the wires coming from the input pins (no X's). If they do not, you may not have specified the multiplier parameters correctly when configuring the IP. If this is the case, hit the **Esc** key to cancel the symbol placement, and recreate the incorrectly created IP from the IP Catalog using the steps listed earlier, choosing to overwrite the incorrect files when prompted.*

____ 3. Right-click on the **mult** symbol and choose **Properties**.

____ 4. In the Symbol Properties dialog box, change the **Instance name:** from **inst** to **mult_inst**. Click **OK**.

____ 5. Double-click an open area again to reopen the **Symbol** window. This time, select the **ram** symbol and place it in the schematic file where indicated.

*Note: With **ram**, the lower 4 ports on the left side of the symbol should line up with the wires coming from the input pins. The **data[15..0]** port should be unconnected.*

____ 6. As you did with mult, use the **Symbol Properties** dialog box to change the name of the **ram** instantiation from **inst** to **ram_inst**.

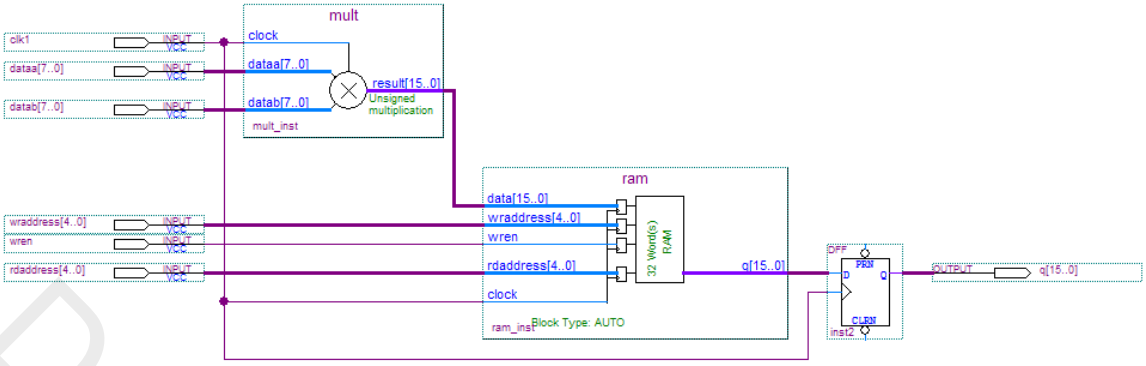
____ 7. Open the **Symbol** window again and this time, as a shortcut, type **output** in the **Name:** field.

*The **Symbol** window found the output symbol automatically by name.*

____ 8. Click **OK** and place the output pin near the output port of the **inst2** register symbol.

____ 9. Double-click the **pin_name1** text and change it to **q[15..0]**.

____ 10. Click the bus drawing tool  found in the schematic editor toolbar, and click and drag to draw the bus connection between the **result[15..0]** output of **mult** to the **data[15..0]** input of **ram**.



The resulting schematic should look like the above figure.

____ 11. Save **pipemult.bdf**.

Step 5: Check the design

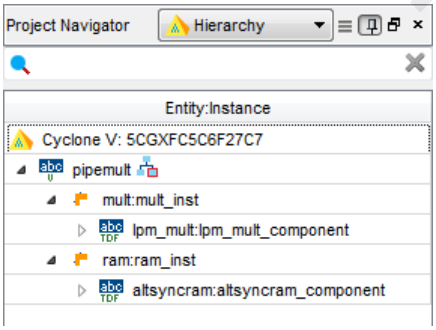
____ 1. From the **Processing** menu, go to **Start** and select **Start Analysis & Elaboration**.

Analysis and elaboration checks that all the design files are present and connections have been made correctly. It also establishes the project hierarchy.

____ 2. If a dialog appears at the end of the analysis and elaboration process, click **OK**.

____ 3. If there are any errors detailed in the **Messages** window, check your connections or, if necessary, return to the MegaWizard Plug-In Manager (through the IP Components category of the Project Navigator) for either IP core to fix the problem, choosing to overwrite any existing IP files. You can safely ignore any blue warning messages that appear, though you shouldn't see any if you've followed all the steps correctly up to this point.

Feel free to explore the Project Navigator that now contains the complete project hierarchy.



Exercise Summary

- Generated a multiplier and RAM using the IP Catalog and incorporated them into a design
- Created a **.hex** file for RAM initialization using the Memory Editor
- Checked the design files using Analysis and Elaboration

END OF EXERCISE 2

Exercise 3

Exercise 3

Objectives:

- *Perform a full compilation*
- *Locate information in the Compilation Report*
- *Explore cross-probing capabilities by viewing logic in various windows*

Device Name	**5CGXFC5C6F27C7**
Total Design	
Logic utilization (in ALMs)	
Total registers	
Total pins	
Total block memory bits	
Total DSP Blocks	
mult sub-design	
ALMs needed (mult)	
Combinational ALUTs (mult)	
Dedicated Logic Registers (mult)	
DSP Blocks (mult)	
ram sub-design	
ALMs needed (ram)	
Combinational ALUTs (ram)	
Dedicated Logic Registers (ram)	
Block memory bits (ram)	
M10Ks (ram)	
Control Signals	
Name	**Fanout**

Step 1: Compile the design

____ 1. Select **Start Compilation** from the **Processing** menu or click  located in the toolbar to perform a full compilation of the design. You can also double-click **Compile Design** in the **Tasks** window.

____ 2. A dialog box may appear to indicate when the compilation is complete. Click **OK** if it does.

You can safely ignore any warnings that appear.

Step 2: Gather information from the Compilation Report

The Compilation Report provides all information on design processing. You use it to understand how the compiler interpreted your design and to verify results. It is organized by compiler executables, with each one generating its own folder. Throughout the following steps, feel free to explore the many reports generated by the tool.

____ 1. The **Compilation Report** should have opened in the Quartus Prime GUI automatically when you started the compilation. If it didn't, click  in the toolbar or choose **Compilation Report** from the **Processing** menu.

____ 2. From the **Flow Summary** section of the **Compilation Report**, record the **Logic utilization (in ALMs)**, **Total registers**, **Total pins**, **Total block memory bits**, and **Total DSP Blocks** in the table at the beginning of this exercise.

An adaptive logic module (ALM) is an FPGA logic structure made up of 2 adaptive look-up tables (ALUTs). These two ALUTs can be used together to implement a single logic equation of up to 7 inputs or independently to implement two logic functions of up to 6 inputs. ALMs are arranged in groups of ten. A grouping of 10 ALMs is a logic array block, or LAB.

An ALM can also be configured to implement a 32x2, 64-bit memory. When configured in this way, the LAB is refered to as a memory LAB, or MLAB, of 640 bits.

A DSP block contains two multipliers and an adder unit to support various multiply-add/accumulate configurations. See the device handbook or user guide for more details.

____ 3. Expand the **Fitter** folder in the Compilation Report.

a. Expand the **Resource Section** folder.

b. From the **Resource Utilization by Entity** report, record in the table the resource counts for the **mult:mult_inst** and **ram:ram_inst** sub-designs. Collapse the sub-designs in the **Compilation Hierarchy Node** column to make it easier to view the report, and scroll to the right to find the appropriate columns of information.

Fitter Resource Utilization by Entity

	Compilation Hierarchy Node
1	|pipemult
1	|mult:mult_inst|
2	|ram:ram_inst|

c. From the **Control Signals** report, also in the **Resource Section** folder, record the control signals found and their fan-out.

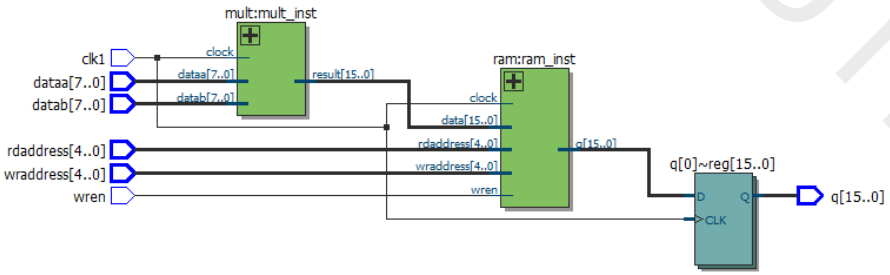
*From these results, you can see that this small design is using very few device resources. An ALM is used for the top-level design, but it's not clear (yet) what it's being used for. Memory and DSP blocks are used to implement **ram** and **mult**, respectively. No other logic is being used. Feel free to explore some of the additional reports listed here and compare these resource reports with the reports found in the Analysis & Synthesis folder.*

In the next few steps, you will take a look at some additional ways to analyze the results of your compilation, determine the location of the input and output registers, and understand what the ALM is used for.

Step 3: Explore the design logically using the RTL Viewer

*The **RTL Viewer** allows you to view a logical representation of an analyzed design graphically. It is a very helpful tool for debugging HDL synthesis results.*

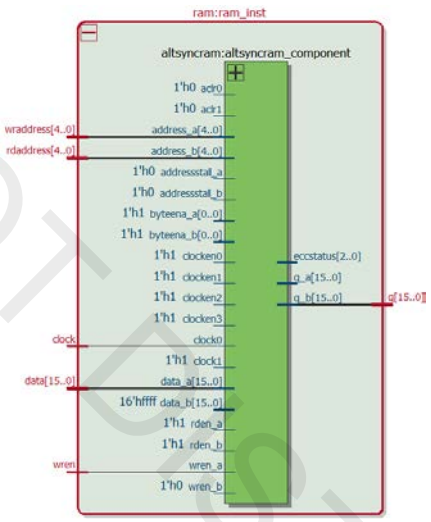
____ 1. From the **Tools** menu, open the **RTL Viewer** (under **Netlist Viewers**). You can also access the RTL Viewer from the **Compile Design** section of the **Tasks** window (**Analysis & Synthesis** folder → **Netlist Viewers**).



*You should see a schematic similar to the one shown here. The output register block is named **inst2[15..0]** in the schematic version of the project, but the rest of the diagram is the same for all three versions. This is a graphical view displaying the logical representation of the design. It shows the I/O, the instantiation of the **mult** and **ram** sub-designs, and an additional set of output registers. Notice the registers are external to the memory block per the original design.*

____ 2. Select the **ram** sub-design to highlight it. The outline of the block turns bright red when the block is selected. Notice **ram:ram_inst** is highlighted in the **Netlist Navigator** tab on the left side of the RTL Viewer.

____ 3. Double-click the **ram** sub-design block.

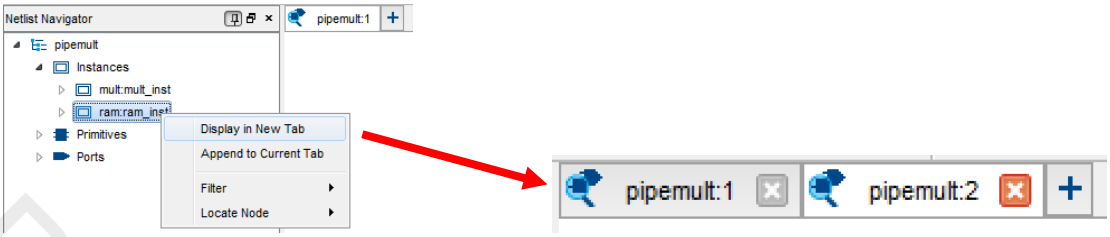


*You have now descended the hierarchy into the **ram** sub-design. As a result, the view changes to the above image. (Note: Your image may be slightly different than the above, with the hardwired ports not shown.) This shows that the **ram** sub-design is made up of a single IP block called **altsyncram**. You could continue double-clicking blocks to descend the hierarchy to its lowest level: single-bit RAM functions. But let's view this lowest level of the hierarchy in a different way.*

____ 4. With the outer **ram:ram_inst** block still highlighted in red, right-click on any empty area and select **Expand to Upper Hierarchy**.

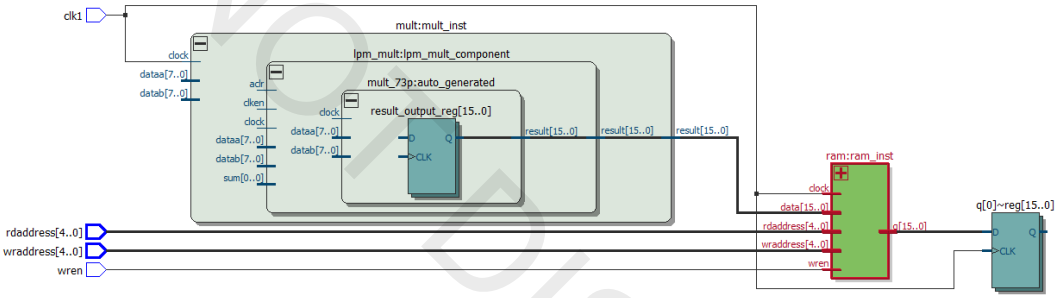
*This returns the Viewer to the next hierarchical level (the top in this case). If you've descended further into the hierarchy, you may need to do this a few times to return to the top. To ascend the hierarchy this way, the outer block needs to be highlighted. You can also just double-click on the top-level module **pipemult** in the **Netlist Navigator**.*

____ 5. Right-click the instance **ram:ram_inst** in the **Netlist Navigator** and select **Display in New Tab**.



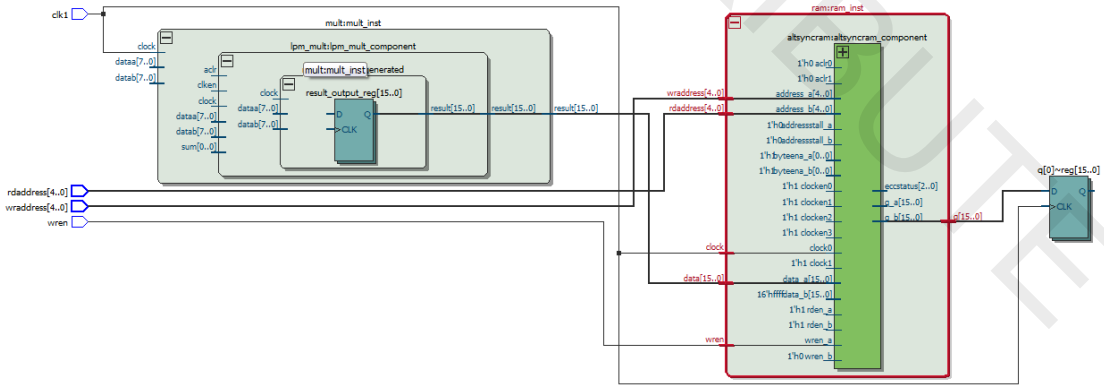
*A second tab named **pipemult:2** opens in the **RTL Viewer**. Displaying information in a new tab allows you to have multiple views open so you can compare and correlate what you see.*

____ 6. Right-click **ram:ram_inst** and select **Filter → Sources and Destinations**.



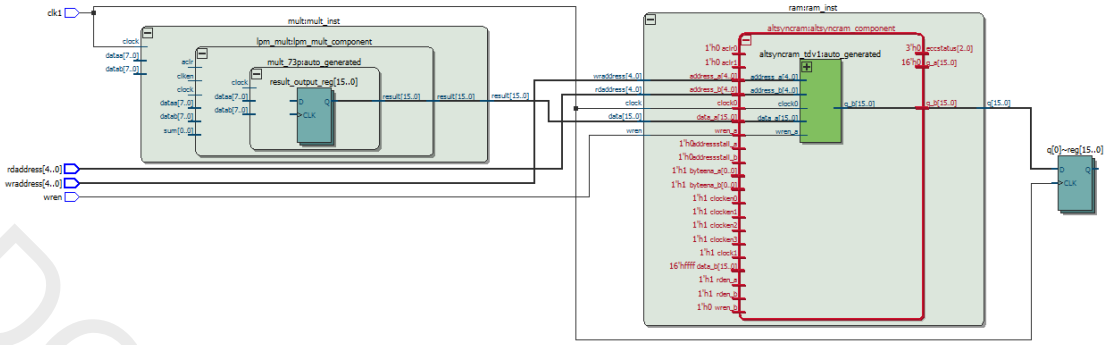
*The display is now restricted to the logic driving and driven by the **ram** block. This can help greatly in understanding the connections in a design, even ones you may not know are there. The shaded area around a block indicates a hierarchical boundary.*

____ 7. Click the "unfold"  symbol on **ram:ram_inst**. (Use the **View** menu → **Fit in Window** option or the  toolbar button to see the entire diagram.)



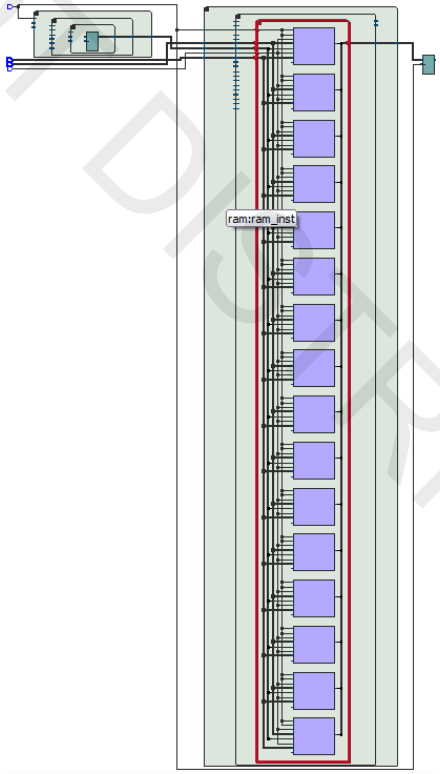
*Instead of displaying the lower hierarchical level alone (like double-clicking on a block), the **unfold** feature displays the lower level within the current hierarchical level (may appear slightly different than the above depending on design entry method). The **altsyncram** module is now visible.*

____ 8. Unfold the **altsyncram** module.



*Descending the hierarchy again, the internal logic of the **altsyncram** module (**altsyncram_???**) is displayed including any module I/O that are not connected. The ??? denote random characters the tool adds to differentiate the ram blocks.*

____ 9. Unfold the **altsyncram_???** module. Zoom out with the magnifying glass tool (right-click) or use the **Fit in Window** zoom to see more of the schematic, if necessary.



*Moving down into the **altsyncram_???** module, the viewer indicates that this module is represented by 16 smaller RAM functions, one function for each input/output data bit. Remember, this is a FUNCTIONAL representation of the design and may NOT represent how the design will ultimately be implemented in the target **Cyclone V GX** device.*

Step 4: Explore the ram and mult sub-designs physically using the Technology Map Viewer

The Technology Map Viewer allows you to see how a design is actually implemented using FPGA/CPLD resources. Use this tool as an aid during constraining and debugging to see changes in resource usage as settings and options are adjusted.

____ 1. Cross-probe the **ram** block from the RTL Viewer into the Technology Viewer by doing the following:

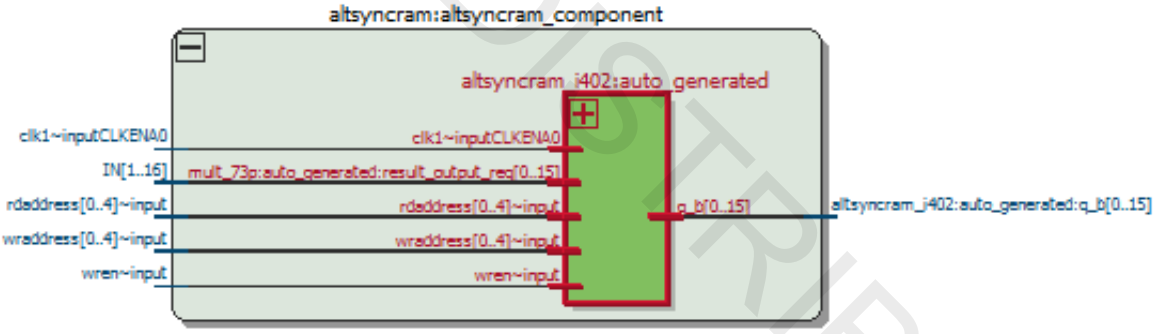
a. In the RTL Viewer, switch back to the standard arrow cursor if you switched to the magnifying glass to zoom in or out.

b. **Fold** the innermost hierarchical ring for the ram function (**altsyncram_???**) by clicking the ⊟ button.

c. Click the next higher hierarchical level named **altsyncram:altsyncram_component** to select it.

d. Right-click the selected hierarchy, go to the **Locate Node** submenu and select **Locate in Technology Map Viewer**.

e. Unfold the **altsyncram:altsyncram_component** block.



*The Technology Map Viewer opens and displays the **altsyncram_???** module inside the **altsyncram:altsyncram_component** in a tab named **pipemult:2**. The tab named **pipemult:1** contains the top-level **ram:ram_inst**. You can explore the rest of the design in a similar fashion to the RTL Viewer.*

____ 2. Unfold the **altsyncram_???** module.

*Now, instead of showing the 16 functional memory blocks as shown in the **RTL Viewer**, the **Technology Map Viewer** displays the ACTUAL device resource that was used, a single M10K memory block resource (**ram_block1a0**). You can verify this through the resource's properties.*

____ 3. Right-click **ram_block1a0** and select **Properties**.

*In the Properties tab, you can look at a schematic view of the block resource and how it was configured for this design, including signal connections (**Fan-in** and **Fan-out** tabs) and parameter settings (**Parameters** tab).*

____ 4. Fold up the **altsyncram_???** module.

____ 5. Switch from the Properties tab to the Netlist Navigator tab, and double-click **pipemult** at the top of the tab to see the top-level design in the Technology Map Viewer.

Compare this view to the RTL Viewer. All parts of the design are now represented by actual hardware resources in the device instead of the functional schematic view seen in the RTL Viewer.

____ 6. Unfold the **mult:mult_inst** block until it cannot be unfolded anymore.

The multiplier function has been placed in a single DSP resource.

____ 7. Select the DSP resource, **Mult0~mac**, and examine the properties of the DSP block resource. If you closed the Properties tab, right-click **Mult0~mac** and select **Properties** to reopen it.

The Properties tab displays a schematic representation of the resource along with IP parameter settings and input and output port information, just like the memory block resource. If you'd like, try going back to the RTL Viewer and following the same steps. This comparison should help you to better understand the ways the different viewers present information about the design at each stage of compilation.

Step 5: Use the Chip Planner to view connections to the ram sub-design

*The **Chip Planner** will give you a sense of where logic has been placed in the design. This can be very helpful when trying to understand design performance, as proximity is the key to performance in most newer FPGAs and CPLDs. Though the **Chip Planner** can be used for manually locating and moving logic, we only want to use it to evaluate results.*

____ 1. In the **Technology Map Viewer**, select the RAM block **altsyncram_???** to highlight it, if it is not already highlighted. Right-click on the block.

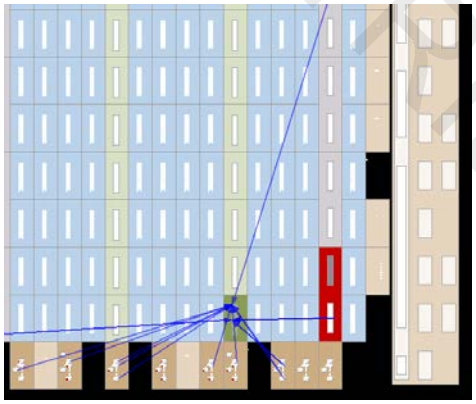
____ 2. From the **Locate Node** submenu, select **Locate in Chip Planner**.

*You should now see the **ram** sub-design zoomed in in the Chip Planner. You can use the zoom cursor again to zoom out (right-click) to see where in the Cyclone V device it was placed by the Fitter. Feel free to explore the view of the device.*

*If you click elsewhere and deselect the memory block, you can find and highlight it again by going to the **Locate History** tab at the bottom of the Chip Planner. Expand the **Located 1 nodes** item and click or double-click the **ram_block1a0** item. Clicking it highlights the RAM block in blue, while double-clicking highlights the block and zooms into it.*

If you further explore the Chip Planner Chip View, you should see that the red shaded area in the Chip Planner is the DSP block.

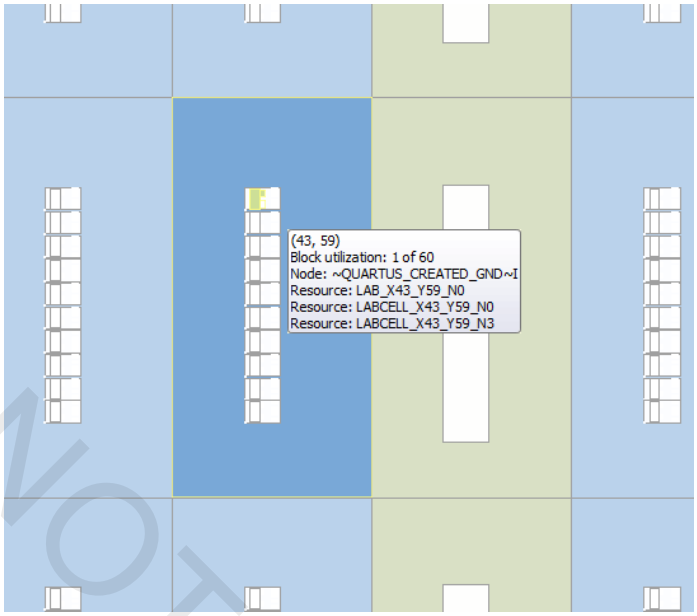
____ 3. With the memory block highlighted in the Chip Planner, click the **Generate Fan-In Connections** toolbar button. As mentioned above, if you clicked on anything else so that the memory block is no longer highlighted, you can re-highlight the memory block using the **Locate History** tab.



*The Chip Planner now displays the signal connections that are feeding into the **ram** sub-design (coming from device I/O and the multiplier). You can use a similar method to see the fan-out.*

____ 4. Turn off the fan-in/fan-out by first clicking on any non-highlighted part of the device. Then click the **Clear Unselected Connections** toolbar button .

____ 5. Zoom out and look around the Chip View to find the single ALM being used by the design. It will be the only LAB with a darker blue color.



____ 6. Zoom in and hover over the colored logic element (LE) to display a tooltip that indicates what it is being used for.

*If you want to explore this further, try selecting the resource and cross-probing (right-click **Locate Node** submenu) to the Technology Map Viewer.*

It appears that the ALM is simply being used as an electrical ground for the design.

____ 7. *(Optional; time permitting)* Experiment with the **Layers Settings** and the **Tasks** & **Reports** windows. Try to understand the differences between enabling/disabling a layers setting and creating a report overlay.

____ 8. When you are done exploring, close the RTL Viewer, Technology Map Viewer, and Chip Planner.

Exercise Summary

- Performed a full compilation
- Gathered information from the compilation report
- Cross-probed between windows to analyze design processing results in different ways using the RTL Viewer, Technology Map Viewer, & Chip Planner

END OF EXERCISE 3

Exercise 4

Exercise 4

Objectives:

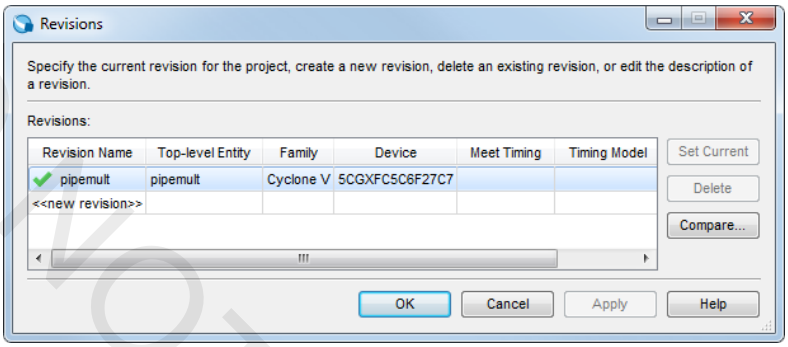
- *Create a new revision to store new constraint settings*
- *Make design constraints using the Assignment Editor*

Device Name	**5CGXFC5C6F27C7**
Total Design	
Logic utilization (in ALMs)	
Total registers	
Total pins	
Total block memory bits	
Total DSP Blocks	
mult sub-design	
ALMs needed (mult)	
Combinational ALUTs (mult)	
Dedicated Logic Registers (mult)	
DSP Blocks (mult)	
ram sub-design	
ALMs needed (ram)	
Combinational ALUTs (ram)	
Dedicated Logic Registers (ram)	
Block memory bits (ram)	
M10Ks (ram)	
Control Signals	
Name	**Fanout**

Step 1: Create a new revision to store constraint changes

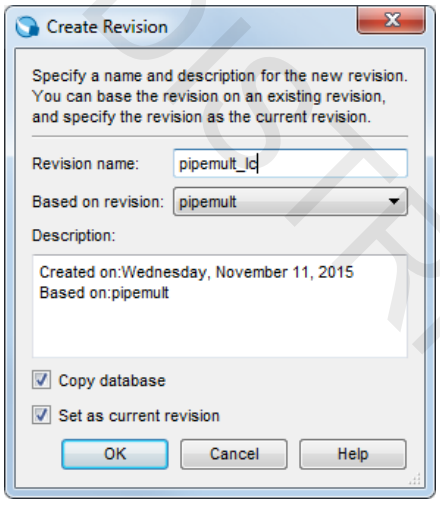
*In order to experiment with different settings or assignments and to see how these changes affect your results, the Quartus Prime software has support for creating revisions, with each revision building a new Quartus Settings File (**.qsf**). You can also compare the compilation results of your various revisions.*

____ 1. From the **Project** menu, select **Revisions**.



____ 2. In the **Revisions** dialog box, double-click **<<new revision>>**.

____ 3. In the **Create Revision** dialog box, set the **Revision name** to **pipemult_lc**.



*Notice the revision is automatically based on the original revision **pipemult**. You could use the drop-down to base it on a blank revision (like starting a new project).*

____ 4. Leave all other settings at their defaults and click **OK.**

____ 5. Click **OK** to close the **Revisions** dialog box if it remains open.

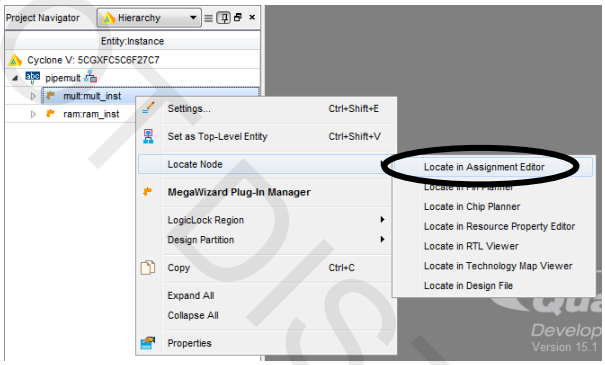
Step 2: Implement the multiplier in logic elements instead of embedded multipliers

Cyclone V GX DSP blocks are a valuable resource for implementing multiply operations. They provide a better usage of resources for multiplication compared to using ALMs. However, DSP blocks are a limited resource compared to the number of ALMs available. DSP blocks also have dedicated locations whereas ALMs are available all throughout the device. If your design has many multipliers, it may be advantageous to implement smaller or non-speed critical multipliers in ALMs (or even memory blocks) instead. This can be done using an option in the MegaWizard Plug-In Manager or on a multiplier-by-multiplier basis using a logic option in the Assignment Editor.

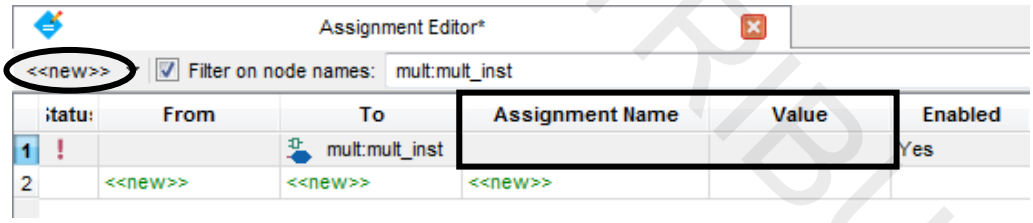
____ 1. In the Project Navigator Hierarchy, expand the **pipemult** hierarchy.

____ 2. Right-click on the **mult** entity in the hierarchy.

____ 3. From the **Locate Node** submenu, select **Locate in Assignment Editor**.



____ 4. Click the **<<new>>** button in the top left hand corner of the Assignment Editor.



*The name **mult:mult_inst** gets automatically entered into the **To** field.*

____ 5. Double-click the cell in the **Assignment Name** column and select **DSP Block Balancing** from the list.

Tip: By quickly typing "DSP", you may find the option faster than scrolling.

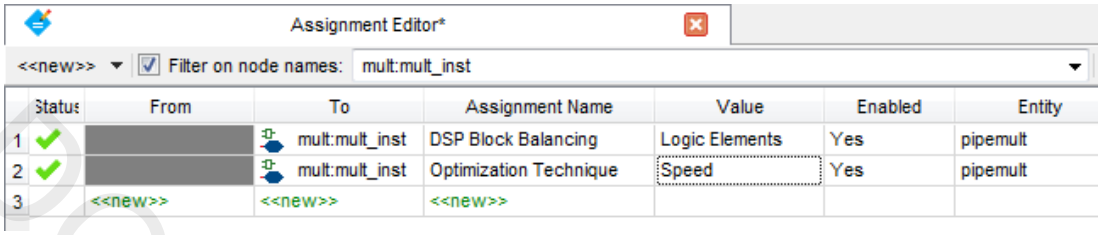
____ 6. Hit the **Enter** key or click elsewhere to accept the assignment name.

____ 7. Double-click the cell in the **Value** column for the **DSP Block Balancing** assignment, and select **Logic Elements** from the drop-down menu.

____ 8. Hit the **Enter** key or click elsewhere to accept the assignment value.

____ 9. Click the **<<new>>** button in the upper left again, and set the Assignment Name to **Optimization Technique**.

____ 10. Double-click the cell in the **Value** column for the **Optimization Technique** assignment and select **Speed**. The Assignment Editor should look like this:



____ 11. From the **File** menu, **Save** the Assignment Editor.

____ 12. Click to compile the design.

Step 3: Gather resource information from the Compilation Report

____ 1. Open the **Compilation Report** from the **Processing** menu if it's not already open.

____ 2. From the **Flow Summary** section of the **Compilation Report**, again record the **Logic utilization (in ALMs)**, **Total registers**, **Total pins**, **Total block memory bits**, and **Total DSP Blocks** in the table at the beginning of this exercise.

____ 3. In the **Fitter** folder of the **Compilation Report**, do the following:

a. Expand the **Resource Section** folder.

b. From the **Resource Utilization by Entity** report, record in the exercise table the resource counts for the **mult** and **ram** sub-designs.

c. From the **Control Signals** table, also in the **Resource Section**, record the fan-out for control signals **clk1** and **wren**.

How can half (0.5) of an ALM be used? ALMs can be split in two to create separate logic functions. The Compilation Report is indicating that some logic in the design only requires half of the logic available in an ALM.

From the results, you can see that in this revision, the multiplier implementation was moved from the embedded multipliers into the logic array blocks.

*Feel free to open up the tools used in the prior exercise (RTL Viewer, Technology Map Viewer, Chip Planner) to examine the results of this compilation. The unfolded **mult** block should look significantly different in both the RTL Viewer and the Technology Map Viewer.*

Exercise Summary

- Created a new revision to evaluate different constraints
- Controlled logic options (constraints) using the Assignment Editor

END OF EXERCISE 4

Exercise 5

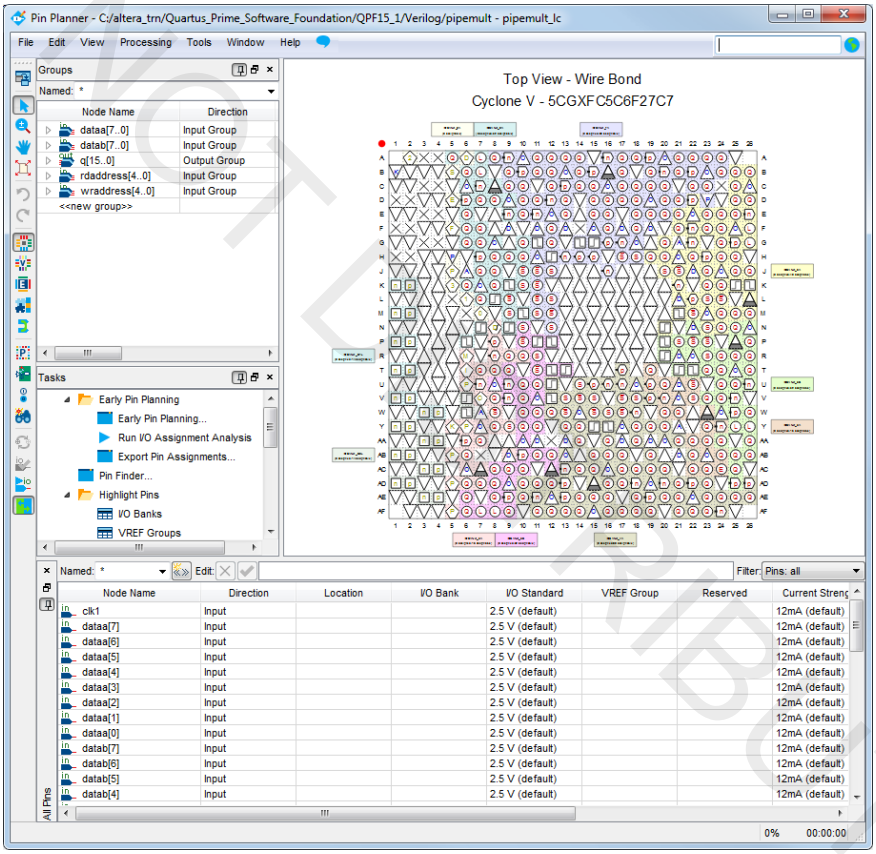
Exercise 5

Objectives:

- *Assign I/O pins and perform I/O Assignment Analysis*
- *Back-annotate pin assignments to lock placement*
- *Use the Pin Migration View to see the effect of device migration on I/O assignments*

Step 1: Use Pin Planner to assign I/O pins & set I/O standards

____ 1. From the **Assignments** menu or the **Assign Constraints** folder of the **Tasks** window, open the **Pin Planner**.



Remember, if not already detached, you can detach the Pin Planner from the main Quartus Prime application window to make it easier to work with. You can also undock the Groups and All Pins lists.

____ 2. Make sure the **Top View** of the device is displayed, indicated by the red dot in the upper left hand corner of the package. If not, select **Package Top** from the **View** menu.

____ 3. Open up the Pin Legend and experiment with different views using the options in the **View** menu. Also try generating Pin Planner reports using the **Highlight Pins** tasks in the **Tasks** window.

Try to understand the differences between how information is presented using the views vs. the Pin Planner reports.

____ 4. Turn off any reports you may have created in the previous step by disabling all the reports in the **Report** window, or right-clicking in the **Report** window and selecting **Delete All**.

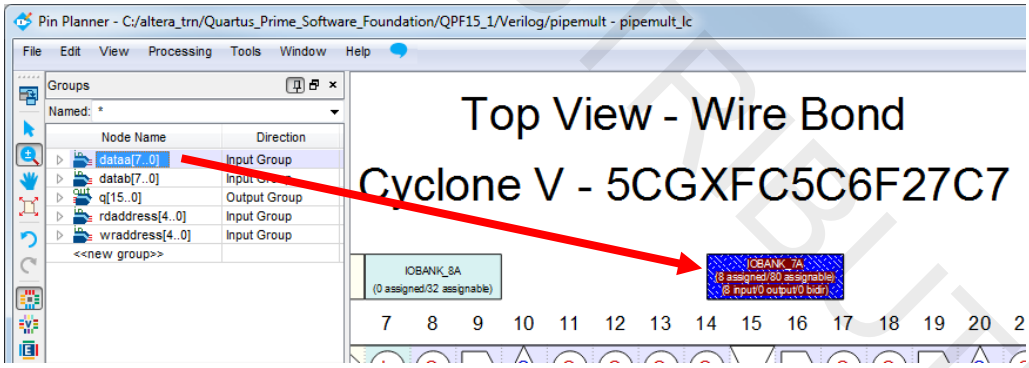
____ 5. In the **Pin Planner** toolbar, make sure that the **Show I/O Banks** toolbar button is selected. This is the original default view in the Pin Planner. If you are unsure of which button this is, you can also find this option in the **View** menu.

____ 6. In the **Groups List** window of the **Pin Planner**, locate the **dataa** input bus, and click it once to select it.

*Tip: You may notice, if you left either the Technology Map Viewer or the Chip Planner open from the previous exercises, that the **dataa** bus is automatically selected and highlighted in those other tools when you select it here in the Pin Planner. The Chip Planner even does a highlighting animation to show you exactly where the selected I/O pins have been placed by the Fitter. This is even faster than using cross-probing!*

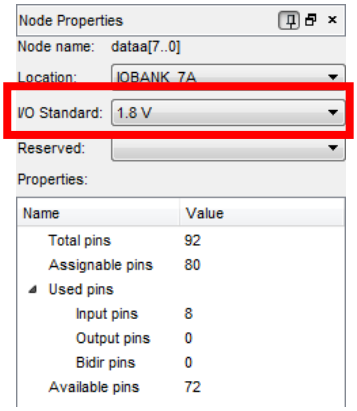
____ 7. Still in the Groups list, right-click the **dataa** bus and select **Node Properties**.

____ 8. In the Node Properties dialog box (located to the right of the Package View by default), from the **Location** menu, select **IOBANK_7A**.



The display for IOBANK_7A will now change to indicate that the bank has 8 assigned pins out of 80 total assignable pins. Assigning signals to an I/O bank like this (instead of to individual pins) gives the Fitter the freedom to place the signals on any pins within the specified I/O bank.

____ 9. Back in the **Node Properties** dialog box, set the **I/O standard** for **dataa** to **1.8 V**.



*You could have assigned the location and changed the I/O standard using the All Pins list instead of the Groups list. However, be aware that **individual** assignments made in the All Pins list (or on individual signals within groups in the Groups list) take precedence over **group** assignments made in the Groups list. The best way to ensure that all the signals in an entire bus or signal group inherit an assignment like this is to make the assignment to the whole group in the Groups list.*

____ 10. Using the Groups list with the Node Properties window or with drag and drop, assign the **datab** bus to **IOBANK_7A**. *Do not change the I/O standard for this bus yet!*

____ 11. Click in any empty space in the Package View to de-select **datab**.

Using drag and drop for location assignments to I/O banks selects the I/O bank the group is dragged to. This can affect subsequent assignments made in the Node Properties window. De-selecting pins assigned to I/O banks prevents this from happening.

____ 12. Assign the **q** output bus to **IOBANK_8A** (located at the top left of the package) and set the bus's **I/O standard** to **1.8 V**. You can do this in either the Node Properties or by scrolling right in the Groups list and double-clicking the cell in the **I/O Standard** column. De-select the **q** bus when finished.

____ 13. Assign both the **rdaddress** and **wraddress** buses to **IOBANK_8A** and set their I/O standard to **1.8 V**, de-selecting each bus after making assignments for each.

____ 14. Using the **All Pins** list, the Node Properties window, or drag and drop, assign **clk1** to pin **L8** (pin has a clock edge ⎍ symbol) and set its I/O standard to **1.8 V**.

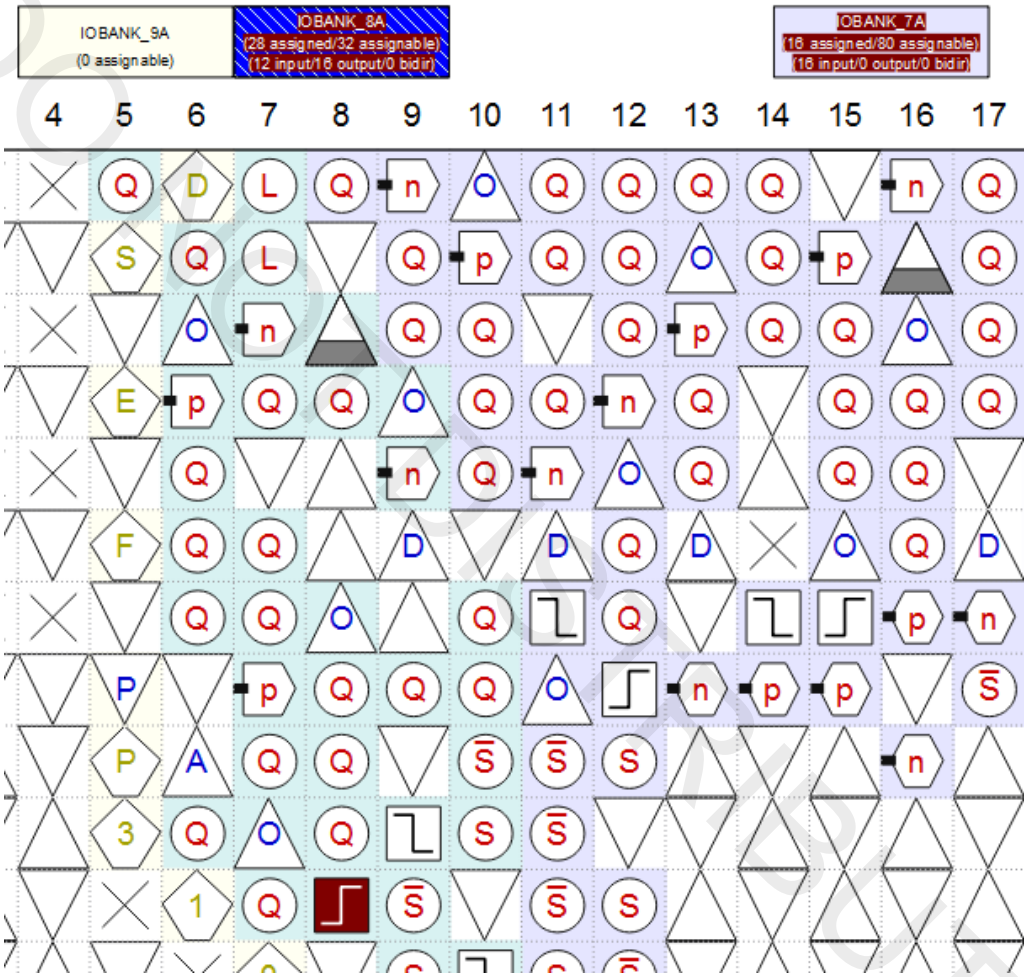
____ 15. Using any method, assign **wren** to **IOBANK_8A** and set its **I/O standard** to **1.8 V**.

Groups

Named: *

Node Name	Direction	Location	I/O Bank	I/O Standard
dataa[7..0]	Input Group	IOBANK_7A	7A	1.8 V
datab[7..0]	Input Group	IOBANK_7A	7A	2.5 V (default)
q[15..0]	Output Group	IOBANK_8A	8A	1.8 V
rdaddress[4..0]	Input Group	IOBANK_8A	8A	1.8 V
wraddress[4..0]	Input Group	IOBANK_8A	8A	1.8 V
<<new group>>				

The Package View of the Pin Planner should look like the following screenshot:



Step 2: Analyze I/O assignments

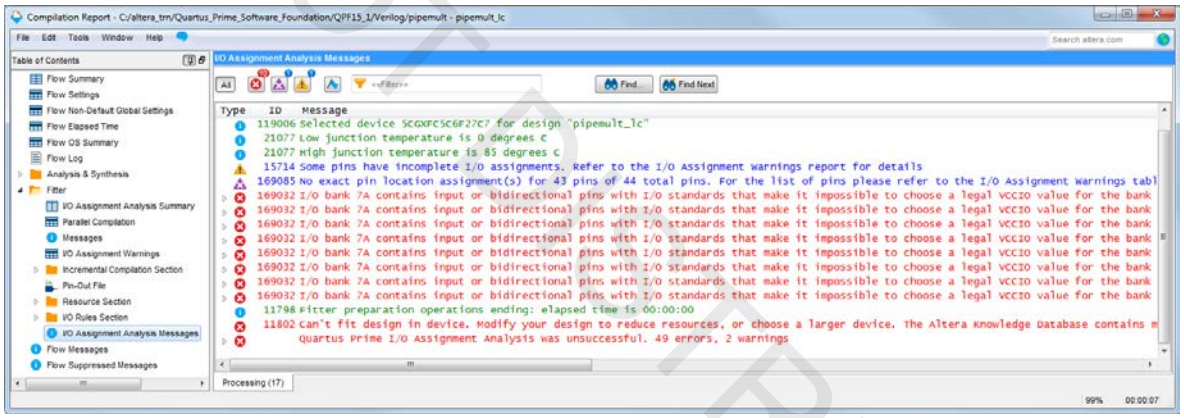
Now that you have made general I/O assignments, you can check the validity of those assignments without running a full compilation using I/O Assignment Analysis. This way you can quickly and easily find I/O placement and assignment issues and correct them.

____ 1. Back in the main Quartus Prime window, synthesize the design by going to the **Processing** menu, go to the **Start** submenu, and select **Start Analysis & Synthesis** or click the button in the toolbar.

____ 2. From the **Processing** menu, go to the **Start** submenu, and select **Start I/O Assignment Analysis** or click in the Pin Planner toolbar.

____ 3. Bring the **Compilation Report** to the foreground. If it's not open, click in the Quartus Prime toolbar or choose **Compilation Report** from the **Processing** menu.

____ 4. Expand the **Fitter** folder and click on **I/O Assignment Analysis Messages**. You can also look at the Processing tab in the Messages window.



Was the analysis successful? Check the messages. I/O Assignment Analysis has found errors with the I/O assignments.

____ 5. Review the **I/O Assignment Analysis Messages** and determine the cause of the error. Expand the error messages to get more detail as to why each pin of the **dataa** bus is not being placed successfully.

*Determining the cause of the I/O placement failure requires reading the error messages carefully and having a little understanding of **Cyclone V GX** devices or general FPGA I/O blocks. See if you can understand and correct the cause of the errors on your own. If you do not have a lot of **Cyclone V GX** device or general FPGA experience, the next steps will show you how to correct the errors.*

____ 6. Bring the **Pin Planner** to the foreground.

____ 7. Scroll to the right in the Groups list to see the **I/O Standard** column. If the **I/O Standard** column does not appear, right-click in the Groups list and select **Customize Columns** to make the column visible.

*Notice that you have assigned the **dataa** and **datab** input buses to **I/O Bank 7A** but set different **VCCIO (1.8 & 2.5)** voltage levels for them. Cyclone V GX FPGAs, like all Altera FPGA devices, allow for only one V_{CCIO} per I/O bank.*

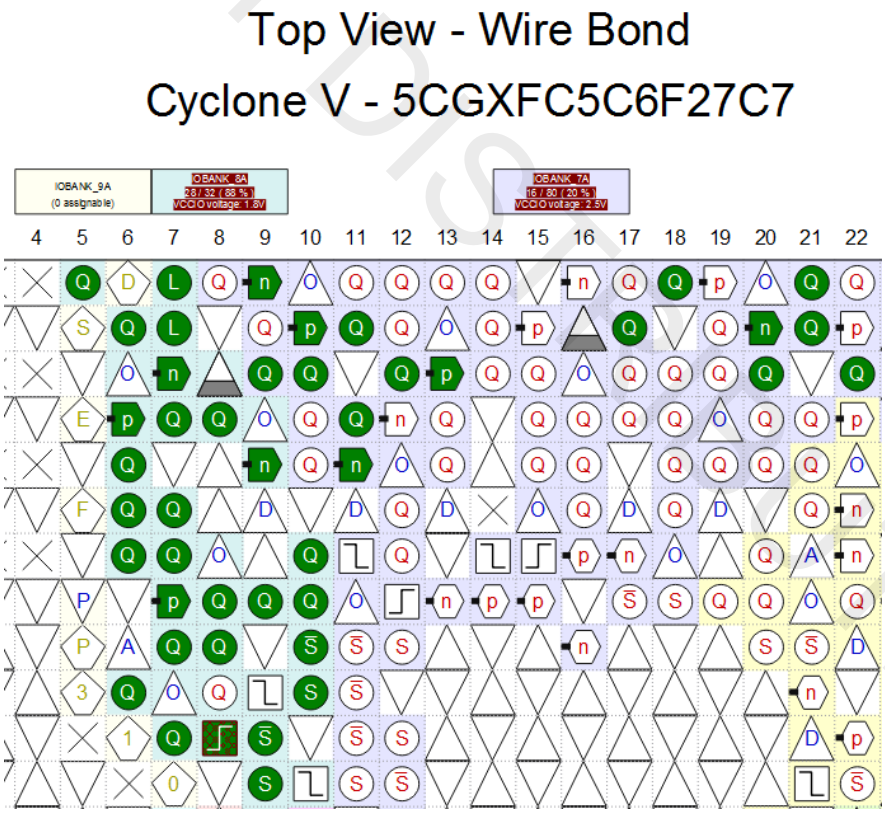
____ 8. Using the Groups List, change the I/O standard assignment for the **dataa** group to **2.5 V** (or **2.5 V (default)**).

____ 9. Re-run **I/O Assignment Analysis**.

See how quickly and easily you can check your I/O placement assignments without running a full compilation!

Now let's take a look at the I/O location assignments selected by the Fitter.

____ 10. In the Pin Planner, click the button in the toolbar or, from the **View** menu, go to the **Show** submenu and select **Show Fitter Placements**.

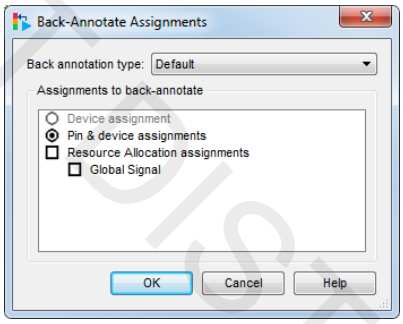


*You can see that Fitter-selected pin locations appear in green in the Pin Planner. Performing I/O Assignment Analysis partially ran the Fitter, and by adding I/O bank location assignments, you constrained the signals to be placed in the selected I/O banks. Without these assignments, the Fitter could have placed the signals anywhere on the chip. Notice that **clk1**, the only signal you manually assigned to a specific pin (L8), appears green with a brown hatched pattern. Remember that this indicates that your user assignment overlaps the location assignment selected by the Fitter.*

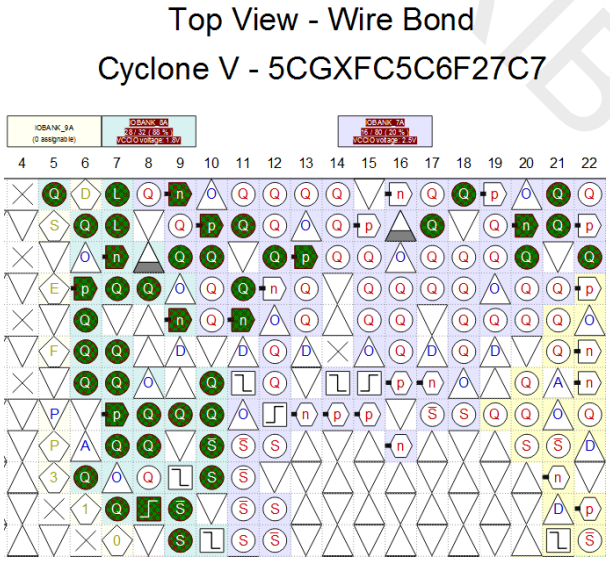
Step 3: Back-annotate pin assignments to lock placement

At this point in the I/O design flow, you may decide that you like the I/O location assignments made by the Fitter, and you've verified the pin-out to begin board design. Now you need to make sure that the pin locations are not moved during successive compilations.

____ 1. From the **Assignments** menu in the main Quartus Prime window, select **Back-Annotate Assignments** to open the **Back-Annotate Assignments** dialog box. The dialog box may appear slightly different from the screenshot below.



____ 2. In **Assignments to back-annotate**, enable **Pin & device assignments** (default setting) as shown above. Click **OK**.



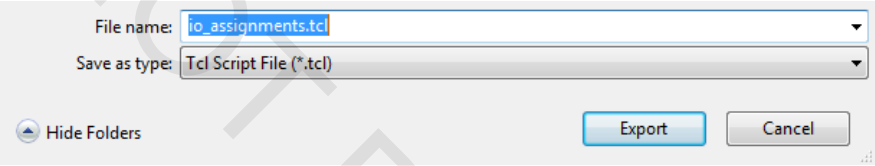
*Notice how all the I/O pins that were green earlier have changed to the green and brown hatch pattern. This indicates that the locations that were Fitter-assigned I/O are now user-assigned I/O and have been written into the **.qsf** file as constraints.*

Step 4: Transfer pin assignments to original revision

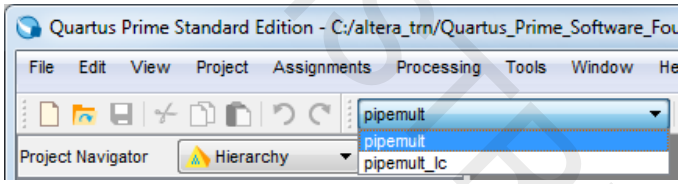
Now you should carry these pin assignments from the current design revision to the original revision (in case you decide to choose that revision later). Remember that all assignments, including I/O-related assignments, are unique to a particular project revision, so the assignments must be copied over if you want them to match between the two revisions. To do this, you will export the assignments as a Tcl script and import them into the original pipemult revision by running the script.

____ 1. With the **Pin Planner** active, go to the **File** menu and select **Export**.

____ 2. In the **Export** dialog box, choose Save as type **Tcl Script File**. Set the **File name** to **io_assignments.tcl** and click **Export**.

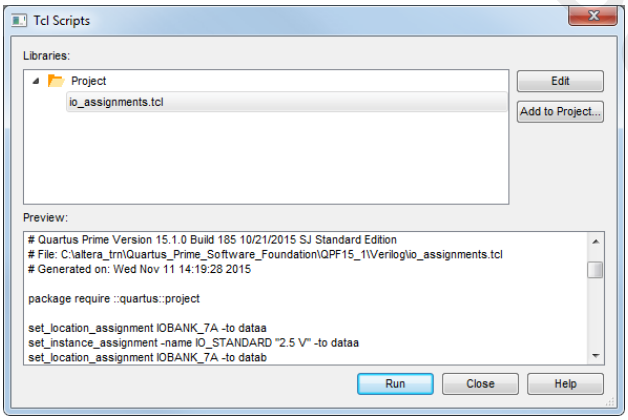


____ 3. Go back to the main Quartus Prime window. From the drop-down menu at the top of the window, change the revision back to **pipemult**.



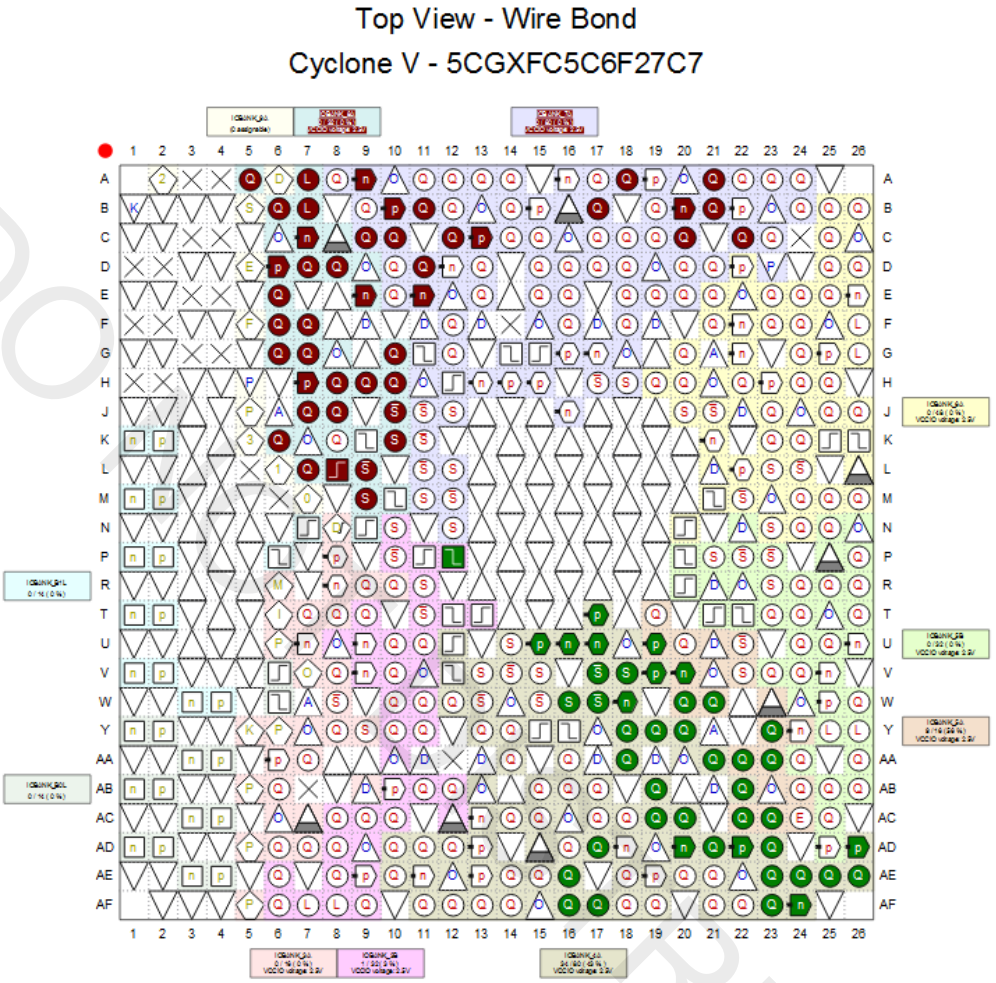
____ 4. From the **Tools** menu, select **Tcl Scripts…**

____ 5. Highlight the **io_assignments.tcl** file in the Project folder. Click **Run**.



____ 6. Click **OK** when complete and then click **Close**.

____ 7. Reopen the **Pin Planner** to see that the assignments were imported correctly. If **Show Fitter Placements** is turned off, turn it back on.



*Remember that the green pins are the Fitter-placed locations from the previous compilation of the revision **pipemult**. The reddish brown pins are the pins imported from the **pipemult_lc** revision, and the green and red hatched pins (if any; depends on how the Fitter placed the I/O in the earlier compilation) are intersecting assignments (think of the intersecting area of a Venn diagram) between the two revisions of the project. The green locations would all be removed if this revision were compiled again since the user-assigned I/O pins (in reddish brown) have precedence.*

____ 8. Turn off the viewing of Fitter placements using the toolbar button or by turning off the option in the **Show** submenu of the **View** menu.

*You can now easily see that the locations match the back-annotated locations from the **pipemult_lc** revision.*

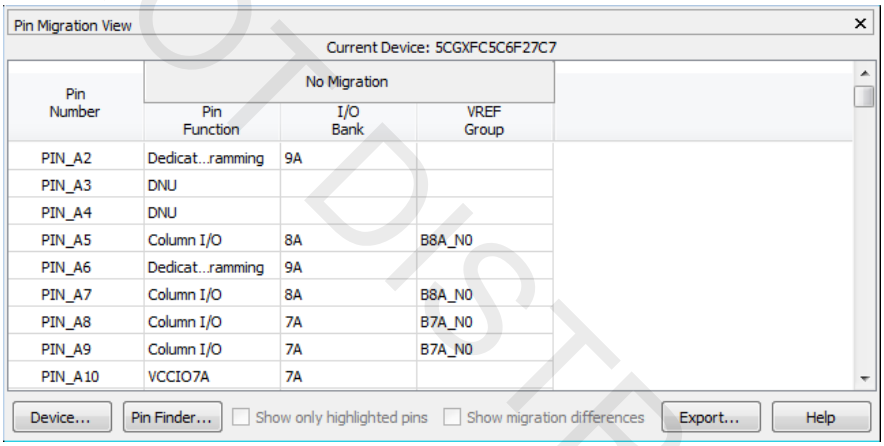
Step 5: Add migration devices and validate cross-migration I/O

Now let's imagine that, in addressing feature requests for your design, you are concerned that it may grow beyond your target device. You may need to select a larger FPGA while still using the same package with the same pinout. To confirm your pinout is transferable, you will use the device migration feature of the Quartus Prime software. You can add migration devices to a project at any point if you expect to later move the design to another device. The Pin Planner Pin Migration View keeps track of possible issues you may have with your I/O assignments during a migration.

____ 1. Create a new revision of the project called **pipemult_migration**, based off of the original **pipemult** revision. Switch to the new revision in either the Revisions dialog box or the menu in the toolbar.

____ 2. Reopen the Pin Planner.

____ 3. From the **View** menu (or by right-clicking in the Package view), select **Pin Migration Window**.

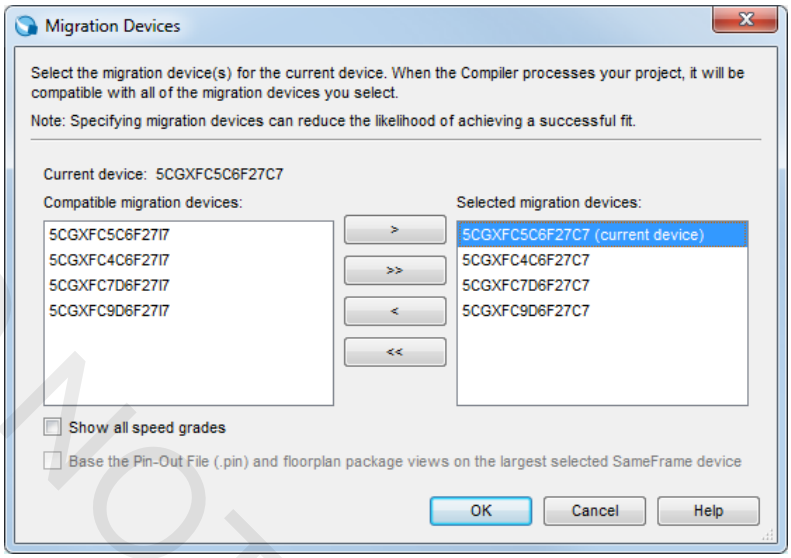


The Pin Migration View appears. It only displays pin information for the current device because you have not selected any migration devices yet.

____ 4. Click **Device** to open the Device settings dialog box.

____ 5. Click **Migration Devices**.

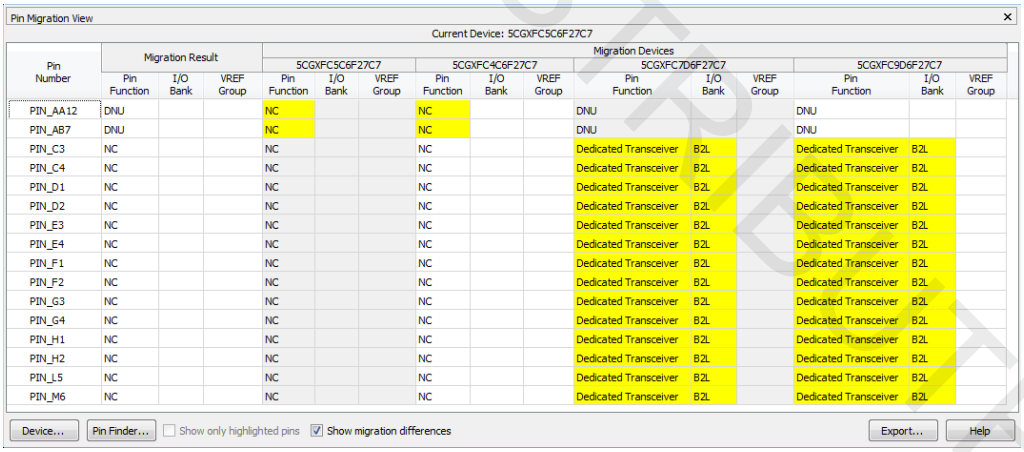
____ 6. Control-click the three **C7** (commercial; speed grade 7) devices and click > to move them to the list of migration devices. You can also double-click each device individually.



____ 7. Click **OK**.

____ 8. Click **OK** again to close the **Device** settings.

____ 9. In the Pin Migration View window, you can now see pin information about the migration devices. Turn on the **Show migration differences** option at the bottom of the window.



This option highlights the pin differences between the devices and how they will resolve during migration. Highlighting any line in the Pin Migration View will highlight the pin in the Package View so you can quickly cross-reference any non-migratible pin assignments visually. Looking at the Package View of the Pin Planner, you should see a number of pins marked as No-Connects (NC) with 'X'.

For final confirmation of "migratability," use I/O Assignment Analysis.

____ 10. Close the Pin Migration View.

____ 11. From the Quartus Prime **Processing** menu, go to **Start** and select **Start I/O Assignment Analysis** or click  in the Pin Planner toolbar.

I/O Assignment Analysis has confirmed that your current pin-out is valid for all possible target devices. If there was a failure, then you could simply look at the Package View, which now displays only "migratable" locations, to see where the conflicts exist. You could then choose a new pin location for the offending I/O.

You would now perform a full compilation of the completely constrained design. Once the compilation completes, you would be able to perform static timing analysis to verify all timing requirements are met and then program the FPGA device using the bitstream file generated by the Quartus Prime Assembler.

Exercise Summary

- Assigned pin locations using the Pin Planner
- Back-annotated pin locations from prior compilation
- Verified I/O placement and constraints using I/O assignment analysis
- Used the Pin Migration View to see the affect of device migration on an I/O assignment

END OF EXERCISE 5

Exercise 3

Device Name	5CGXFC5C6F27C7
Total Design	
Logic utilization (in ALMs)	*1*
Total registers	*16*
Total pins	*44*
Total block memory bits	*512*
Total DSP Blocks	*1*
mult sub-design	
ALMs needed (mult)	*0*
Combinational ALUTs (mult)	*0*
Dedicated Logic Registers (mult)	*0*
Block memory bits (mult)	*0*
DSP Blocks (mult)	*1*
ram sub-design	
ALMs needed (ram)	*0*
Combinational ALUTs (ram)	*0*
Dedicated Logic Registers (ram)	*0*
Block memory bits (ram)	*512*
M10Ks (ram)	*1*
Control Signals	
Name	**Fanout**
clk1	*18*
ram clock enable (wren)	*1*

Exercise 4

Device Name	5CGXFC5C6F27C7
Total Design	
Logic utilization (in ALMs)	*30*
Total registers	*68*
Total pins	*44*
Total block memory bits	*512*
Total DSP Blocks	*0*
mult sub-design	
ALMs needed (mult)	*29.5*
Combinational ALUTs (mult)	*59*
Dedicated Logic Registers (mult)	*52*
Block memory bits (mult)	*0*
DSP Blocks (mult)	*0*
ram sub-design	
ALMs needed (ram)	*0*
Combinational ALUTs (ram)	*0*
Dedicated Logic Registers (ram)	*0*
Block memory bits (ram)	*512*
M10Ks	*1*
Control Signals	
Name	**Fanout**
clk1	*69*
ram clock enable (wren)	*1*